Survey: On the Landscape of Graph Databases

Miguel E. Coimbra[1]
INESC-ID, [R. Alves Redol 9, 1000-029 Lisboa], Portugal
IST, [Av. Rovisco Pais 1, 1049-001 Lisboa], Portugal

Lucie Svitáková
Faculty of Information Technology, CTU in Prague, [Thákurova 9, 160 00, Prague], Czech Republic

Domagoj Vrgoč
Department of Computer Science (DVRG), Pontificia Universidad Católica de Chile, [Av. Vicuña Mackenna 4860, Edificio Hernán Briones, 2do piso, 7820436, Macul, Santiago], Chile

Alexandre P. Francisco
INESC-ID, [R. Alves Redol 9, 1000-029 Lisboa], Portugal
IST, [Av. Rovisco Pais 1, 1049-001 Lisboa], Portugal

Luís Veiga
INESC-ID, [R. Alves Redol 9, 1000-029 Lisboa], Portugal
IST, [Av. Rovisco Pais 1, 1049-001 Lisboa], Portugal

Abstract

Graph databases have become essential tools for managing complex and interconnected data, which is common in areas like social networks, bioinformatics, and recommendation systems. Unlike traditional relational databases, graph databases offer a more natural way to model and query intricate relationships, making them particularly effective for applications that demand flexibility and efficiency in handling interconnected data.

Despite their increasing use, graph databases face notable challenges. One significant issue is the irregular nature of graph data, often marked by structural sparsity, such as in its adjacency matrix representation, which can lead to inefficiencies in data read and write operations. Other obstacles include the high computational demands of traversal-based queries, especially within large-scale networks, and complexities in managing transactions in distributed graph environments. Additionally, the reliance on traditional centralized architectures limits the scalability of Online Transaction Processing (OLTP), creating bottlenecks due to contention, CPU overhead, and network bandwidth constraints.

The growing relevance of graph databases in handling large-scale datasets across diverse fields—such as healthcare, industry, artificial intelligence (AI), life sciences, social networks, and software engineering—drives the motivation for this study. Addressing the challenges they face requires efficient query languages and optimized storage solutions to improve their overall performance.

This paper presents a thorough survey of graph databases. It begins by examining property models, query languages, and storage architectures, outlining the foundational aspects that users and developers typically engage with. Following this, it provides a detailed analysis of recent advancements in graph database technologies, evaluating these in the context of key aspects such as architecture, deployment, usage, and development, which collectively define the capabilities of graph database solutions.

We conducted a comprehensive analysis of numerous graph database systems and surveys to shed light on the most critical intricacies within this ecosystem. Alongside this, we outlined a range of features to assist developers and researchers in identifying the specific requirements of their use cases and the technologies suited to their needs.

The next decade will likely reveal fascinating trends among commercial players and prioritized features. We anticipate that graph databases will continue to advance as the volume of data grows, particularly within the realm of AI.

[1] Corresponding author

2012 ACM Subject Classification Computer systems organization → Architectures; Information systems → Data management systems; Information systems → Query languages for non-relational engines; Information systems → Graph-based database models; Information systems → Data access methods; Information systems → Record and block layout; Information systems → Database management system engines; Computer systems organization → Cloud computing; Information systems → Database administration

Keywords and phrases graph databases, graph query languages, graph storage architecture, graph models and representations

Digital Object Identifier

Related Version This is the publication version of:
arXiv Version: `Survey: Graph Databases, arXiv` [43]

Funding This work was supported by national funds through Fundação para a Ciência e a Tecnologia, I.P. (FCT) under projects UID/50021/2025 (DOI: https://doi.org/10.54499/UID/50021/2025) and UID/PRR/50021/2025 (DOI: https://doi.org/10.54499/UID/PRR/50021/2025), and 2023.16986.ICDT. This work was supported by the CloudStars project, funded by the European Union's Horizon research and innovation program under grant agreement number 101086248.

1 Introduction

Graph databases have become increasingly vital in handling complex, interconnected data, prevalent in domains like social networks, bioinformatics, or recommendation systems [87, 196]. Unlike traditional relational databases, graph databases offer a more intuitive model for complex and interrelated data, making them particularly suited for applications that require efficient and flexible modeling and querying of complex relationships.

The motivation for this study lies in the growing relevance of graph databases in handling large-scale data with diverse use-cases in multiple domains (e.g. health [228], industry [213], artificial intelligence (AI) [157], life sciences [38], social networks [118], network analysis [188] and software engineering [138]), and the need for efficient query languages and storage solutions in this context.

The complexity of the graph database ecosystem has increased in the last decade, with the ecosystem flourishing with new commercial offerings often benefiting from community engagement. Choosing the appropriate solution requires an understanding of the knowledge on graph storage, graph representations, query languages and system architectures. Such knowledge is essential to properly guide the questions which will help the technological decision-makers of both industry and academia in identifying their use-cases.

Performance and lower processing costs are relevant, but so is the ability to control data on-premises, or the offering of quality documentation. An open-source community, with whose members it is possible to quickly iterate ideas and troubleshoot problems, can reduce technological friction and significantly speed up development tasks. We set out to clarify these and other aspects in this paper, and observe if and how they are considered by different graph database systems which are part of an exhaustive list we have identified from the ecosystem.

Despite the growing popularity and application of graph databases, several challenges persist. For example, the irregularity in graph data such as the sparseness of its structure (e.g., manifested in the concept of its adjacency matrix) may lead to inefficiencies in data reads and writes. Other challenges include the high cost of traversal-based queries, especially in large-scale networks,

and issues related to transaction processing in distributed graph environments. Moreover, the traditional centralized system architectures often limit the scalability of Online Transaction Processing (OLTP) in graph databases, presenting bottlenecks in terms of contention likelihood, CPU overheads, and network bandwidth.

This paper presents a comprehensive survey of graph databases, focusing initially on property models, query languages and storage architectures, to frame the main aspects that users and developers typically have to deal with. Following, we make an in-depth analysis of recent and relevant graph database works, according to the previous aspects, and also across a number of broader dimensions (or areas), each embodying classes of key features (or aspects) of graph database solutions' architecture, deployment, usage, and development. These include support, licensing, programming model, operating system, deployment, data representation, concurrency, performance, security, scalability, fault tolerance, or integration, to name a few.

The rest of the paper is organized as follows. In Section 2, we present relevant graph models and query languages that a potential user of a graph database might encounter because the models and languages play a decisive role in the selection of a graph database to match the consumer's use case and capabilities properly. Section 2.3 describes and shows the advantages of various storage architectures, from unstructured to advanced ones, applied in actual graph database solutions. We follow with introductions of each surveyed graph database in Section 3, opening with features that were evaluated for each one and classified into their respective categories. We also analyze and discuss the previously listed graph databases and their capabilities. Since the field of graph database research has already become rich, we recommend related work in the following Section 4, concluding our survey in the last Section 5.

2 Graph Models and Query Languages

This section introduces the most common graph models used in graph database systems. For these models, several graph query languages have been developed, which are presented afterward. These languages often serve not only for basic create-read-update-delete (CRUD) operations but also for sophisticated querying of the graphs.

2.1 Graph Data Models

Two most common graph data models in use today are *property graphs* and *edge-labeled graphs*. The former are widely adopted in commercial systems, while the latter form the backbone of many open-source solutions in the Semantic Web ecosystem. Several generalizations extending or mixing the two models have also been developed, but are not as popular. Next we further elaborate on each data model.

2.1.1 Property graphs

Nowadays, a widely accepted general model for graph data is the property graph (PG) model. The property graph model defines the organization of interrelated data as nodes (vertices), relationships (edges), and the properties of both types of elements [167]. Properties themselves are usually defined as key-value pairs and by default do not need to conform to a specific schema, allowing for fully flexible data representation. The model is also often associated with the adjectives "labeled" or "directed" since the nodes or edges often have assigned labels, and the relationships can be directed from a source node to a target node. More recent incarnations also support undirected edges [55]. Whether just one label per node/edge is required (so-called "type"), the assignment of

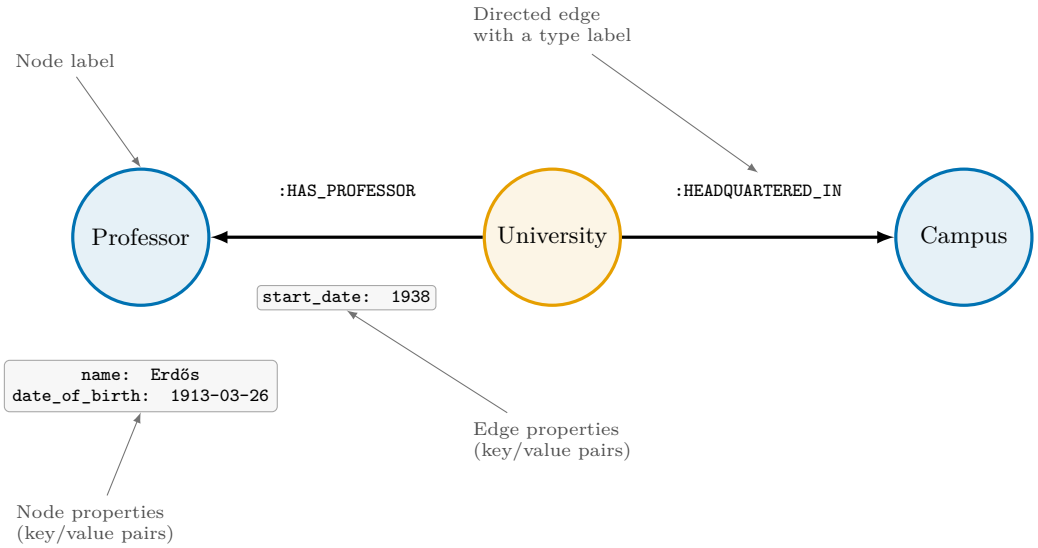

Figure 1 Illustration of the property graph model, inspired by the memory of Professor Paul Erdős.

any number or at least one of the labels is optional, or the labeling ability is even provided, varies from one particular graph database management system to another. Instances of this model are usually found coupled to the design of graph query languages [192, 222, 10, 85]. Figure 1 depicts the properties that may be associated with vertices and edges in the context of the property graph model.

2.1.2 Directed edge-labeled graphs (RDF)

The second widely used graph model is RDF - Resource Description Framework [1]. Its central concept is a triple of a subject-predicate-object statements that can be naturally mapped to a directed edge-labeled graph. While conceptually simple, the RDF data model supports a rich hierarchy of data types, identifiers and inference rules, and can in fact be shown to be more expressive than the property graph data model [15]. As the W3C standard of the RDF dates back twenty years, the RDF is supported by various graph database management systems (see [5] for an overview) and was thoroughly explored in the academic literature [195].

2.1.3 Beyond property graphs and RDF

While graph data models are suitable for expressing binary relations, they tend to struggle with n-ary relationships. RDF can handle these through complex reification schemes [97], but the solution is rather cumbersome to use due to its verbose nature. Similarly, models such as hypergraphs (extending the notion of an edge to a hyperedge) or hypernodes (nesting graphs by allowing nodes to be graphs) are considered too complex to enable intuitive manipulation, so their usage as a core graph representation in graph database systems is rare [71]. That being said, a simplified version of hypergraphs has recently been developed [15] and is being deployed as an intermediary model unifying both RDF and property graphs in several solutions [226, 29, 106, 130].

2.1.4 Terminology

We note that across the bodies of literature touching upon the use of graphs, the terms vertex/node are used interchangeably to a certain degree. In this document we adhere to this traditional practice, though referring preferably to a "vertex" when mentioning a graph element (but also "node" may be used when more adequate). When used to refer to a member of a group of machines (e.g., a cluster) used for processing, we will make that explicit (e.g. server node).

2.2 Graph Query Languages

The access to elements offered by graph databases is generally performed using specific query languages which vary depending on the underlying data model and system architecture. When it comes to `RDF`, the situation is rather clear, with all existing engines implementing `SPARQL` [92], which is a W3C standard, or minor variations thereof. On the other hand, the lack of a common standard in the property graph world meant that every engine implemented its own variant of a graph query language. Historically, these came in the form of an access and traversal API, Gremlin [192] being the most prominent example, or since the early 2010s as fully fledged declarative languages with `Neo4j`'s `Cypher` [75] being the most representative example.

While the openCypher project [161], established in 2015, attempted to define a common language for graph querying, it did not succeed in establishing a standard. With the expansion of the graph database market the lack of a standard query language became apparent, and in 2019 the Joint Technical Committee 1 of ISO/IEC started work on the `GQL` (Graph Query Language) standard, which was released in 2024 [110]. In parallel, the same ISO/IEC committee developed the `SQL/PGQ` standard (PGQ standing for *property graph queries*), which extends `SQL` with the ability to define graph views over tables and run read-only queries over such views. The two standards provide the same graph pattern matching capabilities, but serve different purposes, `SQL/PGQ` being an `SQL` extension for graph querying, while `GQL` is a stand alone graph language. `SQL/PGQ` was released in 2023 as part of the new version of the `SQL` standard [109].

Fundamental query features common to most graph query languages [13, 12] can be roughly classified as follows:

- *Basic graph patterns*, which amount to matching a smaller graph-shaped pattern to the database;
- *Complex graph patterns*, which extend basic graph pattern with filters and data tests;
- *Path queries*, which search for paths of potentially unbounded length;
- *Navigational graph patterns*, which allow replacing a simple edge in a basic graph pattern with a path specification; and
- *Aggregate graph queries*, which allow for aggregation on top of the previous pattern classes.

Some other atomic tests [222, 86] include matches on the vertices that are adjacent and edges that are incident to a given vertex, or triangles, which look for three vertices adjacent to each other. Generally, when defining constructs of a graph query language, it is important to consider if it shall be declarative or not, and how easy it will be to translate the use of the language into actual computational execution and performance.

Here, we address some of the most relevant graph query languages (and proposals in the scope of graph query languages), projects that structure the access to graph-based data in graph databases as well as open-source constructs that form their basis.

- `Cypher`. An evolving graph query language [75] which debuted with `Neo4j`'s entrance in the field of graph databases. There have been efforts to adopt and develop the language in an open-source approach [85]. `Cypher` has been an open and evolving language as part of the `OpenCypher` project [101]. The project has members involved in aspects such as synergy

of engineering efforts with Apache Spark [233], a language group, and even interoperability features for systems that use Gremlin. This graph query language heavily influenced the ISO project for creating a standard graph query language [161] and has a syntax familiar to developers with knowledge of SQL. Examples of databases where Cypher queries may be run include Neo4j, Graphflow [119], RedisGraph [34], SAP Hana Graph [194], and all databases where Gremlin is supported [162]. This language is also used to express computation in GRADOOP [115] and the Python Ruruki [172] lightweight in-memory graph database.

- GQL and SQL/PGQ. The two recent standards draw heavily on the Cypher syntax and share common pattern matching capabilities [55]. Given their recent publication, there are not many systems that are fully compliant, but multiple vendors started offering support for GQL and SQL/PGQ. For example, Oracle Database 23ai is the first commercially available system with SQL/PGQ support [221] and several engines such as PostgreSQL [62], DuckDB [231] or Google Spanner Graph [82] offer partial coverage of the standard. In the case of GQL, given the closeness of the language with Cypher the coverage is more immediate, with multiple engines claiming full or partial support for the standard. These include Neo4j [143], Nebula Graph [160], Ultipa [217], MillenniumDB [226], Google Spanner Graph, and Microsoft Fabric [153] among others.
- Gremlin. A graph traversal machine and language developed in the scope of the Apache TinkerPop project [74]. This project's development and growth were promoted by the now-defunct Titan graph database [17], which was forked into the open-source JanusGraph database [111] and the commercial DataStax Enterprise Graph (DSE) solution [48]. The traversal machine of Gremlin is defined as a set of three components [192]: the data represented by a graph G, a traversal Ψ (instructions) which consists of a tree of functions called *steps*; a set of traversers T (read/write heads). With this composition, the Gremlin and its traversal machine enable the exploration of multi-dimensional structures that *model a heterogeneous set of "things" related to each other in a heterogeneous set of ways* as detailed in [192]. A traversal evaluated against a graph may generate billions of traversers, even on small graphs, due to the exponential growth of the number of paths that exist with each step the traversers take. Examples of databases supporting Gremlin include: OrientDB [190], Neo4j [229], DataStax Enterprise [48], InfiniteGraph [169], JanusGraph [111], Azure Cosmos DB [179] or Amazon Neptune [21], while the graph processing systems that allow its use are Apache Giraph [41] or Apache Spark [16].
- SPARQL. The standard query language for RDF data (triples), also known as the query language for the semantic web [92]. We mention it due to its graph pattern matching capability [180] and scalability potential of querying large RDF graphs [103]. As RDF is a directed labeled graph data format, SPARQL becomes a language for graph-matching. Its queries have three components: a *pattern matching part* allowing for pattern unions, nesting, filtering values of matchings, and choosing the data source to match by a pattern; a *solution modifier* to allow modifications to the computed output of the pattern, such as applying operators as projections, orderings, limits and distinct; the *output*, which can be binary answers, selections of values for variables that matched the patterns, construction of new RDF data from the values or descriptions of resources. While graph databases may not necessarily be triple stores, the graph query languages they support may allow, for example, the RDF-specific semantics of a SPARQL query to be translated to Cypher, Gremlin or another language. For example, SPARQL is also supported (analytics) over GraphX [198] and the higher-level graph analytics tool GraphFrames [19]. Most prominent systems supporting SPARQL are Virtuoso [63], Jena TDB [112], BlazeGraph [212], Amazon Neptune [21] and AllegroGraph [76]. Some new engines like Stardog [208], MillenniumDB [226] or QLever [20] are also gaining traction, most notably

as contenders for large public service endpoints such as Wikidata [177]. For a detailed overview of over a hundred SPARQL engines, please consult [5].

- GraphQL. A framework developed and internally used at Facebook for years before its reference implementation was released as open-source [66]. It introduced a new type of web-based interface for data access. As a framework, one of its core components is a query language for expressing data retrieval requests sent to web servers that are GraphQL-aware. The queries are syntactically similar to JavaScript Object Notation (JSON). The GraphQL specification implicitly assumes a logical data model implemented as a virtual, graph-based view over an underlying database management system [94]. It has been studied with the semantics of its queries formalized as a labeled-graph data model, and the total size of a GraphQL response has been shown to be computable in polynomial time [95]. GraphQL is more than a query language - it defines a contract between the back-end and front-end over an agreed-upon type system, forming an application data model as a graph. It is helpful as it proposes a decoupling between the back-end and front-end, allowing each component to be changed independently of the other. For example, to serve queries in the graph, the data in the back-end could come from databases (e.g., Neo4j as the back-end serving GraphQL queries received at a web endpoint [164]), in-memory representations, or other APIs.
- Academic languages. There is a rich history of graph query languages coming from the academic community, some of these with a direct impact on query standards. Early works focused on extracting paths [46] and introduced regular path queries (RPQs), which form the core of virtually all standards when it comes to path extraction. More complex patterns allowing to combine paths, relational joins and aggregation were soon introduced [45] and adopted in the literature. Another example is G-Log, a declarative query language on graphs, which was designed to combine the expressiveness of logic, the modeling of complex objects with identity, and the representations enabled by graphs [176]. In recent years many academic proposals directly influenced features of concrete query languages [135]. For example RPQs [46] and their extensions with data tests [134] influenced the design of Oracle's PGQL and together with STRUQL [70] and regular queries [187], inspired the G-CORE proposal [12] from the Graph Data Council (formerly known as the Linked Data Benchmark Council).[2] G-CORE incorporates multiple query facets from academic proposals and unifies them in a single setting, including complex path patterns, relational operations and the ability to manipulate path as atomic objects. G-CORE is considered a fundamental influence on the design of GQL and SQL/PGQ patterns.

These different query languages were proposed across decades, some inspired by the lessons of their elders, others branching off from the same idea. They were influenced by the graph data models that were created to express the representation of graphs. The standardization efforts for a graph query language are helpful to integrate the lessons from across the literature.

Together with the mentioned main languages, a non-negligible amount of proprietary languages has arisen. Often, a particular graph database product develops a custom flavor of an existing language to properly fit its specific cases. An example of a language based on Cypher can be CypherPlus - a proprietary language of PandaDB, or Transwarp Extended - a Cypher extension of Stellar DB. Gizmo, on the other hand, is a Gremlin-like language of Google Cayley database, and UQL (Ultipa Query Language) a GQL's extension of Ultipa database.

Multi-model databases also like to extend SQL that is still the most familiar language for their users (AQL of ArangoDB, or the language of OrientDB). Naturally, Oracle's Spatial and Graph

[2] Website: https://ldbcouncil.org/

provides an SQL-based language PGQL (Property Graph Query Language). But even a native graph database, such as TigerGraph, queries its data with GSQL, a proprietary language based on SQL.

Some databases develop their graph languages more independently of the existing main approaches, such as DQL (Dgraph Query Language) of Dgraph, or TypeQL of TypeDB. Development inspirations can also find their foundations in other known languages or concepts with FQL (Fauna Query Language) of FaunaDB being an example of a TypeScript-like approach, or a JSON-based FlureeQL of the Fluree database.

There are even projects focused on analytics which offer the ability to explore datasets using graph query languages without actually running a graph database, such as GRADOOP [116] and GraphFrames [156].

2.3 Storage Architectures

The combined efforts of academia and industry players have created a vibrant ecosystem of graph databases, the dimensions of which we address in this survey. Among these dimensions, the matter of the internal representation of graphs becomes especially relevant and directly related to storage architecture design.

In this section, we focus on selected unique storage architecture choices of the graph database solutions we analyzed, if provided. We organize them into four main categories, according to the data structures they employ to architect the data storage and manage data internally: from more loosely to more strictly prescribed and advanced.

2.3.1 Unstructured or Loosely Structured Architectures

In one extreme, a graph database may not adhere to any specific external data structure and store the data just as binary (large) objects with no limitation to the internal structure.

- **BLOB-based Unstructured Data:** PandaDB [201] is based on Neo4j and therefore, the data is stored in the form of a byte array, which is not useful by itself in understanding the metadata structure before deserializing it. This database adds a semantic layer to the graph representation by manipulating unstructured data, which include metadata values such as MIME, length and version, among others. PandaDB changes Neo4j's property format by introducing binary large objects (BLOBs) as a data type to store unstructured data. Specifically, Neo4j's PropertyStore byte array is adapted so that its last 28.5 bytes store the metadata of unstructured data. In summary, for unstructured data, the metadata is stored in the property store file (extended from Neo4j's version), and the literal content is stored as BLOBs, either as separate files or in HBase.

  The semantic information of the unstructured data can be computed and stored lazily or eagerly, with potential indexing in the latter case. Attention must be paid to the eager case, which demands a caching mechanism to maintain semantic information to avoid constant recomputation. It has a caching mechanism that assigns the values of a semantic space to a specific AI model with a serial number. When the AI model is updated, so is its serial number. The semantic values associated with that AI model are valid in the cache only if their associated serial number matches the latest serial number of the AI model. PandaDB chooses a suitable index for different semantic information, taking into account that, for example, facial features are represented as vectors or that the text content of the audio is in string format,

among other examples. Numerical semantic data is indexed using the `B-Tree` structure [44, 83], with an inverted index [230, 239] being used for semantic information comprised of texts and strings, and high-dimension vector data relies on inverted vector search [31]. Indices can be built in batch or dynamic mode (the latter to update an already existing model based on new data). Additional details can be found in [201].

2.3.2 Linear Structured Architectures

The most typical design for storage architecture employed by graph databases is to resort to a pre-defined linear data structure. This approach introduces some restrictions on how graph data is stored but simplifies management, allowing some degree of improved efficiency (e.g., direct access to properties) and extended semantics (e.g., transactional).

- **Double-linked list:** `Neo4j` [154] stores properties as a double-linked list of property records, each holding a key and a value pointing to the next property. A specific type of record used in this scheme stores the content of properties in binary format.[3] `Neo4j` makes use of pointers for storing graph structure and properties of nodes and edges (which can become challenging if the entire graph does not fit into the server's memory).
- **Sortledton:** [77] is a universal graph data structure using an adjacency list-like design, which has one adjacency index and an adjacency list for each neighborhood. The neighborhoods are sets of destinations, and the index is a map. The authors chose this design based on their insight that optimizing for sequential vertex access is less beneficial than for any other pattern. They consider three advantages: *1)* an adjacency list-like data structure is embarrassingly parallel at the granularity of vertices because its neighborhoods are independent; *2)* the maintenance of the index is simple and cheap because it is independent of changes to the neighborhoods; *3)* the adjacency list-like design allows reusing well-studied map and set data structures from prior research.
- **Record-based:** `ArcadeDB` [139] defines records as vertices, edges, and documents with a record ID (RID) assigned on creation. These are stored in pages (default size of 64KB) loaded in RAM as `ByteBuffer` objects. There is the concept of **buckets**, which are physical files made of pages with multiple buckets per type to foster parallel job execution. Each bucket has an associated **index**, which uses the Log-structured merge-tree (LSM-Tree) algorithm [175] for efficient insertion speed and concurrent compaction in the background. `ArcadeDB` enables data partitioning based on properties, which allows using a specific index on a lookup by key.
- **Key-value abstraction:** `JanusGraph` [111] maps the graph to a key-value abstraction applied to an underlying storage layer (e.g., `Cassandra` or `HBase`). While this achieves scalability, this abstraction also limits graph store flexibility, as storing node properties or adjacency lists as a single opaque object impedes finer-grained access to elements.
- **Transactional Log-based:** `Transactional Edge Log` (`TEL`) [237] is a data structure introduced by `LiveGraph` which was designed in tune with the database's concurrency control algorithm. `TEL` combines a sequential memory layout with multi-versioning, containing cache-aligned timestamps and counters, which are leveraged to preserve the sequential nature of scans even when processing concurrent transactions. Vertices are stored in an individual way in what are described as *vertex blocks*, while edges are grouped in what the authors describe as `TELs`, which are stored in a single large memory-mapped file managed by a memory allocator

[3] See: `https://neo4j.com/developer/kb/understanding-data-on-disk/`

in LiveGraph. There is a vertex index and an edge index, which store pointers to appropriate blocks via vertex/edge ID. Vertices are used with a standard copy-on-write approach so that the newest version of the vertex can be found through the vertex index, with each version pointing to the previous one within the vertex block. Adjacency lists are stored as multi-versioned logs to support snapshot isolation efficiently, allowing read operations to proceed while avoiding interference with each other and write operations. While the default isolation level of many other commercial DBMSs, such as Neo4j, is read-committed, LiveGraph provides stronger snapshot isolation (see [237]).

2.3.3 Non-Linear Structured Architectures

Alternatively, graph databases can define architectures that use non-linear data structures, in order to organize data storage and management. This approach is typically taken to achieve improved flexibility (e.g., tracking modifications) or performance (e.g., indexing) at the expense of simplicity.

- **Tree-based:** ByteGraph's edge-tree [132] is a data structure similar to the B-tree to store vertex adjacency lists. It can be configured to fit different workloads with various read-write requirements in order to balance read and write amplifications. Regarding the durable storage layer, a persistent key-value store is used for fine-grained storage management of the edge tree. This is due to most data not having a fixed size (e.g., the number of neighbors or the length of edge/vertex properties). Each adjacency list is divided according to edge types and directions, with each list being stored as what the authors name the edge tree. It is composed of three types of nodes: root node, meta node, and edge node. The first two serve the purpose of indexing, while the edge nodes store the physical edge data. An edge tree will begin only with the root node and edge nodes, but as it increases in edge number, the meta nodes are created as a middle layer to index edge nodes (see Section 4.1 of [132]).
- **Layered abstraction:** TerminusDB introduces its *terminus-store* data structure [219], that may be regarded actually as a meta architecture. In fact, it represents a graph database using layers that conceptually map to the scheme of objects used in the git version control system. Each layer is identified by a unique 20-byte name. The initial layer represents a simple graph by using of succinct data structures such as front-coded dictionaries, bit sequences, and wavelet trees inspired by HDT [146]. New layers are added over the initial one to add new RDF subjects, objects, predicates, or values [219].

2.3.4 Relational storage

Many systems rely on classical relational architecture to store their data. Although this might be obvious for engines that aim to support SQL/PGQ, the practice is far more prevalent, particularly in the RDF setting, where the dataset is viewed as one giant table with three columns denoting the subject-predicate-object element of an RDF triple. Similar modeling is also possible in the case of property graphs. Engines can roughly be separated into the two common relational storage categories: row-based and column-based.

- **Row-based storage.** Traditionally many RDF/SPARQL engines deploy this approach and view data as one big table (commonly dubbed *triplestore*) [5]. The table itself might contain three or four columns, depending on whether named graphs are supported. In many engines the underlying architecture is based on B+trees indexes which contain dictionary encoded IRIs in multiple subject-predicate-object permutations to speed up query execution. These

can be clustered or unclustered, depending on the particular engine. Notable examples include `Virtuoso` [63], `TDB` (an RDF storage and query component of `Jena`) [112], `Blazegraph` [212], `MillenniumDB` [226] and `RDF-3X` [168]. Some engines like `MillenniumDB` also support property graphs, storing them using the same clustered B+tree architecture used in their RDF engine.

- **Column-based storage.** Columnar engines are prevalent in data analytics workspace and are increasingly being used for managing graph data. For example `Kuzu` [114] is a fully fledged graph database using columnar storage. Another example is `DuckDB`'s extension `DuckPGQ` [231]. Both of these deploy some sort of CSR-based encoding of the graph, which can be naturally stored in a columnar format. Interestingly `Virtuoso` offers both row-based and columnar storage capabilities.

2.3.5 Advanced Storage Architectures

Finally, we address graph databases that employ more advanced storage architectures. They aim to address two main objectives: i) space efficiency through some form of information summarization or compression, and ii) scalability (and also performance) by making use of (distributed) shared memory to manage data across multiprocessors or clusters of machines.

- **Succinct Data Structures:** `ZipG` [124] incorporates summarization techniques from the `Succinct` data structure [4] to achieve a memory-efficient representation. It transforms the input graph into a flat unstructured file, strategically storing a small amount of metadata along with the original input graph data in a way that the primitives of `Succinct` can be extended to efficiently implement the interactive graph queries of workloads such as those of Facebook `TAO`. Each server in `ZipG` stores a set of update pointers to ensure that during execution, only the necessary logs (and their relevant bytes) are accessed for query execution. The layout of `ZipG` has two flat unstructured files. It has the `NodeFile`, which stores IDs of all vertices with a small amount of metadata to trade off storage (in uncompressed representation) for efficient random access into node properties. The other file is the `EdgeFile`, which holds all edge records and uses metadata and conversion of variable-length data into fixed-length data (before compressing) to enable the trade-off between storage (uncompressed representation) and optimization of random access into the records and complex operations such as binary searches over timestamps. For more details see Sections 2 and 3 of [124].
- **Data Compression:** There are data compression schemes where the graph is compressed by different techniques in the literature. Among these techniques, we find examples like the compressed sparse row (CSR) [33], also known as the *Yale format*, the compressed sparse column variant, the `WebGraph` [26] format, and the k^2-tree structure [28, 158], among others, enabling the analysis of graphs over a compressed representation, effectively enabling graph analysis with a lower memory footprint while using commodity hardware.

  These techniques take advantage of aspects such as sparsity and the application of ordering criteria to its elements, to name a few. However, changing the graph typically requires a conversion to an uncompressed format, applying the graph-modifying operations and then converting back to the compressed representation. To overcome this bottleneck, the literature also saw the creation of compact graph schemes, enabling the modification of the graph while still retaining the benefits of compression. Techniques may be used to enable modifications over sets of static structures [159], with a recent example of a dynamic k^2-tree [42, 158]. Other techniques exist, harnessing the benefits of compression with very low-overhead decompression operations as seen in `LogGraph(Graph)` [23].

- **Shared Memory:** `G-Tran`'s graph-native data store [37] was designed to address the challenges of graph data irregularity and multi-hop traversal-based query costs. The memory space for each machine is split into two parts, one following an architecture where nothing is shared to store a local piece of the graph partition, while the other part obeys a shared-memory architecture, composing an RDMA-based distributed memory space to enable remote access of property data. The data store is composed of a `storage layer` and an `OLTP` layer. It offers a multi-version schema to support a consistent view for concurrent transactions.

  The `storage layer` represents a property graph in two parts: one part for graph topology, the vertices and edges as adjacency lists; the other part represents property data (the keys and values of vertices and edges). `G-Tran` uses an edge-cut partitioning strategy with hashing to partition a graph into N shards among N server nodes, with each shard storing one shard of vertices together with their in/out edges. A global address space is built over all deployed nodes so that the location of any object in the `storage layer` can be retrieved by ID. The `OLTP` layer has a group of worker threads to process incoming transactions by interacting with the data store, with each server node registering a chunk of memory at NIC, dividing the memory space of the node into RDMA-allocated memory and local memory. This is done to allow remote data access and fast communication. For more details, see Sections 4.1 and 4.2 of [37].

The nature of these contributions is important in the realm of space-efficiency, but additional dimensions must be considered when thinking of integrating them in full-fledged databases. The desire to enable parallel processing within a machine and/or across machines in a cluster adds a layer of complexity in the design of storage architecture of graph databases. Some graph databases use compression techniques with an explicit focus on single-machine setups, offering only the feature of high-availability to the cluster scenario (e.g., `Neo4j` [167]). Others were designed with sharding in mind (distributing the graph across machines), demanding the integration of the internal graph representation with considerations pertaining to consistency, coordination and latency of the storage architecture when designing it (e.g works adressing consistency [185, 84]). This is required not only to distribute a graph across machines but also to enable graph queries or global graph algorithms to effectively make use of all the computing resources.

3 Graph Databases

There are many dimensions to graph databases, both in the realm of the technological challenges their developers must address and also in the breadth of features/properties that different systems offer. In this section we highlight relevant aspects which must be considered in the design and development of graph databases. This is followed by a comprehensive set of features which are crucial for evaluating graph databases as well as an analysis of the features across a plethora of databases from both industry and open-source origins. The purpose of this extensive analysis is to provide guiding principles for the analysis and choice of graph database solutions, raising awareness of both existing graph database system as well as aspects that must be taken in consideration for the analysis and choice of graph databases. This section thus sheds light on key-aspects for both those who would set out to design and construct graph databases as well as those who would develop with and use them.

3.1 Challenges

In the scope of distributed graph transaction processing, several challenges have been noted in the literature [37]:

1. The irregularity of graph data [210] can lead to inadequate locality for reads and writes after continuous updates.
2. After multi-hops (fan-out [200]), the cost of traversal-based queries can become very high. The reason lies in the common power-law distribution on vertex degrees and the small-world phenomenon (i.e., high clustering coefficient and low distances) of many real-world graphs.
3. As a result, very large read/write sets may be involved in graph transactions. Adding to this, the connectedness of graph data can increase the contention/abort rate and lead to lower throughput when processing transactions.
4. Traditional centralized system architectures (with a primary node coordinator) may limit OLTP scalability (with the already mentioned contention likelihood, CPU overheads, and network bandwidth being typical sources of bottlenecks [234]).

3.2 Databases and Storage Systems Review

In this section, we list and describe different graph databases, which we group according to their type of supported graph model (e.g., property graph model, `RDF`, hybrid).

How to read this section. Section 3.2 can be read either top-down, or selectively by graph model: property graphs (Section 3.2.2), RDF engines (Section 3.2.3), multi-model systems (Section 3.2.4), and alternative models (Section 3.2.5). For a quick overview and cross-system comparison, we recommend starting with Tables 1 to 12, which summarize feature coverage across all systems, and then jumping to the summarized per-system descriptions for details. The overall aggregate analysis and main takeaways from the feature distribution analysis are discussed in Section 3.3.

We address the graph query languages of the graph databases. They either implement their own custom language (neglecting interoperability even though there may be automatic converters developed by third parties), or adhere to those that have been widely used and adopted by many projects (e.g., `Cypher`, `Gremlin`, `SPARQL`). The list includes open-source and commercial products, as well as database systems implemented to explore novel techniques as part of research activities.

For further information on the evolution of graph databases, we point the reader to contributions [14, 184], which cover, among other aspects, data models and query languages.

In cooperation with industry, we gathered forty six features relevant for potential users of graph databases. We group these into three main dimensions (or areas) (*Product*, *Database* and *Data*), each divided into a few classes consisting of up to seven features (or aspects) that we evaluated for every listed graph database.

This is illustrated in Figure 2. We address each dimension after explaining our scoring approach.

3.2.1 Scoring Methodology

We identified candidate systems from benchmark/industry lists, prior surveys, and curated community directories, and retained those providing persistent graph storage and a documented graph model and query/access interface. For each feature, we relied primarily on official vendor documentation, repositories/release notes, authoritative benchmark/academic sources and developer mailing lists. A feature was credited only when explicitly documented or evidenced in an official release; ambiguous cases were scored conservatively and the rationale recorded in the per-system

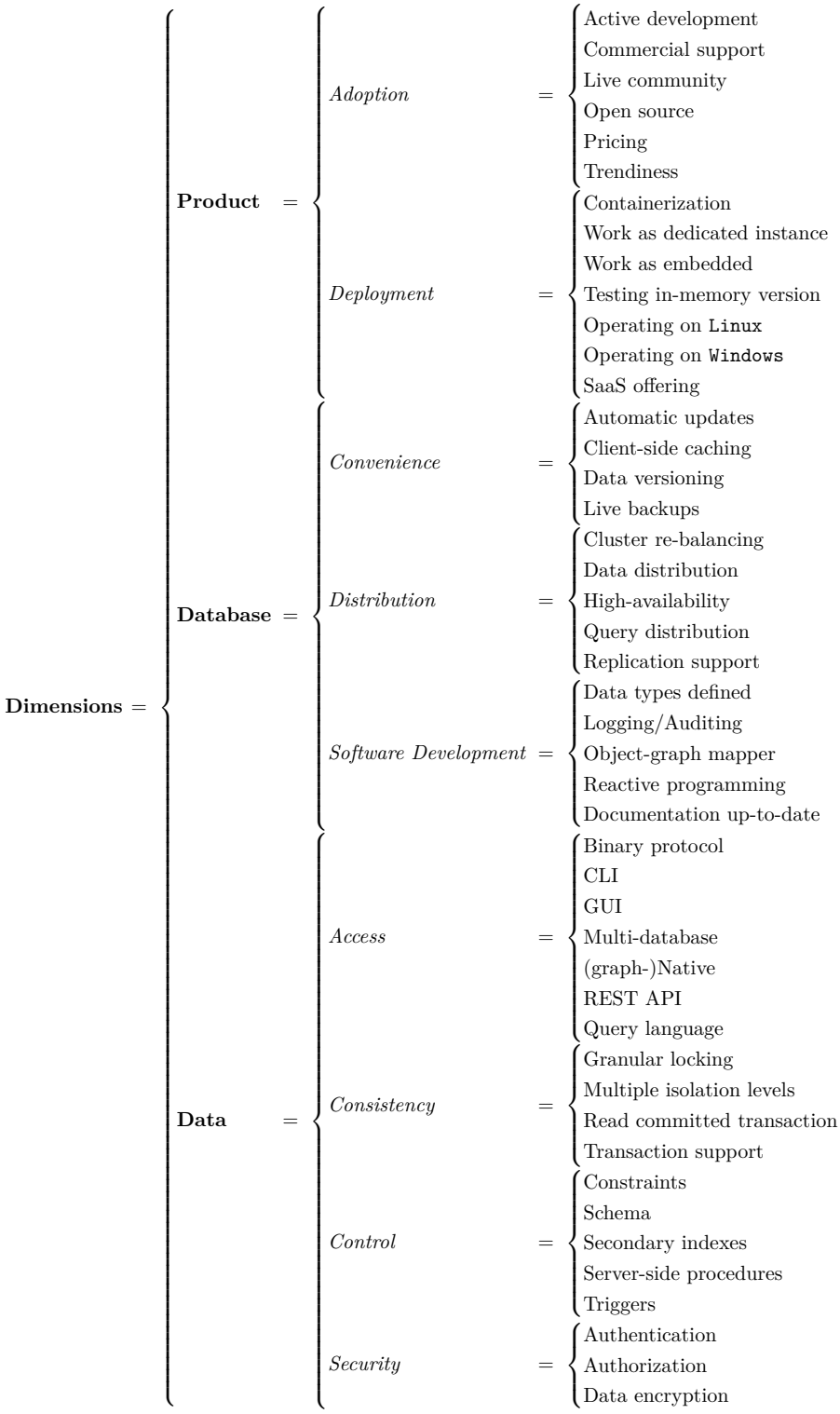

Figure 2 Description of dimensions, classes and features of analysis.

appendix in Tables 15 and 16. Intermediate scores (0.25/0.5/0.75) capture limited/partial support.

Disagreements were resolved by re-checking primary sources and reaching consensus.

Product

The Product dimension contains features specific for embracing a software product as well as its provided manipulation means. These features are, therefore, common for any software system but at the same time considered to be very important for potential graph-database users. We divide them into the following two classes with their respective features.

- *Adoption* - Active development / Commercial support / Live community / Open source / Pricing / Trendiness.
- *Deployment* - Containerization / Work as dedicated instance / Work as embedded / Testing in-memory version / Operating on `Linux` / Operating on `Windows` / SaaS offering.

For *active development* and *trendiness*, we consider a reference date of 2026-01-01 for scoring, and we check the relevant activity in the time preceding this date. We note that *active development* has been obtained from a mixture of automated and manual verifications.

For databases with public repositories in GitHub/GitLab/SourceForge, automatic verification is deployed to check the date of the latest commit, release and details of alternative branches in the event that the master branch does not demonstrate relevant activity within the last six months before the reference date. If a repository has been expressively archived or the repository discontinued with no mention of a replacement, the corresponding database receives a score of 0 for *active development*. Tables 17–21 in the paper appendix show the details of activity verification for all the databases considered in this survey.

In the event that databases do not have repositories available, manual verification is performed by checking the date of the latest release and the date of the latest update on the official website, as well as checking for any news or announcements regarding the database.

Regarding the *trendiness* feature, it is based on downloaded CSV data from Google Trends. We note that due to the topic overlap for certain database names, it was necessary to adjust the query keywords for some databases to obtain more accurate results:

- `ChronoGraph` used the expression *ChronoGraph graph database* (colliding with the concept of watches).
- `Gaffer` used the expression *Gaffer graph database* (colliding with a type of tape).
- `Graphflow` used the expression *Graphflow graph database*.
- `Kuzu` used the expression *Kuzu graph* (colliding with the name of a fibrous vine native to East Asia).
- `LiveGraph` used the expression *LiveGraph storage*.
- `TAO` used the expression *TAO graph database* (colliding with unrelated Bittensor's $TAO cryptocurrency).
- `Virtuoso` used the expression *Virtuoso graph* (colliding with the concept of a virtuoso).
- `Weaver` used the expression *Weaver transactional graph database* (colliding with a verb with many overlaps).
- `ZipG` used the expression *ZipG graph* (colliding with the concept of a zip and the name of a file format).

Database

Features specific for any database product, specifically focused on needs of graph database users, are grouped within the Database dimension. These features are categorized into three main classes

covering distribution capabilities, specifics for software development on top of the graph database, and features enabling smooth experience.

- *Convenience* - Automatic updates / Client-side caching / Data versioning / Live backups.
- *Distribution* - Cluster Re-balancing / Data Distribution / High-Availability / Query Distribution / Replication support.
- *Software Development* - Data types defined / Logging/Auditing / Object-Graph Mapper / Reactive programming / Documentation up-to-date.

Data

In the Data dimension, we address operations with data organized into four main feature classes: *Access*, *Consistency*, *Control* and *Security*. Data access covers means of read/write operations such as API and query languages as well as architectural level of access - whether the database is graph-native or accesses the data through an extra graph layer. Other areas focus on ensuring proper locking and transactional support, schema and optimization capabilities, and user access, rights, and encryption.

- *Access* - Binary protocol / CLI / GUI / Multi-database / (graph-)Native / REST API / Query language.
- *Consistency* - Granular locking / Multiple isolation levels / Read committed transaction / Transaction support.
- *Control* - Constraints / Schema / Secondary indexes / Server-side procedures / Triggers.
- *Security* - Authentication / Authorization / Data encryption.

For completeness, we include a short description of each feature in the paper appendix (Tables 15 and 16). They can also be accessed in our GitHub repository devoted to this survey.[4]

Review Results

All the systems were extensively reviewed and the result of their analysis is shown in Tables 1 to 12. The tables show the assessment of all the considered features on the databases studied, with features lined up grouped by class (double line separators), and classes ordered by their area - or dimension (triple line separator), following the same sequence by which they were described earlier.

The scoring assigned to each feature always ranges in the interval from 0 to 1 (i.e. 0, 0.25, 0.50, 0.75, 1.00) based on whether feature is absent (0), or completely fulfilled/present/provided/supported (1.00). When relevant and possible, we also differentiate when this is achieved only limitedly (0.25), partially (0.50), or significantly (0.75). For clarity and ease of reference, we resort to associating such values with a graphical representation (i.e., respectively *empty* ○, *quarter-filled* ◔, *half-filled* ◓, *three quarter-filled* ◕, *and full circle* ●).

Next, we will further cover individual systems in detail analyzing how they cumulatively fulfill or address each of the dimensions (or areas), as a whole, along with a short description of more relevant aspects. For every dimension (or area) (*Product, Database* and *Data*) we summarize the evaluation of its features to one index, counted as a simple average of all the underlying numerical values of feature scores (in the scale 0, 0.25, 0,50, 0,75, 1.00 presented earlier).

Note that the evaluation is based on information available to public and on scientific papers, if present.

[4] `https://github.com/svitaluc/survey-gdbs`

Regarding the open source feature, we note that the absence of evidence is not necessarily evidence of absence (especially regarding enterprise products): to the best of our ability, we have analyzed each system's information ecosystem for clues on licensing status.

Consequently, proprietary databases of private companies (such as `TAO` of Meta or `ByteGraph` of ByteDance) may have more features than publicly reported that are understandably kept as a corporate secret.

Our evaluation is necessarily documentation-driven, relying on publicly available material and scientific papers when present. When a feature is not publicly documented, we conservatively score it as absent or not verifiable according to the rubric (Tables 15 and 16). Therefore, the reported scores should be interpreted as a lower bound on publicly documented capabilities, and may underestimate proprietary/enterprise systems whose implementations or operational features are not disclosed.

We mitigate this by prioritizing primary sources and applying consistent, conservative scoring in ambiguous cases.

3.2.2 Property Graph Model

The following graph databases we list are focused on the property graph model. It means that these graph databases directly employ the property graph model, and they do not support any query languages for `RDF` in their features.

3.2.2.1 Alibaba Graph Database / TuGraph

Product 0.38 Database 0.46 Data 0.53

The Ant Group, which is behind the Alibaba B2B marketplace, has a cloud-oriented graph database service, providing horizontal scalability, ACID transactions and supporting the `TinkerPop` stack. It supports the property graph model, the `Gremlin` query language and there are programming interfaces for `Go`, `Java`, `.NET`, `Node.js` and `Python`. For visualization, a paid third-party tool *G.V() - Gremlin Graph Database IDE and Visualization Tool* [2] can be utilized. Among online resources, we found a page which refers to this offering as `Graph Database` [6] and another page with the same graphics but naming the offering as `TuGraph` [216], the latter in a public repository under `Apache License 2.0`.

3.2.2.2 Apache HugeGraph

Product 0.62 Database 0.75 Data 0.71

`HugeGraph` [133] is a graph database written in `Java` and compliant with the `TinkerPop 3` framework. It adopts the edge-cut partition scheme and offers a decoupling between the actual `storage layer`, supporting technologies such as `RocksDB`, `Cassandra` and `HBase` among others. It integrates with `Flink`, `Spark` and other data processing engines. `HugeGraph` also has a `graph engine layer` which intermediates with the `storage layer`, supports both `OLTP` and `OLAP` graph computation types and offers a `REST` API to query the graph as well as APIs for service monitoring and operations. Above the `graph engine layer` there is an `application layer` which contains the following relevant components: *a)* `Hubble`, a visual analytics platform for the process of data modelling, online/offline analysis and guided workflow for operating graph applications; *b)* the `Loader` data importing tool to convert from different formats and bulk import into the database; *c)* `Tools`, command-line utilities to deploy, manage and restore data; *d)* `Computer`, a distributed graph processing system which implements `Pregel` and can run on `Kubernetes`; *e)* `Client`, a `Java`-based client to operate `HugeGraph`, with the mention of other languages being a possibility

Table 1 Summary of graph database distinctive features (Part I-a).

		Feature	Alibaba GDB	ChronoGraph	DataStax Enterprise	Dgraph	Graphflow	JanusGraph	NebulaGraph	Neo4j	RedisGraph/FalkorDB
Product	Adoption	Active development	○	●	○	●	○	●	●	●	◑
		Commercial support	●	○	●	●	○	○	●	●	●
		Live community	○	○	◔	◑	○	●	●	●	●
		Open source	○	●	○	●	●	●	●	●	●
		Pricing	●	○	○	○	●	●	○	◑	●
		Trendiness	○	○	○	●	○	●	●	●	●
	Deployment	Containerization	○	○	●	●	○	●	●	●	●
		Work as dedicated instance	●	○	●	●	●	●	●	●	●
		Work as embedded	○	●	○	○	○	●	○	●	○
		Testing in-memory version	○	○	○	●	◑	●	○	●	●
		Operating on Linux	●	●	●	●	●	●	●	●	●
		Operating on Windows	○	●	◑	●	●	●	◑	●	◑
		SaaS offering	●	●	●	●	●	●	◑	●	○
Database	Convenience	Automatic updates	●	○	○	○	○	○	○	○	○
		Client side caching	○	○	○	○	○	○	○	○	◑
		Data versioning support	○	●	○	○	○	○	○	○	○
		Live backups	●	○	●	●	○	●	●	●	◑
	Distribution	Cluster re-balancing	○	○	●	●	○	○	●	○	○
		Data distribution	○	○	●	●	○	●	●	●	○
		High-availability	●	○	●	●	○	●	●	●	◑
		Query distribution	◑	○	●	●	○	○	●	●	○
		Replication support	●	○	●	●	○	●	●	●	●
	Software Development	Data types defined	○	●	●	●	●	●	●	●	●
		Logging/Auditing	●	●	●	◑	◑	●	●	●	◑
		Object-graph mapper	○	●	●	○	○	○	○	●	●
		Reactive programming	○	○	○	○	○	○	○	●	◑
		Documentation up-to-date	●	◔	◑	◑	○	◕	●	●	●

for the future. Its source code is available online under the `Apache License 2.0`, having an active mailing list and development community.

3.2.2.3 ByteGraph

Product 0.12 Database 0.46 Data 0.32

`ByteGraph` [132] is a distributed graph database, motivated by the use cases and massive amounts of data produced by platforms such as TikTok, Douyin and Toutiao (products of ByteDance). The authors of `ByteGraph` define three categories of graph workloads: online analytics processing

Table 2 Summary of graph database distinctive features (Part I-b).

		Feature	Alibaba GDB	ChronoGraph	DataStax Enterprise	Dgraph	Graphflow	JanusGraph	NebulaGraph	Neo4j	RedisGraph/FalkorDB
Data	Access	Binary protocol	●	◕	●	●	○	●	●	●	●
		CLI	○	●	●	○	●	●	●	●	●
		GUI	○	○	●	●	○	○	●	●	○
		Multi-database	○	○	○	○	○	●	○	●	○
		Graph-native data	●	○	○	●	●	●	●	●	●
		REST API	○	●	○	●	○	●	◑	●	●
		Query language	●	●	●	●	◕	●	○	●	●
	Consistency	Granular locking	●	○	○	○	○	◑	○	●	○
		Multiple isolation levels	●	○	○	○	○	◑	○	○	○
		Read committed transaction	●	●	○	○	○	◑	●	●	○
		Transaction support	●	●	○	●	○	●	●	●	●
	Control	Constraints	◑	○	●	○	○	●	○	●	●
		Schema support	◑	◑	●	●	○	●	●	●	○
		Secondary indexes	●	●	●	●	○	●	●	●	●
		Server side procedures	○	○	○	○	○	○	●	●	●
		Triggers	○	○	●	○	○	◑	●	●	◑
	Security	Authentication	●	○	●	●	○	●	●	●	◑
		Authorization	○	○	●	●	○	●	●	●	◑
		Data encryption	○	○	●	●	○	○	○	○	◑

(OLAP) such as knowledge graphs and risk management; transaction processing (OLTP) such as what is used in recommendation and GNN sampling; and serving of fresh data to applications in real time (OLSP) such as e-commerce. This graph database makes use of a two-tier architecture consisting of a durable storage layer and a cache layer for low-latency data access. `ByteGraph` optimizes the caching layer by introducing the `edge-tree`, a data structure similar to the B-tree in order to store the adjacency lists of vertices. This provides high parallelism when accessing the neighbourhood of high-degree vertices and reduces the overall data loaded into memory for queries such as edge searching. The authors propose two adaptive optimizations on thread pools and indexes to adapt to sudden changes such as rapid workload increases. Scenarios of machine failure are also considered, as `ByteGraph` employs several techniques to guarantee availability such as geographic replications and weighted consistent hashing [155]. Computation and storage are decoupled to enhance scalability. It supports the `Gremlin` `GQL` (the authors provide both a rule-based and a cost-based optimizer) and the full set of queries used in the evaluation of `ByteGraph` are displayed online on GitHub [131]. We did not find its source code online. It is an in-house solution for which we did not find release notes or changelog, but it has been continuously updated based on recent publications [25].

Table 3 Summary of graph database distinctive features (Part II-a).

		Feature	SAP Hana	Sparksee	TigerGraph	Weaver	AllegroGraph	BlazeGraph	BrightstarDB	Cray Graph Engine	Ontotext GraphDB
Product	Adoption	Active development	○	○	●	○	○	○	○	○	●
		Commercial support	●	●	●	○	●	○	○	●	●
		Live community	○	○	◔	○	○	◔	●	○	●
		Open source	○	○	○	●	○	●	●	○	○
		Pricing	●	○	○	○	●	○	○	○	○
		Trendiness	●	○	●	○	●	●	◒	○	●
	Deployment	Containerization	●	○	●	◒	●	○	○	○	●
		Work as dedicated instance	●	●	●	●	●	●	●	●	●
		Work as embedded	○	●	●	○	○	●	●	○	●
		Testing in-memory version	○	○	◒	○	○	○	○	○	○
		Operating on Linux	●	●	●	●	●	◒	◒	●	●
		Operating on Windows	○	●	◒	◒	○	◒	◒	○	●
		SaaS offering	●	○	○	○	●	○	○	○	○
Database	Convenience	Automatic updates	○	○	○	○	○	○	○	○	○
		Client side caching	●	○	○	○	●	○	○	○	○
		Data versioning support	●	○	○	○	●	○	○	○	●
		Live backups	●	●	●	○	●	○	○	●	◒
	Distribution	Cluster Re-balancing	●	○	○	○	●	○	○	○	○
		Data distribution	●	○	●	●	●	●	●	●	◒
		High-availability	●	○	●	●	●	●	●	◒	●
		Query distribution	●	○	●	●	●	●	●	●	◒
		Replication support	●	○	◒	◒	●	●	●	○	●
	Software Development	Data types defined	●	●	●	○	●	◒	◒	◒	●
		Logging/Auditing	●	○	◒	◒	●	●	●	●	●
		Object-graph mapper	●	○	○	○	●	○	○	●	●
		Reactive programming	●	●	○	○	●	○	○	○	○
		Documentation up-to-date	●	●	◒	○	●	●	●	●	●

3.2.2.4 ChronoGraph

Product 0.45 Database 0.30 Data 0.38

`ChronoGraph` [89] is a `TinkerPop` compliant (offering `Gremlin` to query the data in property graph model) graph database supporting ACID transactions, system-time content versioning and analysis. It is implemented as a key-value store enhanced with temporal information, using a B-tree data structure. It currently exists as part of the Chronos Project, an ecosystem of projects which has `ChronoDB` (versioned key-value store), `ChronoGraph` (the database) and `ChronoSphere` (versioned Eclipse Modeling Framework repository). The project is available online [88] under the `aGPL v3` (open-source and academic purposes) and commercial licenses are available on demand.

Table 4 Summary of graph database distinctive features (Part II-b).

		Feature	SAP Hana	Sparksee	TigerGraph	Weaver	AllegroGraph	BlazeGraph	BrightstarDB	Cray Graph Engine	Ontotext GraphDB
Data	Access	Binary protocol	●	○	○	●	○	●	●	●	○
		CLI	●	○	●	○	●	●	●	●	●
		GUI	●	○	●	○	●	●	●	●	●
		Multi-database	●	○	●	○	●	○	○	○	○
		Graph-native data	○	●	●	○	●	●	●	●	●
		REST API	●	○	●	○	●	●	●	●	●
		Query language	○	●	●	○	●	●	●	●	●
	Consistency	Granular locking	●	○	○	○	○	○	○	○	○
		Multiple isolation levels	●	●	○	○	●	●	●	○	○
		Read committed transaction	●	○	◒	◒	○	●	●	○	●
		Transaction support	●	●	●	●	●	●	●	○	●
	Control	Constraints	●	●	○	○	●	○	○	◒	●
		Schema support	●	●	●	○	●	○	○	◒	◒
		Secondary indexes	●	●	○	○	●	●	●	●	●
		Server side procedures	●	○	●	○	●	●	●	●	●
		Triggers	●	○	○	○	●	○	○	○	○
	Security	Authentication	●	○	●	○	●	○	○	●	●
		Authorization	●	○	●	○	●	○	○	●	●
		Data encryption	●	●	●	○	○	○	○	○	○

It is to be deployed as a process-embedded library. No visualization functionalities accompany the database.

3.2.2.5 DataStax Enterprise Graph (DSE)

Product 0.43 Database 0.68 Data 0.58

`DataStax Enterprise Graph (DSE)` [48] is a proprietary fork of `Titan` and supporting `Java`. It integrates with the `Cassandra` [129] distributed database (over which it provides graph data models) and supports `TinkerPop` (property graph with `Gremlin`). It is complemented by the `DataStax Studio`, which allows for interactive querying and visualization of graph data similarly to `Neo4j` and `SAP Hana Graph`. There are several repositories belonging to the DataStax company, many under `Apache License 2.0`, among which a fork of the `Cassandra` database with code pertaining to the company's product.[5] There is a repository with release notes of their products [49].

[5] For a list of repositories: `https://github.com/orgs/datastax/repositories`

Table 5 Summary of graph database distinctive features (Part III-a).

		Feature	Neptune	Altair Graph Lakehouse	ArangoDB	IBM System G	OrientDB	OSG	Stardog	Virtuoso
Product	Adoption	Active development	●	◒	●	○	●	●	●	●
		Commercial support	●	●	●	○	●	●	●	●
		Live community	●	○	◒	○	◒	◒	●	●
		Open source	○	○	●	○	●	○	○	●
		Pricing	●	●	○	○	○	●	○	●
		Trendiness	●	○	●	●	●	○	●	●
	Deployment	Containerization	○	●	●	○	●	◒	●	●
		Work as dedicated instance	●	●	●	●	●	●	●	●
		Work as embedded	○	○	○	○	●	○	○	○
		Testing in-memory version	○	●	●	●	●	○	○	○
		Operating on Linux	●	●	●	●	●	●	●	●
		Operating on Windows	○	○	●	○	●	●	○	●
		SaaS offering	●	○	○	○	○	●	●	●
Database	Convenience	Automatic updates	●	○	○	○	○	●	○	○
		Client side caching	●	○	○	○	○	●	○	○
		Data versioning support	○	●	○	○	○	●	○	○
		Live backups	●	●	●	○	●	●	●	●
	Distribution	Cluster Re-balancing	○	○	●	○	○	◒	○	○
		Data distribution	○	●	●	●	●	●	○	●
		High-availability	●	●	●	○	●	●	●	○
		Query distribution	◒	●	●	●	◒	●	●	●
		Replication support	●	●	●	○	●	●	●	●
	Software Development	Data types defined	◒	◒	●	◒	●	◒	◒	○
		Logging/Auditing	●	●	●	○	●	●	●	●
		Object-graph mapper	○	○	●	○	○	●	○	○
		Reactive programming	●	○	○	○	○	●	●	○
		Documentation up-to-date	●	●	●	○	◒	●	●	◒

3.2.2.6 Dgraph

Product 0.86 Database 0.57 Data 0.58

`Dgraph` [57, 58] was written in `Go` and it is a distributed graph database offering horizontal scaling and ACID properties. It is built to reduce disk seeks and minimize network usage footprint in cluster scenarios. `Dgraph` is licensed under two licenses: the `Apache License 2.0` and a `Dgraph Community License`. It automatically moves data to rebalance cluster shards. It uses a simplified version of the `GraphQL` query language. There is a proprietary enterprise version (conditions

Table 6 Summary of graph database distinctive features (Part III-b).

		Feature	Neptune	Altair Graph Lakehouse	ArangoDB	IBM System G	OrientDB	OSG	Stardog	Virtuoso
Data	Access	Binary protocol	●	○	●	●	●	●	○	○
		CLI	●	●	●	●	●	●	●	●
		GUI	●	●	●	○	●	●	●	●
		Multi-database	○	○	●	○	●	●	●	●
		Graph-native data	●	●	○	●	●	●	○	○
		REST API	●	●	●	●	●	●	●	●
		Query language	●	●	◒	●	●	●	●	●
	Consistency	Granular locking	●	○	○	○	○	●	●	●
		Multiple isolation levels	●	○	○	○	●	●	●	●
		Read committed transaction	●	○	○	○	●	●	○	●
		Transaction support	●	●	●	●	●	●	●	●
	Control	Constraints	●	○	●	○	○	●	●	●
		Schema support	◒	●	◒	○	●	●	●	○
		Secondary indexes	●	●	●	○	●	●	●	●
		Server side procedures	○	○	◒	○	●	◒	●	●
		Triggers	○	○	○	○	●	●	○	●
	Security	Authentication	●	●	●	○	●	●	●	●
		Authorization	●	●	◒	○	●	●	●	●
		Data encryption	●	●	●	○	●	●	○	○

specified under a custom `Dgraph Community License`) with advanced features for backups and encryption. Official clients include `Java`, `JavaScript` and `Python`. `Dgraph` does not offer native visualization and global analytics functionalities (but third-party tools supporting JSON format can be applied).

3.2.2.7 Galaxybase

Product 0.50 Database 0.54 Data 0.65

`Galaxybase` [215] is a commercial distributed graph database, offering a *Pioneer* version (with limited storage capacity[6]) and two superior tiers: *Enterprise* and *Ultimate*. The website states it offers compressed data storage, a native graph storage engine and partitioned sharded storage.[7] It supports the property graph model and the enterprise line offers properties types such as lists and collections, as well as cleaning of invalid data. Indices may be created for properties

[6] See: `https://createlink.com/download`
[7] See: `https://createlink.com/galaxyProduct`

Table 7 Summary of graph database distinctive features (Part IV-a).

		Feature	Cosmos DB	FaunaDB	Google Cayley	HyperGraphDB	Objectivity/DB	KatanaGraph	TerminusDB	MemGraph
Product	Adoption	Active development	○	○	●	○	○	○	●	●
		Commercial support	●	●	○	○	●	●	●	●
		Live community	◒	○	○	◒	○	◒	◒	◒
		Open source	○	◒	●	●	○	●	●	○
		Pricing	●	●	○	○	○	●	●	○
		Trendiness	●	○	●	○	○	○	●	●
	Deployment	Containerization	◒	●	●	○	○	○	●	●
		Work as dedicated instance	●	●	●	●	●	○	●	●
		Work as embedded	○	○	●	●	○	○	○	○
		Testing in-memory version	●	○	●	○	○	○	○	◒
		Operating on Linux	○	●	●	●	●	●	●	●
		Operating on Windows	●	○	○	◒	●	○	◒	◒
		SaaS offering	●	●	○	○	●	○	●	●
Database	Convenience	Automatic updates	●	○	○	○	○	○	○	○
		Client side caching	○	○	◒	●	○	○	○	○
		Data versioning support	○	●	○	●	○	○	●	○
		Live backups	●	●	○	○	○	○	●	●
	Distribution	Cluster Re-balancing	◒	○	○	○	○	●	○	●
		Data distribution	●	●	●	◔	●	●	○	●
		High-availability	●	●	◒	○	●	○	○	●
		Query distribution	●	●	●	◔	○	○	○	●
		Replication support	●	●	●	○	●	○	○	●
	Software Development	Data types defined	○	◒	●	●	◒	●	◒	○
		Logging/Auditing	●	●	●	○	●	○	●	●
		Object-graph mapper	●	○	●	●	○	○	○	●
		Reactive programming	●	○	○	○	○	○	○	○
		Documentation up-to-date	●	●	●	●	○	○	●	◒

and read-committed transactions are supported. Queries may be written with `Cypher` and there are APIs for `Java`, `Go`, `Python` and `JavaScript`. With respect to operational management, it offers data backups/recovery, load balancing, role-based access control at the graph level, with fine-grained permissions for edges and attributes. It has a visual interface to explore the graph.

3.2.2.8 Gaffer

Product 0.23 Database 0.46 Data 0.32

`Gaffer` is a graph database framework, providing modular storage for very large graphs with properties on vertices and edges. [80]. It allows the use of storage options such as `Accumulo` [90], `HBase` [79] and `Parquet` [225] (though documentation in the current version claims the last

Table 8 Summary of graph database distinctive features (Part IV-b).

		Feature	Cosmos DB	FaunaDB	Google Cayley	HyperGraphDB	Objectivity/DB	KatanaGraph	TerminusDB	MemGraph
Data	Access	Binary protocol	○	○	○	●	●	○	○	●
		CLI	●	●	●	○	●	○	○	●
		GUI	●	○	●	○	●	○	○	●
		Multi-database	●	○	○	●	○	○	○	○
		Graph-native data	○	●	●	○	●	●	○	●
		REST API	●	○	●	○	●	○	●	○
		Query language	●	●	●	◒	◒	○	◒	●
	Consistency	Granular locking	○	○	○	○	○	○	●	○
		Multiple isolation levels	●	○	○	◒	○	○	○	○
		Read committed transaction	●	○	○	◔	○	○	○	●
		Transaction support	◒	●	●	●	●	●	●	●
	Control	Constraints	○	●	◒	●	○	○	●	●
		Schema support	○	●	◒	◒	●	○	○	◔
		Secondary indexes	●	●	●	●	○	○	○	●
		Server side procedures	●	●	○	○	○	○	○	●
		Triggers	●	○	○	◒	○	○	○	●
	Security	Authentication	●	●	○	○	●	○	●	●
		Authorization	●	●	○	○	●	○	●	●
		Data encryption	●	○	○	○	○	○	○	○

two will be deprecated[8] in version 2 of `Gaffer`). It was created by UK's intelligence, security, and cyber agency. `Gaffer` is licensed under the `Apache License 2.0` and is covered by Crown Copyright. Its source code is available online [80] with development instructions.[9] `Gaffer` contains a `time-library` enabling timestamps to be set on stored entities and edges. It does this by offering classes such as `RBMBackedTimestampSet` which stores timestamps truncated to a certain resolution (e.g., minutes, hours and so on) and `BoundedTimestampSet` which allows for the accumulation of timestamps up to a maximum of N (if it goes over this number, N are sampled from the total). It is written in `Java` and `JavaScript` and is able to interoperate with `Spark` `RDD`, `DataFrame` and `GraphFrame` representations. It also offers a `REST` API. Any distributed processing afforded by `Gaffer` is achieved by virtue of the underlying storage technology and use of `Spark`.

3.2.2.9 Graphflow

Product 0.46 Database 0.11 Data 0.14

`Graphflow` [119, 151, 149] was released as a prototype active graph database. It is an in-memory graph store employing the property graph model and supporting both one-time and continuous

[8] See: `https://github.com/gchq/Gaffer/issues/2556` and `https://github.com/gchq/Gaffer/issues/2590`
[9] See: `https://gchq.github.io/gaffer-doc/v1docs/summaries/getting-started.html`

Table 9 Summary of graph database distinctive features (Part V-a).

		Feature	PandaDB	Gaffer	ZipG	LiveGraph	G-Tran	ByteGraph	Apache HugeGraph	TypeDB	SurrealDB
Product	Adoption	Active development	○	○	○	○	○	○	●	●	●
		Commercial support	○	○	○	○	○	○	○	●	●
		Live community	○	○	○	○	○	○	●	●	◔
		Open source	●	●	●	●	●	○	●	●	◔
		Pricing	○	○	○	○	○	○	○	●	○
		Trendiness	◔	○	○	○	●	◔	○	●	●
	Deployment	Containerization	●	○	○	○	○	○	●	●	●
		Work as dedicated instance	●	○	●	●	●	●	●	●	●
		Work as embedded	●	○	○	○	○	○	○	○	●
		Testing in-memory version	○	○	○	●	●	○	●	●	●
		Operating on Linux	○	●	●	●	●	○	●	●	●
		Operating on Windows	●	●	◔	◔	●	○	●	●	●
		SaaS offering	○	○	○	○	○	○	○	●	●
Database	Convenience	Automatic updates	○	○	○	○	○	○	●	○	○
		Client side caching	●	○	○	○	○	○	○	○	○
		Data versioning support	○	○	○	○	○	●	○	○	○
		Live backups	●	○	○	○	○	○	●	○	○
	Distribution	Cluster Re-balancing	○	○	○	○	○	○	●	○	●
		Data distribution	○	●	●	○	●	●	●	○	●
		High-availability	●	○	○	○	○	●	●	●	●
		Query distribution	◔	●	●	○	●	●	●	●	●
		Replication support	●	●	○	○	○	●	●	●	●
	Software Development	Data types defined	●	◔	◔	◔	◔	◔	◔	◔	◔
		Logging/Auditing	●	●	○	○	○	●	●	●	●
		Object-graph mapper	●	●	●	○	●	○	●	○	●
		Reactive programming	●	○	○	○	○	○	○	●	○
		Documentation up-to-date	●	●	●	●	●	○	●	●	●

sub-graph queries. `Graphflow` uses a one-time query processor called *Generic Join* and a *Delta Join* which enables the continuous sub-graph queries. It extends the `openCypher` language with triggers to perform actions upon certain conditions. We did not find information regarding its direct use beyond academic purposes nor about supporting ACID transactions. Its code was available online [120] under the `Apache License 2.0`, but now returns a 404 error as of March 2026.

Table 10 Summary of graph database distinctive features (Part V-b).

		Feature	PandaDB	Gaffer	ZipG	LiveGraph	G-Tran	ByteGraph	Apache HugeGraph	TypeDB	SurrealDB
Data	Access	Binary protocol	●	○	○	○	○	○	●	●	●
		CLI	●	○	○	○	○	○	●	●	●
		GUI	●	○	○	○	○	○	●	●	●
		Multi-database	●	○	○	○	○	○	○	○	●
		Graph-native data	●	●	●	●	●	●	●	○	○
		REST API	●	●	○	○	○	○	●	○	●
		Query language	●	○	○	○	●	●	●	◒	◒
	Consistency	Granular locking	●	○	○	●	●	●	●	○	○
		Multiple isolation levels	○	○	○	○	○	○	◒	◒	◒
		Read committed transaction	●	○	○	○	○	●	●	●	●
		Transaction support	●	●	○	●	●	●	●	●	●
	Control	Constraints	●	○	○	○	○	○	○	●	●
		Schema support	●	●	○	○	○	●	●	●	●
		Secondary indexes	●	○	○	○	○	○	●	●	●
		Server side procedures	●	○	○	○	○	○	○	○	●
		Triggers	●	○	○	○	○	○	○	●	●
	Security	Authentication	●	●	○	○	○	○	●	●	●
		Authorization	●	●	○	○	○	○	●	●	●
		Data encryption	○	○	○	○	○	○	○	○	○

3.2.2.10 G-Tran

Product 0.46 Database 0.32 Data 0.21

`G-Tran` [37] is available online [36] under `Apache License 2.0` and written in `C++`. It is a remote direct memory access (RDMA)-enabled distributed in-memory graph database, offering support for serialization and snapshot isolation. The authors introduce a graph-native data store to enable desirable data locality and fast data access for transactional updates and queries. `G-Tran` offers a fully-decentralized architecture, relying on RDMA to process distributed transactions with the massively parallel processing (MPP) model, aiming to maximize the usage of computer resources. The authors propose a novel multi-version optimistic concurrency control (`MV-OCC`) protocol to handle the challenge of large read/write sets in graph transactions. It was benchmarked against `JanusGraph`, `ArangoDB`, `Neo4j` and `TigerGraph` (Developer Edition). It offers snapshot isolation and strong consistency, aiming to provide low latency and high throughput using: *a)* graph-native data store with efficient data memory layouts to support data locality and fast access for read/write graph transactions under frequent updates; *b)* a decentralized architecture based on RDMA to avoid the bottlenecks of centralized transaction coordination. `G-Tran` uses `Gremlin` for querying and supports the property graph model.

Table 11 Summary of graph database distinctive features (Part VI-a).

		Feature	Ultipa	BangDB	ArcadeDB	Fluree	HGraphDB	StellarDB	AgensGraph	GalaxyBase	MillenniumDB	Kuzu	TAO
Product	Adoption	Active development	◔	●	●	●	●	●	●	○	●	○	○
		Commercial support	●	●	○	●	○	●	●	●	○	○	○
		Live community	●	○	●	◔	●	●	◔	○	◔	●	○
		Open source	○	●	●	●	●	○	●	○	◑	●	○
		Pricing	●	●	●	○	○	○	○	○	○	○	○
		Trendiness	◔	○	◑	○	○	○	○	○	○	●	○
	Deployment	Containerization	●	●	●	●	○	○	○	●	●	●	○
		Work as dedicated instance	●	●	●	●	○	●	●	●	●	○	●
		Work as embedded	○	●	●	●	○	○	●	●	○	●	○
		Testing in-memory version	○	●	○	○	○	○	○	○	○	●	○
		Operating on Linux	●	●	●	●	●	●	●	◔	●	●	○
		Operating on Windows	●	○	●	●	●	○	●	◔	◔	●	○
		SaaS offering	●	●	○	●	○	●	●	●	○	○	○
Database	Convenience	Automatic updates	●	○	●	●	○	○	○	○	○	○	○
		Client side caching	○	○	○	○	●	○	○	○	○	●	●
		Data versioning support	○	○	○	●	◔	○	○	○	○	○	○
		Live backups	●	○	●	●	○	○	●	●	○	○	●
	Distribution	Cluster Re-balancing	○	○	○	○	●	○	○	○	○	○	○
		Data distribution	●	●	○	○	●	●	○	●	○	○	●
		High-availability	●	●	●	○	●	○	●	●	○	○	●
		Query distribution	●	●	○	○	●	○	○	●	○	○	○
		Replication support	●	●	●	○	●	○	●	●	○	○	●
	Software Development	Data types defined	●	◓	●	●	●	◓	◓	◓	●	●	●
		Logging/Auditing	●	●	●	●	●	○	●	●	○	●	○
		Object-graph mapper	○	○	○	○	●	○	○	○	○	○	○
		Reactive programming	○	○	●	○	○	○	○	○	○	○	○
		Documentation up-to-date	●	●	●	●	○	●	●	●	◓	●	○

3.2.2.11 HGraphDB

Product 0.38 Database 0.68 Data 0.37

`HGraphDB` [232] is an implementation of `TinkerPop` 3, functioning as a client layer to use `HBase` as a graph database and open-sourced under `Apache License 2.0`. It enables the creation of schemas for vertices and edges. Graph analytics can be achieved by providing integration with `Giraph`, `Spark GraphFrames` and `Flink Gelly`, which are able to access the data stored in `HBase` for computation. Because `TinkerPop` is used as a layer over `HBase`, `Gremlin`-based tools are available. The distribution, scalability and storage mechanisms are those of the underlying `HBase`.

Table 12 Summary of graph database distinctive features (Part VI-b).

		Feature	Ultipa	BangDB	ArcadeDB	Fluree	HGraphDB	StellarDB	AgensGraph	GalaxyBase	MillenniumDB	Kuzu	TAO
Data	Access	Binary protocol	○	○	●	○	●	○	○	●	●	●	○
		CLI	●	●	●	○	○	●	●	●	●	●	○
		GUI	○	●	●	○	◒	●	●	●	●	●	○
		Multi-database	○	●	●	○	○	●	●	○	○	○	○
		Graph-native data	●	○	●	●	○	○	○	●	●	●	○
		REST API	●	●	●	●	○	○	○	●	●	●	●
		Query language	◒	●	●	●	●	●	●	●	●	●	○
	Consistency	Granular locking	○	○	○	○	○	○	○	○	○	●	○
		Multiple isolation levels	○	○	●	○	○	○	◒	○	◒	●	○
		Read committed transaction	○	○	●	●	●	○	●	○	●	●	○
		Transaction support	○	●	●	●	◒	○	●	●	●	●	●
	Control	Constraints	◒	◒	◒	◒	○	○	●	○	○	○	○
		Schema support	●	●	●	◒	●	○	●	●	○	●	●
		Secondary indexes	●	●	●	●	●	○	●	●	●	○	●
		Server side procedures	○	○	●	○	○	○	●	●	○	○	●
		Triggers	●	◒	●	○	○	○	●	●	○	○	●
	Security	Authentication	●	●	●	●	●	●	●	●	●	○	○
		Authorization	●	●	●	●	○	●	●	●	○	○	○
		Data encryption	●	○	○	●	○	○	○	○	○	○	○

3.2.2.12 JanusGraph

Product 0.92 Database 0.52 Data 0.74

`JanusGraph` [111] is an open-source project licensed [18] under the `Apache License 2.0`. A database optimized for storing (in adjacency list format) and querying large graphs with (billions of) edges and vertices distributed across a multi-machine cluster with ACID transactions. `JanusGraph`, which debuted in 2017, is based on the `Java` code base of the `Titan` graph database project and is supported by the likes of Google, IBM and the `Linux` Foundation, to name a few. Like `Titan`, it supports `Cassandra`, `HBase` and `BerkeleyDB` (among others) as an underlying storage. `JanusGraph` can integrate platforms such as `Spark`, `Giraph` and `Hadoop`. It also natively integrates with the `TinkerPop` graph stack, supporting `Gremlin` applications, the query language and its graph server, with graphs in the property graph model. Due to supporting `TinkerPop` , one may use one of its drivers to use `Gremlin` from `Elixir`, `Go`, `Java`, `.NET`, `PHP`, `Python`, `Ruby` and `Scala`. It supports global analytics using `Spark` integration as well.

3.2.2.13 KatanaGraph

Product 0.38 Database 0.21 Data 0.11

`KatanaGraph` is a library and set of applications to work efficiently with large graphs. It offers parallel graph analytics and machine learning algorithms, multithreaded execution with load

balancing and a scalability focus on multi-socket systems. It provides scalable concurrent containers (e.g. bag, vector and list) and an interface to implement algorithms. It is licensed under the BSD-3-Clause license. KatanaGraph's enterprise offering is known as Graph Intelligence Platform, providing solutions for scalable purposes such as querying (contextual searches), analytics (path-finding, centrality and community detection), mining (for pattern discovery) and prediction. It has been tested with clusters of over 256 machines, supercomputers and public clusters on Amazon Web Services (AWS), Google Cloud Platform (GCP) and Microsoft Azure, and offers C/C++ and Python programming bindings.[10] KatanaGraph provides different partitioning policies (1D, 2D and hybrid vertex-cut) as well as user-specified policies and an optimized partition-aware communication layer. Its distributed parallel computing framework is based on previous works such as the Gluon [50] communication engine and the Cusp [99] streaming partitioner. KatanaGraph's value proposition is the integration of a graph computing subsystem of a graph database, analytics and machine learning. Although it is not a graph database in itself, it is worth mentioning as a scalable distributed system allowing for the ingestion and processing of data from many different sources while harnessing many types of computational resources. Its last release was in 2022, and we did not find information about its future development plans.

3.2.2.14 Kuzu

Product 0.62 Database 0.29 Data 0.61

Kuzu [114] is an open-source, embedded property graph database intended to be integrated directly into applications rather than deployed as a standalone server. Its main focus is on high-performance single-node graph analytics. It adopts strongly typed graph data model and provides a Cypher-based (GQL-inspired) query language for expressive graph pattern matching and analytical workloads. It is implemented in C++ with a vectorized execution engine that achieves high query throughput on local machines while supporting ACID transactions under read-committed isolation. The system is cross-platform, running on Linux and Windows, and integrates smoothly with developer environments such as Python. It favours simplicity and development ease through an embedded API and command-line interface, while deliberately omitting server-oriented and enterprise features including clustering, replication, authentication, and high availability; thus, more suited for embedded analytics, data science, and local graph processing rather than large-scale distributed deployments. It is under the MIT License, and the project has been archived as of October 2025 (as per its GitHub repository).[128]

3.2.2.15 LiveGraph

Product 0.37 Database 0.11 Data 0.16

LiveGraph [237] is a graph storage system which optimizes graph workloads by ensuring that adjacency list scans are *purely sequential*, that is, random accesses are never required, even in the face of concurrent transactions. It implements in-memory and out-of-core graph storage on a single server and offers the property graph model. The authors present the Transactional Edge Log (TEL), a graph-aware data structure with a concurrency control mechanism. LiveGraph proposes to be a graph storage system supporting both transactional and real-time analytical workloads. The authors employ single-threaded micro-benchmarks and micro-architectural analysis for comparison of different commonly-used data structures and assess the advantage of a sequential memory layout. This system guarantees the following properties: *1)* that different versions of edges in the same adjacency list have to be stored in contiguous memory locations; *2)* locating

[10] See: https://katanagraph.com/resources/datasheets/katana-graph-intelligence-platform

the right version of an edge should not depend on auxiliary data structures which could lead to the occurrence of random accesses; *3)* the concurrency control algorithm does not require random access during scans. Its source code is available online under Apache License 2.0 and written in C++ [238]. It is a relevant approach to graph storage although the evaluated techniques are presented as the LiveGraph system without being part of a database system (thus lacking all the functionalities typically found in them).

3.2.2.16 Memgraph

Product 0.70 Database 0.63 Data 0.72

Memgraph is a commercial graph database with both an on-premise version and a cloud service [140]. It was developed for real-time streaming and is compatible with Neo4j. It supports ingestion of data from Kafka, SQL and plain CSV files and allows users to query data using Cypher. It is implemented in C/C++, offering an in-memory architecture and ACID compliance. The source code is available online under a custom license called Business Source License 1.1 (BSL), the Memgraph Enterprise License (MEL). It supports dynamic graph algorithms such as dynamic PageRank with a stream of data, where rankings are updated as soon as new graph objects are created in the database. Memgraph provides graph functionality divided by modules. It has a module for traditional graph algorithms, covering classes of graph problems such as centrality measures, path-finding and community detection. Another module contains streaming graph algorithms for centrality and community detection algorithms. It also provides graph machine learning algorithms, such as link prediction and node classification based on graph neural networks (GNNs), as well as computing node embeddings on static graphs (node2vec) and a graph neural network algorithm to predict new edges and node labels from graph structure and available node and edge features. There is also a module that supports integration with NVIDIA CUDA and networkx. The commercial product also provides Memgraph Lab, a visual user interface to connect and explore data in Memgraph instance and MAGE, an open-source repository of query modules and graph algorithms.

3.2.2.17 NebulaGraph

Product 0.69 Database 0.64 Data 0.66

Nebula Graph is an open-source graph database (available online [224]) licensed under Apache License 2.0, provides a custom Nebula Graph Query Language (nGQL) with syntax close to SQL and Cypher support is planned. It supports the property graph model, ACID transactions and is implemented with a separation of storage and computation, being able to scale horizontally. It supports multiple storage engines like HBase [79] (implementing the graph logic over these key-value stores) and RocksDB [65] and has clients in Go, Java and Python. It also has the complementing Nebula Graph Studio for interactive visual querying and analytics.

3.2.2.18 Neo4j

Product 0.96 Database 0.71 Data 0.89

Neo4j [229] is a graph database with multiple editions [163]: a community edition licensed under the free GNU General Public License (GPL) v3, and a commercial one. We note that its public GitHub repository has last been updated two years ago (and therefore consider it is not in active development). However, the company has been continuously updating the product/service offering. It supports different programming languages C/C++, Clojure, Go, Haskell, Java, JavaScript, .NET, Perl, PHP, Python, R and Ruby. Neo4j is optimized for highly-connected data. It relies on methods of data access for graphs without considering data locality. Neo4j's graph processing

consists of mostly random data access. For large graphs which require out-of-memory processing, the major performance bottleneck becomes the random access to secondary storage. The authors created a system which supports ACID transactions, high availability, with operations that modify data occurring within transactions to guarantee consistency. It uses the `Cypher` query language and data is stored on disk as fixed-size records in linked lists. `Neo4j` has a library offering many different graph algorithms. All writes are directed to the `Neo4j` cluster master, an architecture which has its limitations. Among other uses, `Neo4j` has also been employed for building applications using the `GRANDstack` framework [142]. `Neo4j` also has an interactive graph explorer to query and update specific elements of the graph. It also offers the `Aura` automated cloud platform database as a service.

3.2.2.19 PandaDB

Product 0.46 Database 0.68 Data 0.89

`PandaDB` [201] is an open-source distributed graph database for the management and querying of both structured and unstructured data in one framework. `PandaDB` is aimed at applications like recommendation systems, financial technology (e.g., fraudulent cash-out detection) and knowledge graphs (e.g., new data operators to process the information of unstructured data to accelerate graph neural networks). It consists of: *1*; a graph data model to manage the data; *2*; a `Cypher` query language extension to understand the semantic information of the unstructured graph data; *3* a cost model and query optimization techniques to speed up unstructured data processing. It adopts the native graph database `Neo4j` as its foundation and estimates the cost of unstructured data operations (AI model inference is used to understand the unstructured data) with query plan optimization. It is an academic software with multiple versions available.[11] It is available under the `Apache License 2.0` and written mostly in `Scala` [202].

3.2.2.20 RedisGraph

Product 0.79 Database 0.46 Data 0.58

`RedisGraph` [34] is a property graph database which uses sparse matrices to represent a graph's adjacency matrix and uses linear algebra for graph queries. It uses custom memory-efficient data structures stored in RAM, having on-disk persistence and tabular result sets. Queries may be written in a subset of `Cypher` and are internally translated into linear algebra expressions. It has a custom license and client libraries for `RedisGraph` have been developed in `Elixir`, `Go`, `Java`, `JavaScript`, `PHP`, `Python`, `Ruby` and `Rust`, complementing existing accesses that `Redis` already supports. As far as we know, it only works in single-server mode (bounded by the machine's RAM) and it does not support ACID properties. It has a Community Edition under a custom license, the latter with source code available online [141].

3.2.2.21 SAP Hana Graph

Product 0.54 Database 0.93 Data 0.89

`SAP Hana Graph` [194, 104, 197] is a column-oriented, in-memory relational database management system. It performs different type of data analysis, among which, graph data processing with the property graph model and ACID transactions. This graph functionality includes interpretation of `Cypher` and a visual graph manipulation tool. Its graph processing capabilities have served use

[11] See: `https://github.com/grapheco`

cases like fraud detection and route planning. No source code is available online as the product is purely commercial.

3.2.2.22 Sparksee

Product 0.38 Database 0.29 Data 0.42

`Sparksee` [145, 207] (formerly `DEX`) is a property graph database offering ACID transactions and representing the graph using bitmap data structures with high compression rates (with each bitmap partitioned into chunks that fit disk pages). The graphs in `Sparksee` are labelled multigraphs and it has multiple licenses depending on the purpose, with free licenses for evaluation, research and development. It offers APIs in `C++`, `.NET`, `Java`, `Objective-C`, `Python` and mobile devices. We found no capabilities for data visualization in `Sparksee`, though it is able to export data to formats supported by third-party software.

3.2.2.23 StellarDB

Product 0.46 Database 0.18 Data 0.32

`StellarDB` [209] is a distributed graph database with a custom graph storage structure, employing compression algorithms and partitioning strategies to reduce the storage footprint and to focus on even data distribution in the cluster. It offers a `Cypher`-based query language as well as `SQL`, and has a library of graph algorithms that are parallel-capable. Graph visualization is offered through an interface which supports 2D and 3D visualization display. It runs on a family of `Linux` operating systems.[12] `StellarDB` has database management features such as user permission authentication, cluster status monitoring, logging, data encryption and backup recoveries. It is subject to commercial licensing and it is not open-source. `StellarDB` depends on `HDFS`, `Yarn` and `ZooKeeper`. More information is available at the website of `StellarDB`.[13]

3.2.2.24 TAO

Product 0.13 Database 1.00 Data 0.67

`TAO` [30] debuted as Facebook's distributed graph store. It is a geographically-distributed data store, providing efficient and timely access to the social graph using a fixed set of queries. It replaced `memcache` for many data types and runs on thousands of machines in a distributed fashion. It divides the data into logical shards, with each shard assigned to a `MySQL` instance, and being contained in a logical database. Database servers manage one or more shards, requiring tuning shard-to-server mapping to balance load across hosts. All object types are stored in one table, and all association types in another table. `TAO` has a caching layer which implements a complete API for clients and handles all communication with databases. This layer has multiple cache servers (which together form a tier), and the cache is filled on demand with an LRU eviction policy. It was designed in a way to limit maximum size of tiers by creating a hierarchy, with a *leader* tier and *follower* tiers. Later, `RAMP-TAO` was presented [39] as a protocol over `TAO` which ensures atomic visibility for an environment that is geographically-distributed and read-optimized. We chose to include `TAO` due to its research and engineering relevance to enable operation on a massive graph use-case.

[12] See: `https://www.transwarp.cn/doc/tdh/9.3.3/Installation-guide--install-preparation-chapter`
[13] See: `https://www.transwarp.cn/en/product/stellardb`

3.2.2.25 TigerGraph

Product 0.77 Database 0.48 Data 0.66

`TigerGraph` [56, 214] is a commercial graph database (formerly `GraphSQL`[14]) implemented in `C++` and comes in three versions: developer edition (supporting only single-machine, no distribution and is only for non-production, research or educational purposes), cloud edition (as a managed service) and enterprise edition (allowing for horizontal scalability - distributed graphs). It supports ACID consistency, access through a `REST` API, has a custom `SQL`-like query language (`GSQL`) and features a graphical user interface named `GraphStudio` to perform interactive graph data analytics. The `TigerGraph` model was designed with its graph vertices, edges and their attributes to support an engine that performs massively-parallel processing to compute queries and analytics.

3.2.2.26 Ultipa

Product 0.69 Database 0.64 Data 0.53

`Ultipa` [217] focuses on achieving high scalability by employing an `HTAP` (`Hybrid Transactional` and `Analytical Processing`) architecture. This architecture (a practical implementation was published in recent years [102]) aims to remove the need of having multiple copies of data and the data offloading from databases to warehouses. It has its own `Ultipa Query Language` (`UQL`), and the authors intend to evolve it in order to keep up with compatibility and functionality of the developing international `GQL` standard of the LDBC (Linked Data Benchmark Council). There is the `Ultipa Manager` which unifies the management of the database, cluster, datasets, schemas and account management, enabling users to configure and visualize graph data. `Ultipa Cloud` offers a pay-as-you-go model and is a solution integrating the `Ultipa Manager` and `Ultipa Server`. It offers shared, standard and enterprise versions, the last one's price defined on a per-contact basis. On the website, drivers are mentioned for `Java`, `Python`, `C++`, `Go`, `Node` and a `REST` API. Its licensing is commercial and there is no open-source free version.

3.2.2.27 Weaver

Product 0.33 Database 0.29 Data 0.13

`Weaver` [61] is an open-source [60] graph database (custom permissive license) for efficient, transactional graph analytics. It introduced the concept of refinable timestamps. It is a mechanism to obtain a rough ordering of distributed operations if that is sufficient, but also fine-grained orderings when they become necessary. It is capable of distributing a graph across multiple shards while supporting concurrency. Refinable timestamps allow for the existence of a multi-version graph: write operations use their timestamps as a mark for vertices and edges. This allows for the existence of consistent versions of the graph so that long-running analysis queries can operate on a consistent version of the graph, as well as historical queries. `Weaver` is written in `C++`, offering binding options for `Python`. We did not find any support for popular graph query languages.

3.2.2.28 ZipG

Product 0.29 Database 0.32 Data 0.05

`ZipG` was designed as a memory-efficient graph store for interactive querying [124]. It employs a compressed graph representation supporting a range of graph queries which are applied directly on the compressed representation. It was evaluated using queries from different graph query

[14] See: `https://www.zdnet.com/article/tigergraph-a-graph-database-born-to-roar/`

workloads (e.g., such as the Facebook TAO use-cases), achieving an order of magnitude higher in throughput compared to Neo4j and Titan. ZipG was implemented in C++ and has a package running on Spark and written in Scala. It supports distributed execution and its source-code is available online [123], albeit missing licensing information. It warrants mentioning for its use of a compressed graph representation and support for graph query operations, but it does not offer traits of a graph database such as transaction support, replication, graphical user interfaces or graph query languages.

3.2.3 RDF Data Model

The following graph databases we address are focused on the RDF data model and variations (including support for the property graph model by representing them as RDF [93]):

3.2.3.1 AllegroGraph

Product 0.54 Database 0.93 Data 0.79

AllegroGraph [76] is a proprietary commercial graph database with clients under Eclipse Public License v1 (EPL v1) which supports several programming languages (C#, C, Common Lisp, Clojure, Java, Perl, Python, Scala) that was purpose-built for RDF (triple-store). It supports an array of mechanisms to access the information it stores, namely reasoning with an ontology (RDFS++ Reasoning), materialized reasoning (generating new triples based on inference rules - OWL2 RL Materialized Reasoner), SPARQL queries, Prolog and also low-level APIs.

3.2.3.2 Blazegraph

Product 0.44 Database 0.46 Data 0.58

Blazegraph [212] (formerly Bigdata) is an RDF database able to support up to billions of edges in a single machine and available under the GNU General Public License v2.0, supporting SPARQL and the TinkerPop Blueprints API. It has .NET and Python clients. One of its associated internal projects is blazegraph-gremlin, which allows the storage of property graphs internally in RDF format, which can then be queried with SPARQL. It also offers Blazegraph WorkBench as a GUI to visualize and query data. It essentially has an alternative approach to RDF reification, enabling labelled property graph capabilities on RDF graphs, with the ability to query the graphs in Gremlin as well. Stored procedures are supported via SPARQL extensions and transactions are supported through Multi-Version Optimistic Concurrency Control (OCC). Snapshot isolation is offered and authentication may be enabled with mechanisms outside the database itself. It uses a federated architecture to distribute the data across the cluster resources (with queries also functioning across the cluster). Blazegraph has been acquired by Amazon.[15]

3.2.3.3 BrightstarDB

Product 0.46 Database 0.46 Data 0.58

BrightstarDB is an open-source [27] multi-threaded multi-platform (including mobile) .NET RDF store, supporting SPARQL and binding of RDF resources to .NET dynamic objects (it has tools to use .NET interfaces and generate concrete classes to persist their data in BrightstarDB). It is licensed under the MIT License and there is also an Enterprise version. BrightstarDB supports single-threaded writes and multi-threaded reads, with ACID transactions. It does not support horizontal scaling.

[15] See: https://www.trademarkia.com/blazegraph-86498414

3.2.3.4 Cray Graph Engine

Product 0.23 Database 0.50 Data 0.58

`Cray Graph Engine (CGE)` [189] is an `RDF` triple database offering the `SPARQL` query language. As a commercial product, it was designed while considering different architectures of proprietary systems (containing the `Cray Aries interconnect` [7]) of the company behind `CGE`. It offers APIs for `Java`, `Python` and `Spark`, having a number of pre-built graph algorithms. It is not open-source and its back-end relies on internal queries written in `C++` to work with a global address space using multiple processes, across multiple compute nodes, to share data and synchronize operations. This product being both proprietary and reliant on custom hardware has the consequence of not being so widespread. However, its results and special-purpose architecture make it a competitive platform which harnessed innovation in design as a graph database.

3.2.3.5 Fluree

Product 0.73 Database 0.43 Data 0.54

`Fluree` [72] is an open-source (`Eclipse Public License 2.0`) [73] immutable (i.e., append-only) graph database with a cloud-native architecture, supporting the `JSON` linked data (`JSON-LD`) format[16]. It targets knowledge graphs and may be run in containers, embedded in applications (`Clojure` and `Node`) or as a stand-alone JVM service. `Fluree` can run both as a local instance or as a cloud-managed service. For graph querying it supports `SQL`, `SPARQL`, `GraphQL` and the custom `FlureeQL`. Data access control policies are expressed in `JSON-LD` as well. The documentation states that to begin storing data, a ledger (in the sense of blockchain) must be created in `Fluree` (analogous to the creation of a database within traditional database management systems). It records data operations and the transactions/queries will run against it and a ledger can enable the operation of more than one application.

3.2.3.6 Ontotext GraphDB

Product 0.82 Database 0.64 Data 0.66

`Ontotext GraphDB` [170] is a graph database focused on `RDF` data and offering ACID transaction properties. It comes in three editions: free which is used for smaller projects and for testing and is only able to execute at most two concurrent queries; standard which can load and query statements at scale; enterprise edition which offers horizontal scalability and other features. It supports `SPARQL` and offers a `Java` programming API. No source code for the free version is available.

3.2.3.7 TerminusDB

Product 0.81 Database 0.32 Data 0.34

`TerminusDB` is an open-source graph database and document store [219], described also as a toolkit for building collaborative applications. It offers a document API to build knowledge graphs from `JSON` documents. It is available under `Apache License 2.0`, offering bindings for `JavaScript` and `Python` [220] and employing delta encoding schemes similar to those of `git`[17]. It debuts its own `Web Object Query Language` (`WOQL`) and relies on an internal data structure called *`terminus-store`* written in `Rust`. It is focused on the OWL model (not property graph), extending `RDF` with classes, restrictions and strongly-typed properties. `TerminusDB` represents a graph as a

[16] `JSON-LD` is a standard for providing data context and interoperability.
[17] See: `https://git-scm.com/`

series of layers, similarly to git. Vertices and edges are enriched with classes and restrictions to model structure of data to enable finer granularity in how changes are expressed. Rather than different sequential versions of data, the storage is organized as base data and modifications that are stored as new layers over a base version of the graph.

3.2.4 Multiple Data Models

The following graph databases support at least both the property graph model and/or RDF explicitly plus other data models (e.g., at least any of: document collections, relational model, object model):

3.2.4.1 AgensGraph

Product 0.67 Database 0.39 Data 0.71

AgensGraph [205] is a multi-model database that supports graph manipulation (property graph model with JSON to represent properties) using Cypher and SQL in a hybrid query processing model. It is based on the PostgreSQL relational database management system, which offers horizontal and vertical scalability. It includes ACID transactions, multi-version concurrency control, stored procedures, triggers, constraints and monitoring.[18] There is an enterprise edition which offers monitoring, high availability and memory optimizations. A community edition is also available [206] under the Apache License 2.0 (mostly implemented in C). SKAI Worldwide Co., Ltd. (formerly Bitnine Global Inc.) is the company behind AgensGraph.Visualization capability is offered through the complementary product AgensBrowser.[19] AgensGraph is based on PostgreSQL RDBMS.[20]

3.2.4.2 Altair Graph Lakehouse

Product 0.50 Database 0.61 Data 0.58

Altair Graph Lakehouse [35] is a proprietary database built to enable RDF with SPARQL and the property graph with Cypher queries to analyse big graphs (trillions of relationships). It has Java and C++ APIs to create functions, aggregates and services. It supports ACID transactions and also supports an RDF+ inference engine following W3C standards and uses compressed in-memory and on disk storage of data. This database is described as beyond a transaction-oriented database and as a Graph Online Analytics Processing (GOLAP) database, enabling interactive view, analysis and update of graph data, in a way similar to the interactive capabilities of the Neo4j database. It comes as a single-machine (and memory usage limitations) free edition and an enterprise edition which supports unlimited cluster size. It supports third-party visualization tools. We did not find details on its internal data structures nor source code online. Altair Graph Lakehouse[21] was rebranded from AnzoGraph DB[22] (Cambridge Semantics was acquired by Altair), which was previously SPARQLVerse. The company behind this product, Altair Engineering Inc., has been acquired by Siemens AG.[23]

[18] See: https://www.skaiworldwide.com/resource
[19] See: https://worldwide-homepage-bucket.s3.ap-northeast-2.amazonaws.com/manual/agenbrowser+web+manual+pdf_quick+guide_(EN).pdf
[20] See: https://www.postgresql.org/about/news/announcing-the-release-of-agensgraph-v2160-3149/
[21] https://2025.help.altair.com/2025.0/graphlakehouse/userdoc/Home.htm
[22] See: https://hub.docker.com/r/cambridgesemantics/anzograph-db
[23] See: https://press.siemens.com/global/en/pressrelease/siemens-acquires-altair-create-most-complete-ai-powered-portfolio-industrial-software

3.2.4.3 Amazon Neptune

Product 0.62 Database 0.64 Data 0.82

`Amazon Neptune` [21] is a managed proprietary service (freeing the user from having to focus on management tasks, provisioning, patching, etc.) that is ACID-compliant and focused on highly-connected datasets. Among its use cases there are recommendation engines, fraud detection and drug discovery, among others. Its implementation language and internal graph representation have not been disclosed and it supports both the property graph and `RDF` models, offering `Gremlin` and `SPARQL` to query them. As a full-fledged commercial product, it also has many features related to backup, replicas, security and management tasks, using product features such as Amazon's `S3`, `EC2` and `CloudWatch` to offer scalability. Usage samples are available online [8, 9].

3.2.4.4 ArangoDB

Product 0.86 Database 0.71 Data 0.63

`ArangoDB` is an open-source [199] multi-threaded database with support for graph storage (as well as key/value pairs and documents) that is available in both a proprietary license and the `Apache License 2.0`. It is written in `JavaScript` from the browser to the back-end and all data is stored as `JSON` documents. `ArangoDB` provides a storage engine for mostly in-memory operations and an alternative storage engine based on `RocksDB`, enabling datasets that are much bigger than RAM. It guarantees ACID transactions for multi-document and multi-collection queries in a single instance and for single-document operations in cluster mode. Replication and sharding are offered, allowing users to set up the database in a master-slave configuration or to spread bigger datasets across multiple servers. It exposes a `Pregel`-like API to express graph algorithms (implying access to the stored data in the database), has a custom `SQL`-like query language called `AQL (ArangoDB Query Language)` and includes a built-in graph explorer.

3.2.4.5 ArcadeDB

Product 0.73 Database 0.57 Data 0.87

`ArcadeDB` [139] is a multi-model database which shares the same model of `OrientDB` and is licensed under `Apache License 2.0`. It comes with `Docker` and `Kubernetes` support and offers the `ArcadeDB Studio` as a visual tool to facilitate query writing and graph exploration. It can be used through `JVM`-compatible languages and also provides drivers (only a subset of operations are implemented); `Redis` (transient entries in the server, persistent entries in the database) and `MongoDB` (CRUD operations and queries). There is a high-availability module which enables replication, and when a server joins a cluster, an election process takes place (RAFT protocol). It supports `Gremlin` and partially supports `Cypher` through the `cypher-for-gremlin` transpiler. `GraphQL` is partially supported, enabling commands through `Java` and HTTP APIs and a `Postgres` driver. The `Postgres` protocol may be used, enabling the execution of queries through different programming language drivers.

3.2.4.6 BangDB

Product 0.77 Database 0.46 Data 0.58

`BangDB` [204] is a multi-model database under a `BSD` license. It supports representations such as key-value, documents, columns, large files, temporal and graphs. Regarding graph storage, it supports `Cypher` queries and provides data science libraries. A community version is offered, as well as priced enterprise versions (basic and pro) which offer a dashboard and an array of domain-specific applications. Its cloud data platform is called `Ampere`, which provides a set of

composable workflows to enable a no-code approach to implementing functionality and offers a REST API. BangDB integrates its graph functionality with a streaming component, enabling the creation of graphs as data streams into the system. It also offers CEP (complex event processing) on the data to find patterns/anomalies in continuous manner.

3.2.4.7 IBM System G

Product 0.31 Database 0.18 Data 0.32

IBM System G [105] is a suite of functionalities, able to support the property graph (with Gremlin) as well as RDF (though we did not find comments on RDF-specific query languages). It is comprised of proprietary components as well as open-source (TinkerPop and HBase) and comes with visual query capabilities, providing visual feedback into query building and result analysis to ease the debugging process. ACID transactions are supported and the graph is represented in its native store with a data structure similar to compressed sparse vectors, using offsets to delimit, for each graph element, the latest and earliest temporal information of the element. Its Query Manager is responsible for the communication with the distributed shards to enable query execution over the distributed graph. It offers gShell, a bash-like client to manage graph stores. The IBM System G product has been discontinued[24].

3.2.4.8 MillenniumDB

Product 0.46 Database 0.11 Data 0.55

MillenniumDB in an open-source graph system [227] supporting property graphs, RDF and multilayered graph format. The system is actively being developed at IMFD Chile, a research institute focusing on fundamental data problems. MillenniumDB comes equipped with three query languages: GQL for property graph querying, SPARQL 1.1 for RDF, and its own proprietary Cypher-like language for the extended model they propose. The system itself uses classical relational architecture and index-organized storage with four permutations of edge data being stored. One novel aspect of the system is that it deploys worst-case optimal [223] join algorithms whenever possible, showing some perfomance benefits in join-intensive queries [226]. The system provides several access APIs, a console client and drivers for Python and JavaScript. It is under the GNU General Public License 2.0.

3.2.4.9 Oracle Spatial and Graph

Product 0.62 Database 0.93 Data 0.97

Oracle Spatial And Graph (OSG) [173] is a long-standing commercial product which has spatial and graph capabilities, among which the property graph model using PGQL and the RDF model with SPARQL. It also supports a feature-rich studio with notebook interpreters, shell user-interface and graph visualization. There are different Java APIs, one for the Oracle Spatial and Graph Property Graph, another for TinkerPop Blueprints and Database Property Graph.

3.2.4.10 OrientDB

Product 0.81 Database 0.50 Data 0.84

OrientDB [190] is a distributed (property model) graph database that supports TinkerPop and functions both as a graph (native) database and NoSQL document database as well, with a

[24] Now-invalid page: http://systemg.research.ibm.com/download.html

Community Edition licensed [174] under Apache License 2.0 and a commercial edition. There are drivers supporting OrientDB at least in the following languages: Clojure, Go, Java, JavaScript, .NET, Node.js, PHP, Python, R, Ruby and Scala. It supports sharding (horizontal scaling), has ACID support and offers an adapted SQL for querying.

3.2.4.11 Stardog

Product 0.62 Database 0.54 Data 0.74

Stardog [208] is a proprietary (with a limited-time free trial version and a paid Enterprise license) knowledge graph database with a graph model based on RDF and extensions to support the property graph model. It is horizontally-scalable, supports ACID operations, GraphQL, Gremlin and SPARQL for querying and introspection and may be programmed in Clojure, Groovy, Java, JavaScript, .NET and Spring. For exploration, it features Stardog Studio. We did not find its pricing scheme on its website. The commercial offering is based on cloud deployments relying on the Linux (Ubuntu) ecosystem.

3.2.4.12 SurrealDB

Product 0.85 Database 0.65 Data 0.79

SurrealDB [98] was built in Rust as a single library. It is a multi-model database to be used as both an embedded database library and as a database server which may be deployed in a distributed cluster. It uses tables with the added functionality of advanced nested fields and arrays, using inter-document record links to enable high-performance related queries without the overhead of multiple join operations. Its authors claim this storage design choice enables full graph database functionality using simple SQL extensions, with vertices represented as records. A Docker image is offered, with developers having access to different JavaScript client-side app tutorials as well as server-side language bindings (e.g. Golang and Rust). It supports ACID transactions, full-text indexing, graph querying, GraphQL and both structured and unstructured data. SurrealDB offers row-level permissions-based access control. Versioned data features are in development[25]. It is licensed under a custom Business Source License 1.1, claiming that the *Licensed Work* will be made available under the Apache License 2.0 in April 2027.

3.2.4.13 Virtuoso

Product 0.83 Database 0.39 Data 0.79

Virtuoso [63] is a multi-model database management system that supports relational as well as property graphs. It has an open-source [171] edition under the GPLv2 license as well as a proprietary enterprise edition. At least the following programming languages are supported: C/C++, C#, Java, JavaScript, .NET, PHP, Python, Ruby and Visual Basic. It supports horizontal scaling, has functionalities for interactive data exploration and supports SPARQL.

3.2.5 Alternative Data Models

We lastly note the following databases that have been used to represent graphs, though not having an explicit description of supporting the property graph model or RDF:

[25] See: https://surrealdb.com/features

3.2.5.1 Azure Cosmos DB

Product 0.62 Database 0.75 Data 0.71

`Azure Cosmos DB` [179] is a commercial database solution that is multi-model, globally-distributed, schema-agnostic, horizontally-scalable and fully supports ACID. It is classified as a `NoSQL` database, but the multi-model API is a relevant offering, for it can expose stored data for example as table rows (`Cassandra`), collections (`MongoDB`) and most importantly as graphs (`Gremlin`). It has connectors for `Java`, `.NET`, `Python` and `Xamarin`. It is also a fully-managed service with scalability, freeing developer resources from topics such as data centre deployments, software upgrades and other operations. An online repository of source code information (including SDKs, example applications, connectors and helper libraries) is available [152].

3.2.5.2 FaunaDB

Product 0.50 Database 0.61 Data 0.53

`FaunaDB` [67] is a distributed database platform targeting the modern cloud and container-centric environments. It has a custom `Fauna Query Language` (`FQL`) which operates on schema types such as documents, collections, indices, sets and databases. This language can be accessed through drivers in languages such as `Android`, `C#`, `Go`, `Java`, `JavaScript`, `Python`, `Ruby`, `Scala` and `Swift`. `FaunaDB` supports concurrency, ACID transactions and offers a RESTful HTTP API. As of March 2025, the Fauna Inc. services have been discontinued.[26]

3.2.5.3 Google Cayley

Product 0.62 Database 0.57 Data 0.42

`Google Cayley` is an open-source [81] database behind Google's Knowledge Graph, having been tested at least since 2014 (and it is the spiritual successor to `graphd` [148]). It is a community-driven database written in `Go`, including a REPL, a RESTful API, a query editor and visualizer. It supports `Gizmo` (query language inspired by `Gremlin`) and `GraphQL`. `Cayley` supports multiple storage back-ends such as `LevelDB`, `PostgreSQL`, `MongoDB` (distributed stores) and also an ephemeral in-memory storage. The ability to support ACID transactions is delegated to the underlying storage back-end. `Cayley` being distributed depends on the underlying storage being distributed as well.

3.2.5.4 HyperGraphDB

Product 0.38 Database 0.39 Data 0.38

`HyperGraphDB` [107] is as an open-source general purpose data storage mechanism. It is used to store hypergraphs (a graph generalization where an edge can join any number of vertices). The low-level storage is based on `BerkeleyDB` and is implemented in `Java`. Despite the description on the website mentioning distribution, the support for either distributed sharding or distributed replication is not provided in its current implementation [108] (only federating independent databases is possible). It has support for transactions with MVCC based conflict detection.

3.2.5.5 ObjectivityDB

Product 0.38 Database 0.32 Data 0.50

`Objectivity/DB` [169] is a database technology powering the massively scalable graph software

[26] See: `https://fauna-livid.vercel.app/blog/the-future-of-fauna`

platform ThingSpan (formerly known as InfiniteGraph). It is a fully-distributed database (able to scale horizontally) offering APIs in C++, C#, Java and Python. Objectivity/DB is described as a distributed object database, supporting many data models (among which highly complex and inter-related data). It is schema-based and licensing is defined on a case-by-case basis. At the time of writing, the websites of the company are not accessible and their domain appears to have expired.[27]

3.2.5.6 TypeDB

Product 0.92 Database 0.46 Data 0.63

TypeDB [182] is designed to be a database capable of natively expressing inheritance hierarchies and dependencies. This database is offered under GNU Affero GENERAL PUBLIC LICENSE version 3.0, with its own query language TypeQL and is written in Java. It aims to implement conceptual data models without forcing them into predefined structures, which the authors claim to be a direct focus not supported in other databases. TypeDB also claims to focus on the challenges of polymorphism handling in graph databases due to a lack of a proper schema, leading to type hierarchies being implemented as a structure-less data labels. This database offers a cloud-agnostic interface to manage instances from providers such as Amazon, GCP or Azure. It also enables the use of clusters with multiple database instances to improve scalability. Access requires authentication and data encryption is supported. Three versions are offered: TypeDB Core which is open-source and available via Docker container and builds for Windows, Linux and macOS; TypeDB Enterprise which extends the former with security and high-availability features for running in production (as well as authentication); TypeDB Cloud which is built on Kubernetes and enables easy deployment across different teams and projects in different cloud providers.

Other notable mentions include the OQGRAPH [144] graph storage engine of MariaDB, developed to handle hierarchies and vertices with many connections and intended for retrieving hierarchical information such as graphs, routes and social relationships in SQL. We also note the following resources for further deepening of the aspects involved in the evolution of the study of graph databases [113, 32, 203, 126, 218, 54, 96, 191], covering aspects such as the internal graph representations, experimental comparisons, principles for querying and extracting information from the graph and designing graph databases to make use of distributed infrastructures.

3.3 Analysis and Discussion

Here, we draw the main findings about which features appear to be considered more and less relevant by the community and industry, and to what extent they are provided (or supported), either fully, or at least partially to some extent.

We perform this by looking into how each feature is addressed by the analyzed systems and categorize them according to how wide their support is, or lack thereof, across systems, as shown in Tables 13 and 14.

There are multiple angles of interest regarding the relationship between features and systems. We do not intend to be exhaustive as the table can be read across features over different support levels individually and cumulatively. Herein we refer the features in groups that we find to have more statistical impact w.r.t. assessing their relevance in the analyzed systems and how they can influence choice decisions:

[27] See: http://www.objectivity.com/

- **Features supported by majority of systems at a relevant level:**.
  A fair number of features are fully supported by a majority of systems (indicated by *full circle*). We address them according to dimension and class. The Product Dimension includes features related to adoption and deployment. In the Adoption class, active development is a key factor, with 60.78% of graph database systems demonstrating full engagement. Commercial support is fully provided by 64.71% of studied systems. Open source availability is observed in 56.86% of systems. In the Deployment class, work as a dedicated instance is a feature fully supported by 92% of systems. Operating on `Linux` is widely available, with 88.24% of systems providing full compatibility.
  The Database Dimension focuses on features that enhance the database experience. In the Convenience class, live backups are supported by 58% of systems. In the Distribution class, data distribution is fully implemented in 68% of the studied graph database systems. High availability is ensured by 66% of the systems. Query distribution is fully supported by 60.78% of systems. Replication support is a widely available feature, with 70% of systems providing full implementation. In the Software Development class, logging and auditing capabilities are fully implemented in 74% of systems. Documentation up-to-date is a crucial software development feature, with 70.59% of systems ensuring full maintenance.
  The Data Dimension addresses features related to data access, consistency, control, and security. In the Access class, CLI support is fully available in 72% of systems. GUI availability is a relatively widely adopted feature, with 60% of systems providing complete support. Graph-native data storage is fully integrated into 66% of systems. REST API functionality is available in 64% of systems. Query language support is a key feature, with 70% of systems providing full implementation. In the Consistency class, transaction support is usually considered a fundamental requirement, fully available in 84% of systems. In the Control class, schema support if fully provided in 54% of systems, while secondary indexes are widely supported, with full implementation in 74% of systems. In the Security class, authentication is a critical security feature, with 72% of systems ensuring full implementation. Authorization mechanisms are fully supported in 66% of the studied systems.
  If we consider significant support (*three quarter-filled circle*) as relevant, there is another feature that reaches support from a majority of systems: Operating on `Windows`.
  If we further consider partial support (*half-filled circle*) as relevant for a decision, then additional features become included in those that are supported by a majority of systems, all being: Live community, Operating on `Windows` Data types defined, Read committed transaction, and Constraints.

- **Features at most only partially supported in a majority of systems**: We can also analyze the table by checking what are the features that are lacking widespread relevant support, i.e. those where a majority of systems offer no support (*empty circle*), only limited support (*quarter-filled circle*) or partial support (*half-filled circles*). Conversely, we could regard these as the features where significant or full support is only offered by a minority of systems.
  Such features are those that may be considered not essential to provide, or more difficult to implement by most of the community. We identified them as those features that in a majority of the systems analyzed do not go beyond being partially supported.
  Considering the Product dimension, these include Live community (66.67%), Pricing (56.89%), Trendiness (64.71%), Work as embedded (66%), Testing in-memory version (64.7%), and SaaS offering (51.22%),
  Considering the Database dimension, they include Automatic updates (84%), Client side

caching (85.71%), Data versioning support (78%), Cluster Re-balancing (78%), Data types defined (54%), Object-Graph Mapper (58%), Reactive programming (78%),

In the Data dimension, we identify Multi-database (64%), Granular locking (74%), Multiple isolation levels (76%), Read committed transaction (56%), Constraints (56%), Server side procedures (61.22%), Triggers (67.34%), Data encryption (74%).

If instead we consider a more restrictive threshold of only having limited supported (*quarter-filled circle*) to identify features of less interest from the systems analyzed, this set is reduced to:

Considering the Product dimension, they still include Pricing (55.17%), Trendiness (50.98%), Work as embedded (66.0%), Testing in-memory version (60.78%).

In the Database dimension, they include Automatic updates (84.0%), Client side caching (81.63%), Data versioning support (76.0%), Cluster Re-balancing (74.0%), Object-Graph Mapper (58.0%), and programming (76.0%).

Considering the Data dimension, those are Reactive Multi-database (64.0%), Granular locking (72.0%), Multiple isolation levels (64.0%), Read committed transaction (50.0%), Server side procedures (57.14%), Triggers (59.18%), and Data encryption (72.0%).

Finally, if we only consider features without any type of support whatsoever in a majority of systems, as an indicator of very low interest in, or high difficulty of, systems providing them, we still have the following set: Pricing (55.17%), Work as embedded (66.0%), Testing in-memory version (60.78%) in the Product Dimension; Automatic updates (84.0%), Client side caching (81.63%), Data versioning support (76.0%), Cluster Re-balancing (74.0%), Object-Graph Mapper (58.0%), Reactive programming (76.0%) in the Database dimension; and Multi-database (64.0%), Granular locking (72.0%), Multiple isolation levels (64.0%), Server side procedures (57.14%), Triggers (59.18%), Data encryption (72.0%) in the Data Dimension.

Figure 3 presents a triangular visualization of database system distribution across the Product-Database-Data space. Each database is projected onto the three evaluation dimensions (Product, Database, Data). The left corner corresponds to the Product dimension, the right corner to the Database dimension and the top corner to the Data dimension. Position shows the relative balance across dimensions; marker size reflects overall feature coverage; shape indicates the supported graph data model; color indicates licensing. It can be observed that there is a clustering of systems around the center of the triangle. This is due to many databases having a relative balance that is similar among the feature categories. There is a broadly balanced profile across the three dimensions. Points closer to a triangle vertex represent systems whose feature support is concentrated in that dimension (e.g., stronger product maturity/operational support vs. data-model expressiveness), highlighting specialized design emphases. Property graph systems exhibit a wider spread in the landscape, indicating higher heterogeneity in the balance between product maturity, operational database features, and data-model/query capabilities. In contrast, RDF systems cluster more tightly, suggesting a more uniform capability profile within this family.

Figure 4 shows a three-dimensional scatter plot of the database systems across the three dimensions, as well as pairwise 2D projections for each dimension pair. It can be observed that there is a greater diversity (in terms of graph model focus) of systems that score high on both Data and Product, with the lower-scoring ones being mostly focused on the property graph model (bottom left panel). The other two dimension combinations show similar patterns, highlighting a wider distribution of property graph focus across the score ranges (both low and high values), with the systems focusing on other models presenting higher scores.

Table 13 Summary of feature level of adoption across graph databases and systems (Part I).

Feature	○ None	◔ Limited	◓ Partial	◕ Significant	● Full
Active development	45.10%	0.00%	5.88%	0.00%	49.02%
Commercial support	35.29%	0.00%	0.00%	0.00%	64.71%
Live community	41.18%	3.92%	21.57%	3.92%	29.41%
Open source	39.22%	0.00%	3.92%	0.00%	56.86%
Pricing	55.17%	0.00%	1.72%	0.00%	31.03%
Trendiness	45.10%	0.00%	9.80%	0.00%	45.10%
Containerization	38.00%	0.00%	6.00%	0.00%	56.00%
Work as dedicated instance	8.00%	0.00%	0.00%	0.00%	92.00%
Work as embedded	66.00%	0.00%	0.00%	0.00%	34.00%
Testing in-memory version	60.78%	0.00%	3.92%	1.96%	31.37%
Operating on Linux	5.88%	0.00%	1.96%	1.96%	88.24%
Operating on Windows	28.00%	0.00%	12.00%	12.00%	48.00%
SaaS offering	48.78%	0.00%	2.44%	0.00%	46.34%
Automatic updates	84.00%	0.00%	0.00%	0.00%	16.00%
Client side caching	81.63%	0.00%	4.08%	0.00%	14.29%
Data versioning support	76.00%	0.00%	2.00%	0.00%	22.00%
Live backups	38.00%	0.00%	4.00%	0.00%	58.00%
Cluster re-balancing	74.00%	0.00%	4.00%	0.00%	22.00%
Data distribution	28.00%	2.00%	0.00%	2.00%	68.00%
High-availability	28.00%	0.00%	6.00%	0.00%	66.00%
Query distribution	25.49%	1.96%	7.84%	1.96%	60.78%
Replication support	26.00%	0.00%	4.00%	0.00%	70.00%
Data types defined	10.00%	0.00%	44.00%	0.00%	46.00%
Logging/Auditing	16.00%	0.00%	10.00%	0.00%	74.00%
Object-graph mapper	58.00%	0.00%	0.00%	0.00%	42.00%
Reactive programming	76.00%	0.00%	2.00%	0.00%	22.00%
Documentation up-to-date	13.73%	1.96%	7.84%	5.88%	70.59%

4 Related Work

Graph databases have been analysed and reviewed in the literature before, including specific domains such as health [228], industry [213], artificial intelligence [157], life sciences [38], social networks [118], or software engineering [138]. Graph databases are often included in the NoSQL database and Big data domains, on which there is also extensive analysis work [91, 78, 181, 51, 125]. Specifically, graph databases have been the subject of extensive research, covering various aspects such as system design, user-related features, scalability and transactions, among others. This section gives credit to, and addresses, other survey works in such areas.

Table 14 Summary of feature level of adoption across graph databases and systems (Part II).

Feature	○ None	◔ Limited	◓ Partial	◕ Significant	● Full
Binary protocol	46.00%	0.00%	0.00%	2.00%	52.00%
CLI	28.00%	0.00%	0.00%	0.00%	72.00%
GUI	38.00%	0.00%	2.00%	0.00%	60.00%
Multi-database	64.00%	0.00%	0.00%	0.00%	36.00%
Graph-native data	34.00%	0.00%	0.00%	0.00%	66.00%
REST API	34.00%	0.00%	2.00%	0.00%	64.00%
Query language	14.00%	0.00%	14.00%	2.00%	70.00%
Granular locking	72.00%	0.00%	2.00%	0.00%	26.00%
Multiple isolation levels	64.00%	0.00%	12.00%	0.00%	24.00%
Read committed transaction	48.00%	2.00%	6.00%	0.00%	44.00%
Transaction support	12.00%	0.00%	4.00%	0.00%	84.00%
Constraints	44.00%	0.00%	12.00%	2.00%	42.00%
Schema support	26.00%	0.00%	18.00%	2.00%	54.00%
Secondary indexes	26.00%	0.00%	0.00%	0.00%	74.00%
Server side procedures	57.14%	0.00%	4.08%	0.00%	38.78%
Triggers	59.18%	0.00%	8.16%	0.00%	32.65%
Authentication	26.00%	0.00%	2.00%	0.00%	72.00%
Authorization	30.00%	0.00%	4.00%	0.00%	66.00%
Data encryption	72.00%	0.00%	2.00%	0.00%	26.00%

Foundational aspects

Earlier works naturally tend to focus more on theoretical, formal and foundational aspects, with lower emphasis on system, scale and performance issues. An example of initial work is found in [14], laying the groundwork for graph databases. It offers a comprehensive overview of graph database modeling, focusing on data structures, query languages and integrity constraints, from foundational aspects to early developments. In light of such, it extensively reviews the developments at the time in graph database models, particularly their resurgence due to the need to manage graph-like information.

Another survey [117] provides an overview of the different types of graph databases, as well as their typical applications, while assessing and comparing their models based on a set of varied features, such as API and languages, property graph, hypergraph, query language, graph type.

More recently, the authors of [14] delved in more detail over query languages in [13, 11], providing additional insights on graph pattern matching and navigational expressions, and the study of graph structure, properties and motifs has been addressed in [188].

System design aspects

Works dedicated to system design aspects tend to address more core aspects, such as architectural and programmatic ones, of each concrete graph database system, e.g., whether it is centralized or distributed, how data is organized, how queries are executed.

A taxonomy of graph database systems is proposed in [24], covering fundamental categories,

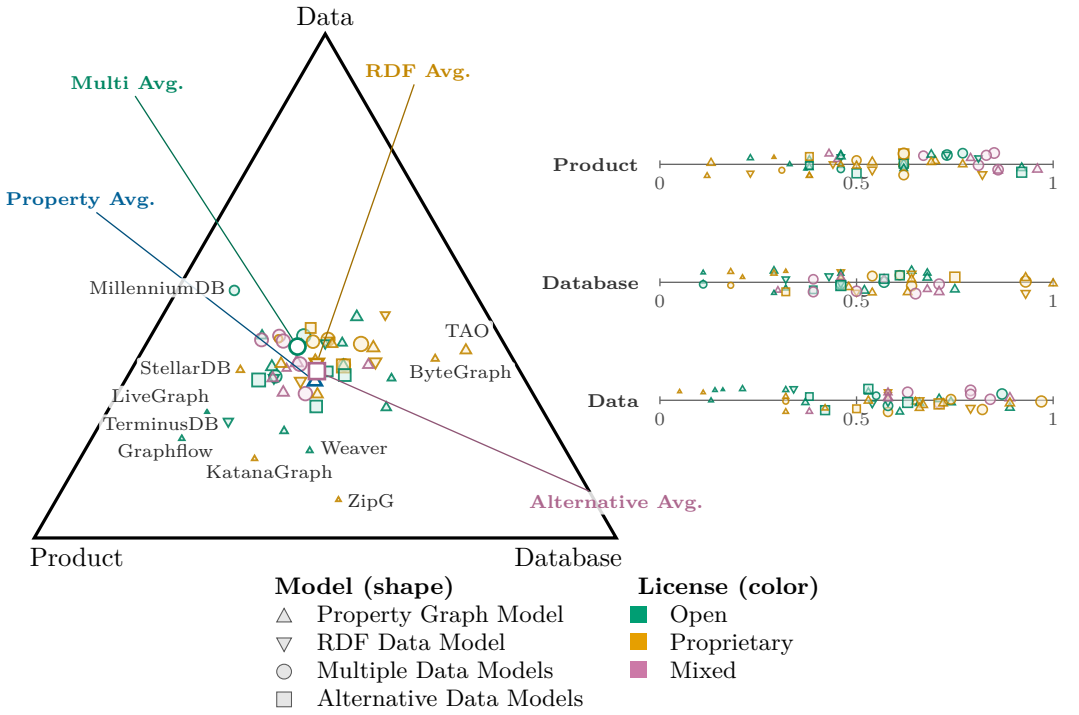

Figure 3 Database system distribution in the Product-Database-Data space.

associated graph models, data organization techniques, and aspects of data distribution and query execution. In [178], the authors review existing graph data computational techniques and research, offering insights into future research directions in graph database management.

The authors of [54] propose a model-driven, system-independent methodology for the design of graph databases with emphasis on minimizing the amount of data accesses to answer queries. Similarly, in [64], the authors propose a diagram for graph databases modeling, demonstrating its effectiveness and compatibility in various case studies.

Transactions and Concurrency Models

While not all graph storage solutions offer transactional semantics, there are several graph databases that do enforce transactional guarantees when manipulating data. This is also often addressed in broader surveys but has also been subject of some specific studies.

Transaction management in cloud-based graph databases is discussed in [127], including different levels of transaction support and concurrency control protocols, data distribution issues, and replication protocols. The work in [40] specifically addresses the challenges of efficiently executing large-scale, long-running read-write transactions (termed 'mammoth transactions') in graph databases. These transactions are prevalent in graph workloads but are not adequately supported by many concurrency control protocols and benchmarks.

Scalability Aspects

Scalability is one the most important aspects in graph databases and one of the most sought after properties when designing or selecting one, and thoroughly studied. Vertical scalability is relevant when processing graphs in machines with large memory and number of cores. Horizontal scaling is increasingly relevant both in storage and in processing, because in the Big data era, graph data

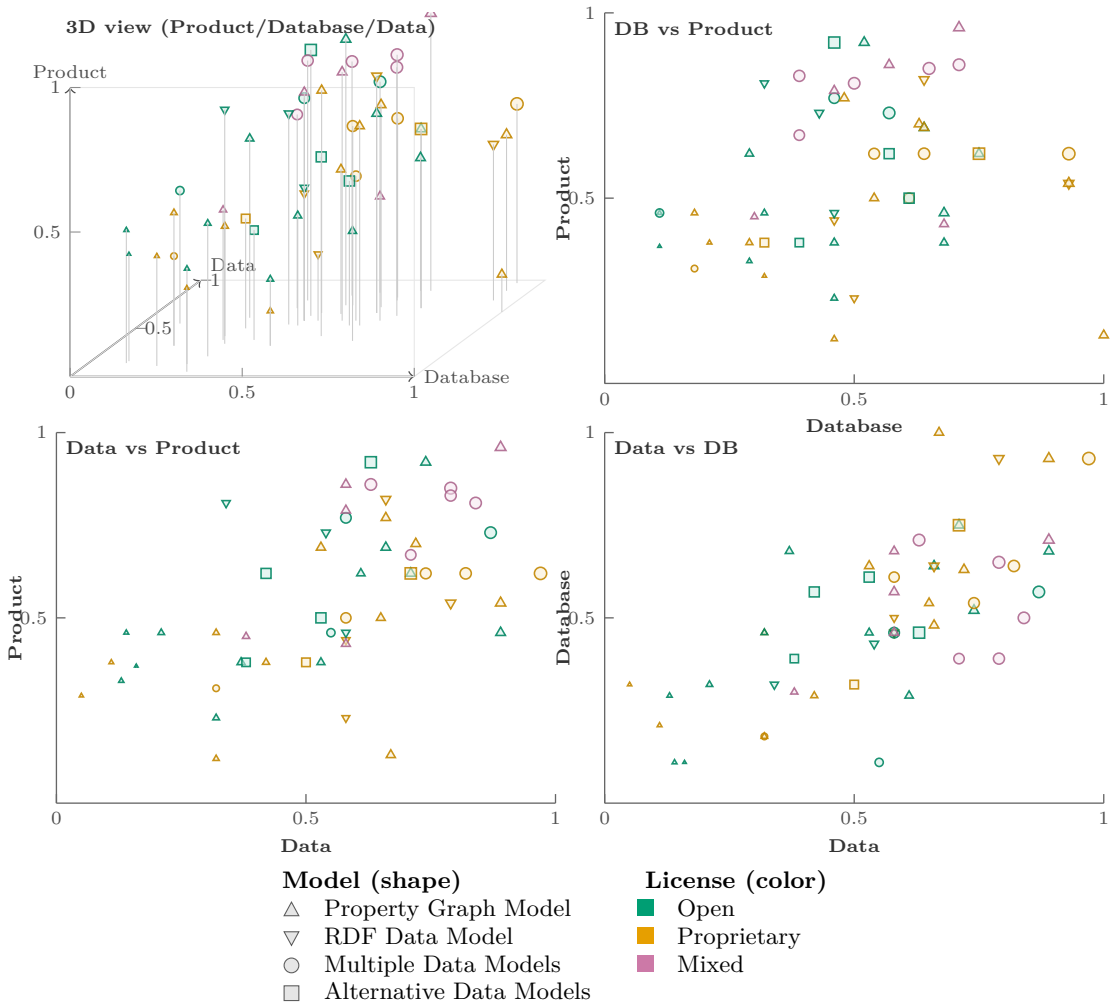

Figure 4 (Top-left) 3D perspective view of Product/Database/Data scores. (Other panels) Pairwise projections. Marker shape encodes model family; color encodes license; marker size encodes overall coverage. The 3D panel provides an overview of the joint distribution; the three 2D projections support accurate pairwise comparison.

can grow to huge sizes that surpass the memory and processing power of single, yet powerful, machines. Often, specific mechanisms such as partitioning and indexing influence the scalability of graph databases and graph query processing.

Scalability challenges in graph databases are discussed in [184], particularly in the context of large graphs and their querying in Big data environments. The authors note that when scaling over the network, these challenges are much harder in the case of graphs than in the case of simpler data models. The authors of [52] evaluate the scalability of the `ScaleGraph` library for large-scale graph analysis, providing insights into scalable graph processing. There is also an in-depth study on the scalability (and performance) of a specific set of graph databases in [157], including `Neo4j`, `JanusGraph`, `Memgraph`, `Nebula Graph`, and `TigerGraph`. Finally, scalability is also globally addressed in [22] with a comprehensive overview of the proposed `Graph Database Interface` (`GDI`), focusing on how it addresses several key challenges in the development of graph databases such as high performance and scalability, inspired by best practices from the

High-Performance Computing (HPC) landscape.

Specifically to partitioning, in [186], the authors discuss the challenges of scaling online social networks and propose a `Social Partitioning and Replication` middleware (`SPAR`) to minimize replication overhead and enhance scalability. Further, in [3], the authors review current problems and challenges related to partitioning and processing graph-structured data in distributed systems, focusing on scalability and efficiency.

Indexing has also been surveyed in particular. A key graph processing operation is the reachability query, which determines whether a path exists between two vertices in a graph, considering plain and edge-labeled graphs. These queries can be resource-intensive, often requiring traversal of extensive graph portions, especially in large graphs. To address this and strive for efficient query processing, considerable research has focused on developing indexing techniques, creating what are known as reachability indexes, which have been surveyed in [235] and [236].

Finally, the authors of [193] present an analysis of various optimization techniques for horizontal scaling in the context of Big data, which is relevant also in the domain of graph databases.

Other User Relevant Aspects

There are other important aspects when selecting a graph database such as it being open-source, free or commercial, the programming language API, benchmark results, hardware optimizations, etc. Thus, we address here survey work dedicated to such analysis.

An initial performance assessment of open-source graph databases is conducted in [147], with a study comparing the performance of 12 open-source graph databases using key graph algorithms on networks with up to 256 million edges, emphasizing the need for intuitive interfaces for both experts and novices in graph algorithm implementation. Referring back to [157], it describes the comparative performance evaluation of recent graph database systems such as `Neo4j`, `JanusGraph`, `Memgraph`, `Nebula Graph`, and `TigerGraph`. The work in [218] also evaluates several contemporary graph databases, both commercial and open-source, from a user-based (e.g., procurement, interface, pricing, support and maturity), system-based (e.g., query and storage distribution, indexing and transactional support), and empirical performance-based perspective with benchmarks. Referring again to the work in [117], its study also includes user related features such as freeness, portability, usability. Furthermore, a comprehensive evaluation of graph database systems with micro-benchmarks is described in [136], with more insights provided than those typically gathered with macro, application-like benchmarks.

Finally, tailoring specific hardware support, such as FPGAs, to non-relational databases (which includes graph databases), and its challenges and design decisions has been addressed in [47].

Summary

While there is extensive previous work on graph databases, some of it addresses very specific features or properties, often only of foundational nature (e.g., query languages, graph models), or just of distributed systems and scalability aspects (e.g., partitioning, replication, distribution, indexing), only transactional and concurrency issues, or exhaustive performance benchmarks of a very limited set of systems. Other issues such as open-source availability, pricing, cloud and containerized offering, community support, schema support, GUI and CLI tools, high availability, automatic updates and live backups, and logging/auditing are seldom analyzed and when so, only for a handful of systems.

We find that this work has a relevant place in this space by virtue of: i) its variety of features encompassed that cover commercial, architectural, system, programming model, scalability, and

user interface topics, ii) the breadth of features and mechanisms addressed in each topic , iii) and the extent and recency of the graph databases that are analysed according to our criteria.

5 Conclusion

Top researchers have said the future is big graphs [196], and in recent years we have continued to move towards this vision. Graph processing systems and graph databases have been designed, implemented and reinvented across academia and industry. As different industries have realized the opportunity of graph-based data manipulation, a sort of arms race was stimulated. Existing technology giants incorporated graph-oriented offerings in their cloud services while startups were created, simultaneously catering to business/developer needs while aiming to establish niches within the graph database ecosystem.

Throughout our research and engineering activities along the years, we observed these different initiatives manifesting, with commercial entities such as `Neo4j` producing educational material to explain graphs and help onboard new clients and developers [154, 167]. Other graph databases followed suit, with introductory material being used to ease clients/developers into the concepts and need for graph-focused database solutions. Parallel to this, the literature saw the addition of surveys on important topics pertaining to graph databases, such as graph database models and data management [14, 117, 178]; the intersection of industry and academia [213]; applied graph databases [118]; transaction performance and benchmarking [59, 40, 53]; feature-based graph database comparison [218]; and graph query languages [100, 13, 183], among others.

We analyzed a plethora of graph database systems and surveys [150, 137, 24] with the purpose of highlighting the most relevant intricacies within the ecosystem. At the same time, we presented an array of features to guide would-be developers and researchers in identifying the specific aspects of their use cases and the technologies available to them. Business and user needs call for the development of technology, which then stimulates the research vectors undertaken by researchers worldwide. This in turn feeds the entrepreneurial drive towards innovative graph database solutions. On an ending note, we believe graph databases will only continue to be more refined as more and more data exists, especially in the context of AI. The next ten years will paint an interesting scenario on the tendencies of existing commercial players and features.

Table 15 Description of graph database and systems analysis features (Part I).

Feature	Description
Active development	Score 1 if the GDB has been updated within three months of the reference date (2026-01-01). Score 0.5 if the GDB has been updated within 3–6 months of the reference date (2026-01-01). Otherwise, score 0.
Commercial support	Score 1 if there is a designated team that provides paid support for the GDB. Otherwise, score 0.
Live community	Score 1 if more than 75% of the issues addressed by users/customers/developers have been addressed. Score 0.75 if 50%–75% of the issues have been addressed. Score 0.5 if 25%–50% of the issues have been addressed. Score 0.25 if less than 25% of the issues have been addressed. Score 0 if no information is available.
Open source	Score 1 if the source code is freely available with an open-source license (e.g., Apache-2.0, GPLv3, ...). Score 0.5 if the source code is available but commercially restrictive. Otherwise, score 0.
Pricing	Score 1 if at least starting or approximate prices are available on the GDB web page, or the GDB has an open license. Otherwise, with no price information, score 0.
Trendiness	Score 1 if the overall trend is upward. Score 0.5 if it is broadly flat but non-zero. Score 0 if it is downward or flat at zero.
Containerization	Score 1 if a container image such as Docker is offered. Otherwise, score 0.
Work as dedicated instance	Score 1 if the GDB can be run as an individual instance. Otherwise, if it is available exclusively in embedded mode, score 0.
Work as embedded	Score 1 if it is possible to run the GDB embedded in the application (e.g., in the same JVM). Otherwise, score 0.
Testing in-memory version	Score 1 if there is an in-memory option of the GDB where data are stored in memory (not on disk) (e.g., for testing purposes). Otherwise, score 0.
Operating on Linux	Score 1 if Linux is explicitly supported. Score 0.75 if the GDB code is open source. Score 0.5 if a Docker image is provided. Otherwise, score 0.
Operating on Windows	Score 1 if Windows is explicitly supported. Score 0.75 if the GDB code is open source. Score 0.5 if a Docker image is provided. Otherwise, score 0.
SaaS offering	Score 1 if the provider offers its GDB as a service. Otherwise, score 0.
Automatic updates	Score 1 if there are mechanisms to seamlessly update the GDB automatically. Otherwise, score 0.
Client side caching	Score 1 if the GDB provides a local cache in an application memory on the client side. Otherwise, score 0.
Data versioning support	Score 1 if the GDB supports versioning of stored data. Otherwise, score 0.
Live backups	Score 1 if the GDB supports live backups. Otherwise, score 0.
Cluster re-balancing	Score 1 if the GDB is able to rebalance its original data distribution across the cluster efficiently. Otherwise, score 0.
Data distribution	Score 1 if the GDB enables data sharding (i.e., it can shard the data across several host servers). Otherwise, score 0.
High-availability	Score 1 if there are mechanisms to achieve high availability (including when the storage layer is graph-unaware, e.g., HBase, but provides HA mechanisms). Otherwise, score 0.
Query distribution	Score 1 if the GDB enables running a query over distributed data in a distributed manner (assigning the query to the correct partition and/or assembling partial results into a correct result). Otherwise, score 0.
Replication support	Score 1 if the GDB has replication mechanisms. Otherwise, score 0.
Data types defined	Score 1 if a composite data type is provided (in any form). Score 0.5 if only basic data types are enabled. Otherwise, if only one data type is available (e.g., all data as String), score 0.

Table 16 Description of graph database and systems analysis features (Part II).

Feature	Description
Logging/Auditing	Score 1 if the GDB logs important events. Otherwise, score 0.
Object-graph mapper	Score 1 if there exists a tool for automatic data conversion between the GDB and an object-oriented programming language. Otherwise, score 0.
Reactive programming	Score 1 if the GDB supports reactive streams that provide asynchronous processing. Otherwise, score 0.
Documentation up-to-date	Score 1 if the documentation has been updated for the last version of the GDB. Score 0.5 if it exists but has not been updated for the last version. Otherwise, score 0.
Binary protocol	Score 1 if the graph database provides an option to communicate via a binary protocol. Otherwise, score 0.
CLI	Score 1 if the GDB can receive commands via a CLI interface. Otherwise, score 0.
GUI	Score 1 if the GDB provides a visual interface. Otherwise, score 0.
Multi-database	Score 1 if the GDB is a multi-model database (i.e., you can also use it as, e.g., a document store, key/value store, or object store). Otherwise, score 0.
Graph-native data	Score 1 if the GDB is graph-native, i.e., designed to both store and process data as a graph. Otherwise, if the GDB provides only a graph abstraction over a key-value store, score 0.
REST API	Score 1 if a REST endpoint is offered to interact with the database. Otherwise, score 0.
Query language	Score 1 if the GDB can be queried with Cypher/Gremlin or any other widely accepted query language for GDBs. Score 0.5 if a non-standard/proprietary query language is provided. Otherwise, if no query language is provided, score 0.
Granular locking	Score 1 if the GDB allows locking of specific objects (usually nodes). Otherwise, if no specific-object locking mechanism is implemented, score 0.
Multiple isolation levels	Score 1 if more than one higher-level isolation level is available (i.e., other than “read uncommitted”). Score 0.5 if only one higher-level isolation level is available. Otherwise, if only “read uncommitted” is available, score 0.
Read committed transaction	Score 1 if a “read committed” transaction is available. Otherwise, score 0.
Transaction support	Score 1 if the GDB supports transactions. Otherwise, score 0.
Constraints	Score 1 if constraints can be set in the GDB. Score 0.5 if only uniqueness constraints are available. Otherwise, score 0.
Schema support	Score 1 if there is a direct tool for defining a schema. Score 0.5 if only constraints and triggers are available. Otherwise, score 0.
Secondary indexes	Score 1 if any index beyond the primary index is supported (e.g., single-property, composite, vertex-centric, full-text, etc.). Otherwise, score 0.
Server side procedures	Score 1 if the GDB enables server-side (stored) procedures. Otherwise, score 0.
Triggers	Score 1 if the GDB enables server-side events (triggers). Otherwise, score 0.
Authentication	Score 1 if the GDB supports some authentication method. Otherwise, score 0.
Authorization	Score 1 if the GDB provides any means for authorization (e.g., roles or privileges). Otherwise, score 0.
Data encryption	Score 1 if the GDB provides a possibility to encrypt stored data. Otherwise, score 0.

Table 17 Database active-development and lifecycle facts (Part 1 of 5)

Database	Evidence role	Evidence date	Supporting fact
`AgensGraph` [205, 206]	Active development	2025-11-28	GitHub non-trivial commit 93725359 on branch main: chore: fix warning in preproc build
`Alibaba Graph Database (GD)` [216]	No recent evidence	2025-03-13	GitHub release: Version 4.5.2. Newer post-cutoff activity was excluded from snapshot scoring.
`AllegroGraph` [76]	Active development	2025-12-17	The page lists dated release links in an unordered list under 'Archived AllegroGraph Releases', for example 'AllegroGraph 8.4.3 [2025-12-17]'. Also: Documentation: Documentation page for AllegroGraph displaying an update date used as...
`Altair Graph Lakehouse` [35]	Active development	2025	The page lists version tags together with 'Last pushed' timestamps. Also: Rebranding: This source is used as rebranding evidence, not as active-development evidence.
`Amazon Neptune` [9, 8, 21]	Active development	2025-12-18	This is strong public evidence of ongoing active development and is suitable for refresh-based date extraction relative to a reference date.
`HugeGraph` [133]	Active development	2025-11-16	GitHub release: Apache HugeGraph 1.7.0 (Release). Newer post-cutoff activity was excluded from snapshot scoring.
`ArangoDB` [199, 69]	Active development	2025-12-24	GitHub non-trivial commit 2c5721f3 on branch devel: Enable vector index tests for Go driver (#22205). Newer post-cutoff activity was excluded from snapshot scoring.
`ArcadeDB` [139]	Active development	2025-12-09	GitHub release: 25.11.1. Newer post-cutoff activity was excluded from snapshot scoring.
`Azure Cosmos DB` [152, 179]	No recent evidence	2023-03-28	GitHub repository activity timestamp
`BangDB` [204]	Active development	2025-12-06	GitHub non-trivial commit b9b1630c on branch master: Update install_bangdb.sh
`Blazegraph` [212, 211]	Discontinuation	–	Repository is archived. Also: Rebranding: W3 Semantic Web Standards wiki page stating 'Blazegraph (Formerly Bigdata®)' and t...
`BrightstarDB` [27]	No recent evidence	2023-05-31	GitHub repository activity timestamp

Table 18 Database active-development and lifecycle facts (Part 2 of 5)

Database	Evidence role	Evidence date	Supporting fact
`ByteGraph` [132, 131]	Active development	2026	Useful as indirect active-development evidence for 2026.
`ChronoGraph` [89, 88]	Active development	2025-11-03	GitHub non-trivial commit 2f7cbded on branch master: Merge pull request #3 from Txture/renovate/all-minor-updates. Newer post-cutoff activity was excluded from snap...
`Cray Graph Engine (CGE)` [189]	Active development	2021	Useful as indirect active-development evidence for 2021.
`DataStax Enterprise Graph (DSE)` [49, 48]	Active development	2026-01-19	The page includes release notes for 6.9.18 dated 2026-01-19, providing current public evidence that DSE is still maintained.
`Dgraph` [57, 58]	Active development	2025-12-12	GitHub release: v25.1.0. Newer post-cutoff activity was excluded from snapshot scoring.
`FaunaDB` [67, 68]	No recent evidence	2025-05-03	GitHub repository activity timestamp
`Fluree` [72, 73]	Active development	2025-12-17	GitHub non-trivial commit 9fded201 on branch main: cljfmt fix. Newer post-cutoff activity was excluded from snapshot scoring.
`G-Tran` [37, 36]	No recent evidence	2022-04-06	GitHub repository activity timestamp
`Gaffer` [80]	Discontinuation	–	Repository is archived.
`Galaxybase` [215]	Active development	2023-06-06	Official Galaxybase release-notes page.
`Google Cayley` [81]	Active development	2025-11-22	GitHub repository activity timestamp
`Graphflow` [119, 120]	Needs review	–	Github repo check returned HTTP 404.

Table 19 Database active-development and lifecycle facts (Part 3 of 5)

Database	Evidence role	Evidence date	Supporting fact
`HGraphDB` [232]	Active development	2025-12-28	GitHub non-trivial commit 22085d9f on branch master: Bump org.apache.maven.plugins:maven-release-plugin from 3.1.1 to 3.2.0 (#452). Newer post-cutoff activity was e...
`HyperGraphDB` [108, 107]	Needs review	–	It lists a latest release labeled 1.3, but no clear recent update date was visible during checking. Manual review is still needed.
`IBM System G` [105]	Discontinuation	2014	The URL is no longer reliably accessible; secondary sources list System G with end year 2014 and current survey material treats the product as discontinued.
`JanusGraph` [18, 111, 157]	Active development	2025-11-21	GitHub repository activity timestamp
`KatanaGraph` [121]	No recent evidence	2022-08-30	GitHub release: v0.4.0
`Kuzu` [128]	Discontinuation	–	Repository is archived.
`LiveGraph` [237, 238]	No recent evidence	2021-04-05	GitHub repository activity timestamp
`Memgraph` [140]	Active development	2025-12-23	GitHub release: v3.7.2. Newer post-cutoff activity was excluded from snapshot scoring.
`MillenniumDB` [226, 227]	Active development	2025-12-16	GitHub non-trivial commit 87743272 on branch dev: Feat/ssl http websocket (#101). Newer post-cutoff activity was excluded from snapshot scoring.
`Nebula Graph` [224, 160]	Active development	2025-10-22	GitHub repository activity timestamp
`Neo4j` [24, 100, 166, 164, 163, 165, 69, 184, 154, 229, 157, 185, 143]	Active development	2025-10-23	GitHub non-trivial commit 343dcdae on branch 2025.10: CVE-2025-11602 Fix init of self-allocated Bolt BitMask (). Newer post-cutoff activity was excluded from snapsh...
`Objectivity/DB` [169]	Needs review	–	This entry is therefore kept as manual-review evidence rather than being labeled active or discontinued.

Table 20 Database active-development and lifecycle facts (Part 4 of 5)

Database	Evidence role	Evidence date	Supporting fact
Ontotext GraphDB [170]	Active development	–	The page documents the current 11.3 release series and provides strong public evidence of ongoing active development.
Oracle Spatial And Graph (OSG) [173]	Active development	–	It states that Oracle Graph Server and Client is released four times a year and lists new features for release 26.1.
OrientDB [24, 122, 190, 174, 69, 184]	Active development	2025-12-24	GitHub release: 3.2.48. Newer post-cutoff activity was excluded from snapshot scoring.
PandaDB [201, 202]	No recent evidence	2022-06-04	GitHub repository activity timestamp
RedisGraph [141, 34]	No recent evidence	2025-07-21	GitHub non-trivial commit 5784cb8b on branch master: Update SSPLv1.txt
SAP Hana Graph [104, 197, 194]	Active development	2025	Useful as active-development evidence for graph capabilities delivered as part of SAP HANA Cloud.
Sparksee [207, 145]	Active development	2024-09-04	This is reasonable active-development evidence, but it is weaker than a dedicated release-notes page.
Stardog [208]	Active development	–	Official Stardog release-notes page.
StellarDB [209]	Active development	2026-01-21	Useful as public active-development evidence.
SurrealDB [98]	Active development	2025-12-30	GitHub non-trivial commit 9ac7269b on branch main: Reduce instrumentation overhead in query record collectors (#6710). Newer post-cutoff activity was excluded from...
TAO [39, 30]	Active development	–	This Meta Research publication presents transaction support layered on TAO, providing direct evidence that TAO was still being actively evolved.
TerminusDB [220]	Active development	2025-12-16	GitHub release: TerminusDB Server v12.0.2. Newer post-cutoff activity was excluded from snapshot scoring.

Table 21 Database active-development and lifecycle facts (Part 5 of 5)

Database	Evidence role	Evidence date	Supporting fact
TigerGraph [214, 56]	Active development	2026-03-11	Official TigerGraph Savanna release-notes page.
TypeDB [182]	Active development	2025-12-13	GitHub release: TypeDB 3.7.2. Newer post-cutoff activity was excluded from snapshot scoring.
Ultipa [217]	Active development	2025-08-11	Official Ultipa database release-notes page.
Virtuoso [24, 171, 63]	Active development	2025-10-15	GitHub release: Virtuoso Open Source Edition v7.2.16.1. Newer post-cutoff activity was excluded from snapshot scoring.
Weaver [61, 60]	No recent evidence	2016-11-14	GitHub repository activity timestamp
ZipG [124, 123]	No recent evidence	2021-09-01	GitHub repository activity timestamp

References

1 Resource Description Framework (RDF), 2014. [Accessed 17-December-2025].

2 G.V() – Gremlin Graph Database IDE and Visualization Tool. [Online; accessed 17-December-2025], 2024. URL: https://gdotv.com/.

3 H. Adoni, Tarik Nahhal, M. Krichen, B. Aghezzaf, and A. Elbyed. A survey of current challenges in partitioning and processing of graph-structured data in parallel and distributed systems. *Distributed and Parallel Databases*, 38:495 – 530, 2019. doi:10.1007/s10619-019-07276-9.

4 Rachit Agarwal, Anurag Khandelwal, and Ion Stoica. Succinct: Enabling queries on compressed data. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 337–350, 2015.

5 Waqas Ali, Muhammad Saleem, Bin Yao, Aidan Hogan, and Axel-Cyrille Ngonga Ngomo. A survey of RDF stores & SPARQL engines for querying knowledge graphs. *VLDB J.*, 31(3):1–26, 2022. URL: https://doi.org/10.1007/s00778-021-00711-3, doi:10.1007/S00778-021-00711-3.

6 Alibaba Cloud. Alibaba Graph Database. [Online; accessed 17-December-2025], 2020. URL: https://cn.aliyun.com/product/gdb.

7 Bob Alverson, Edwin Froese, Larry Kaplan, and Duncan Roweth. Cray XC series network. *Cray Inc., White Paper WP-Aries01-1112*, 2012.

8 Amazon, Inc. Amazon Neptune Samples - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/aws-samples/amazon-neptune-samples.

9 Amazon, Inc. Amazon Neptune Documentation - Use Neptune graph notebooks to get started quickly. [Online; accessed 17-December-2025], 2021. URL: https://docs.aws.amazon.com/neptune/latest/userguide/graph-notebooks.html.

10 Renzo Angles. The Property Graph Database Model. [Online; accessed 17-December-2025], 2018. URL: http://ceur-ws.org/Vol-2100/paper26.pdf.

11 Renzo Angles, Marcelo Arenas, Pablo Barceló, Peter Boncz, George Fletcher, Claudio Gutierrez, Tobias Lindaaker, Marcus Paradies, Stefan Plantikow, Juan Sequeda, et al. G-CORE: A core for future graph query languages. In *Proceedings of the 2018 International Conference on Management of Data*, pages 1421–1432, 2018.

12 Renzo Angles, Marcelo Arenas, Pablo Barceló, Peter A. Boncz, George H. L. Fletcher, Claudio Gutierrez, Tobias Lindaaker, Marcus Paradies, Stefan Plantikow, Juan F. Sequeda, Oskar van Rest, and Hannes Voigt. G-CORE: A core for future graph query languages. In *SIGMOD Conference*, pages 1421–1432. ACM, 2018.

13 Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoč. Foundations of modern query languages for graph databases. *ACM Comput. Surv.*, 50(5), sep 2017. doi:10.1145/3104031.

14 Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Computing Surveys (CSUR)*, 40(1):1–39, 2008.

15 Renzo Angles, Aidan Hogan, Ora Lassila, Carlos Rojas, Daniel Schwabe, Pedro A. Szekely, and Domagoj Vrgoc. Multilayer graphs: a unified data model for graph databases. In Vasiliki Kalavri and Semih Salihoglu, editors, *GRADES-NDA '22: Proceedings of the 5th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA), Philadelphia, Pennsylvania, USA, 12 June 2022*, pages 11:1–11:6. ACM, 2022. doi:10.1145/3534540.3534696.

16 Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, SIGMOD '15, pages 1383–1394, New York, NY, USA, 2015. ACM. URL: http://doi.acm.org/10.1145/2723372.2742797, doi:10.1145/2723372.2742797.

17 Aurelius. Titan: Distributed Graph Database. [Online; accessed 17-December-2025], 2015. URL: http://titandb.io.

18 JanusGraph Authors. JanusGraph - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/JanusGraph/janusgraph.

19 Ramazan Ali Bahrami, Jayati Gulati, and Muhammad Abulaish. Efficient processing of SPARQL queries over GraphFrames. In *Proceedings of the International Conference on Web Intelligence*, pages 678–685, 2017.

20 Hannah Bast and Björn Buchhold. Qlever: A query engine for efficient sparql+text search. In Ee-Peng Lim, Marianne Winslett, Mark Sanderson, Ada Wai-Chee Fu, Jimeng Sun, J. Shane Culpepper, Eric Lo, Joyce C. Ho, Debora Donato, Rakesh Agrawal, Yu Zheng, Carlos Castillo, Aixin Sun, Vincent S. Tseng, and Chenliang Li, editors, *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, CIKM 2017, Singapore, November 06 - 10, 2017*, pages 647–656. ACM, 2017. doi:10.1145/3132847.3132921.

21 Bradley R Bebee, Daniel Choi, Ankit Gupta, Andi Gutmans, Ankesh Khandelwal, Yigit Kiran, Sainath Mallidi, Bruce McGaughy, Mike Personick, Karthik Rajan, et al. Amazon Neptune: Graph Data Management in the Cloud. [Online; accessed 17-December-2025], 2018. URL: https://pdfs.semanticscholar.org/7d53/9db059514f6e7f1a08239031cdd517a106c6.pdf.

22 Maciej Besta, Robert Gerstenberger, Marc Fischer, Michal Podstawski, Nils Blach, Berke Egeli, Georgy Mitenkov, Wojciech Chlapek, Marek Michalewicz, Hubert Niewiadomski, Juergen Mueller, and Torsten Hoefler. The graph database interface: Scaling online transactional and analytical graph workloads to hundreds of thousands of cores. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '23, New York,

NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3581784.3607068.

23 Maciej Besta, Dimitri Stanojevic, Tijana Zivic, Jagpreet Singh, Maurice Hoerold, and Torsten Hoefler. Log (graph) a near-optimal high-performance graph representation. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques*, pages 1–13, 2018.

24 Besta, Maciej and Gerstenberger, Robert and Peter, Emanuel and Fischer, Marc and Podstawski, Michał and Barthels, Claude and Alonso, Gustavo and Hoefler, Torsten. Demystifying Graph Databases: Analysis and Taxonomy of Data Organization, System Designs, and Graph Queries. *ACM Comput. Surv.*, 56(2), September 2023. doi:10.1145/3604932.

25 Runhua Bian, Liqiang Zhang, Jinxin Liu, Jiacheng Zhang, Jianong Zhong, Jiahao Gu, Hao Guo, Zhihong Guo, Yunhao Li, Fenghao Zhang, et al. {Discard-Based} garbage collection for distributed {Log-Structured} storage systems in {ByteDance}. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, pages 493–507, 2026.

26 Paolo Boldi and Sebastiano Vigna. The WebGraph Framework II: Codes For The World-Wide Web. In *2004 Data Compression Conference (DCC 2004), 23-25 March 2004, Snowbird, UT, USA*, page 528. IEEE Computer Society, 2004. doi:10.1109/DCC.2004.1281504.

27 BrightstarDB. BrightstarDB - Source Code. [Online; accessed 17-December-2025], 2015. URL: https://github.com/BrightstarDB/BrightstarDB.

28 Nieves R. Brisaboa, Ana Cerdeira-Pena, Guillermo de Bernardo, and Gonzalo Navarro. Compressed representation of dynamic binary relations with applications. *Information Systems*, 69:106–123, 2017.

29 Willem Broekema, Mohamed Elzarei, Ora Lassila, Carlos-Manuel López-Enríquez, Marcin Neyman, Florian Schmedding, Michael Schmidt, Andreas Steigmiller, Geo Varkey, Gregory Todd Williams, and Amanda Xiang. openCypher Queries over Combined RDF and LPG Data in Amazon Neptune. In Lorena Etcheverry, Vanessa López Garcia, Francesco Osborne, and Romana Pernisch, editors, *Proceedings of the ISWC 2024 Posters, Demos and Industry Tracks: From Novel Ideas to Industrial Practice co-located with 23rd International Semantic Web Conference (ISWC 2024), Hanover, Maryland, USA, November 11-15, 2024*, volume 3828 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2024. URL: https://ceur-ws.org/Vol-3828/paper44.pdf.

30 Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry Li, et al. {TAO}:{Facebook's} distributed data store for the social graph. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 49–60, 2013.

31 Chris Buckley and Alan F Lewit. Optimization of inverted vector searches. In *Proceedings of the 8th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 97–110, 1985.

32 Mike Buerli and CPSL Obispo. The current state of graph databases. *Department of Computer Science, Cal Poly San Luis Obispo*, 32(3):67–83, 2012.

33 Aydin Buluç, Jeremy T Fineman, Matteo Frigo, John R Gilbert, and Charles E Leiserson. Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks. In *Proceedings of the twenty-first annual symposium on Parallelism in algorithms and architectures*, pages 233–244, 2009.

34 Pieter Cailliau, Tim Davis, Vijay Gadepally, Jeremy Kepner, Roi Lipman, Jeffrey Lovitz, and Keren Ouaknine. RedisGraph GraphBLAS Enabled Graph Database. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 285–286. IEEE, 2019.

35 Cambridge Semantics. AnzoGraph® DB. [Online; accessed 17-December-2025], 2020. URL: https://www.cambridgesemantics.com/anzograph/.

36 Hongzhi Chen, Changji Li, Chenguang Zheng, Chenghuan Huang, Juncheng Fang, James Cheng, and Jian Zhang. G-Tran - Source Code. [Online, GitHub; accessed 17-December-2025], 2022. URL: https://github.com/yaobaiwei/GTran.

37 Hongzhi Chen, Changji Li, Chenguang Zheng, Chenghuan Huang, Juncheng Fang, James Cheng, and Jian Zhang. G-tran: a high performance distributed graph database with a decentralized architecture. *Proceedings of the VLDB Endowment*, 15(11):2545–2558, 2022.

38 Jiaoyan Chen, Hang Dong, Janna Hastings, Ernesto Jiménez-Ruiz, Vanessa López, Pierre Monnin, Catia Pesquita, Petr Škoda, and Valentina Tamma. Knowledge graphs for the life sciences: Recent developments, challenges and opportunities. *Schloss Dagstuhl – Leibniz-Zentrum für Informatik*, 2023. doi:10.4230/TGDK.1.1.5.

39 Audrey Cheng, Xiao Shi, Lu Pan, Anthony Simpson, Neil Wheaton, Shilpa Lawande, Nathan Bronson, Peter Bailis, Natacha Crooks, and Ion Stoica. RAMP-TAO: layering atomic transactions on Facebook's online TAO data store. *Proceedings of the VLDB Endowment*, 14(12):3014–3027, 2021.

40 Audrey Cheng, Jack Waudby, Natacha Crooks, Hugo Firth, and Ion Stoica. Mammoths are slow: The overlooked transactions of graph data. *Proceedings of the VLDB Endowment*, 17(4):904–911, 2023. doi:10.14778/3636218.3636241.

41 Avery Ching. Scaling Apache Giraph to a trillion edges. *Facebook Engineering Blog*, page 25, 2013.

42 Miguel E Coimbra, Joana Hrotkó, Alexandre P Francisco, Luís MS Russo, Guillermo de Bernardo, Susana Ladra, and Gonzalo Navarro. A practical succinct dynamic graph representation. *Information and Computation*, 285:104862, 2022.

43 Miguel E Coimbra, Lucie Svitáková, Domagoj Vrgoč, Alexandre P Francisco, and Luís Veiga. Survey: Graph databases. *arXiv preprint arXiv:2505.24758*, 2025.

44 Douglas Comer. Ubiquitous b-tree. *ACM Computing Surveys (CSUR)*, 11(2):121–137, 1979.

45 Mariano P. Consens and Alberto O. Mendelzon. Graphlog: a visual formalism for real life recursion. In Daniel J. Rosenkrantz and Yehoshua Sagiv,

editors, *Proceedings of the Ninth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, April 2-4, 1990, Nashville, Tennessee, USA*, pages 404–416. ACM Press, 1990. doi:10.1145/298514.298591.

46 Isabel F. Cruz, Alberto O. Mendelzon, and Peter T. Wood. A Graphical Query Language Supporting Recursion. In Umeshwar Dayal and Irving L. Traiger, editors, *Proceedings of the Association for Computing Machinery Special Interest Group on Management of Data 1987 Annual Conference, San Francisco, CA, USA, May 27-29, 1987*, pages 323–330. ACM Press, 1987. doi:10.1145/38713.38749.

47 Jonas Dann, Daniel Ritter, and Holger Fröning. Non-relational databases on fpgas: Survey, design decisions, challenges. *ACM Comput. Surv.*, 55(11), feb 2023. doi:10.1145/3568990.

48 DataStax, Inc. DataStax Enterprise Graph (DSE). [Online; accessed 17-December-2025], 2016. URL: https://docs.datastax.com/en/dse/6.8/dse-dev/datastax_enterprise/graph/graphTOC.html.

49 DataStax, Inc. DataStax Enterprise Graph (DSE) - Release Notes. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/datastax/release-notes/tree/master.

50 Roshan Dathathri, Gurbinder Gill, Loc Hoang, Hoang-Vu Dang, Alex Brooks, Nikoli Dryden, Marc Snir, and Keshav Pingali. Gluon: A communication-optimizing substrate for distributed heterogeneous graph analytics. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 752–768, 2018.

51 Ali Davoudian, Liu Chen, and Mengchi Liu. A survey on nosql stores. *ACM Comput. Surv.*, 51(2):43, 2018. doi:10.1145/3158661.

52 Miyuru Dayarathna, Charuwat Houngkaew, Hidefumi Ogata, and T. Suzumura. Scalable performance of ScaleGraph for large scale graph analysis. *2012 19th International Conference on High Performance Computing*, pages 1–9, 2012. doi:10.1109/HiPC.2012.6507498.

53 Miyuru Dayarathna and Toyotaro Suzumura. Benchmarking graph data management and processing systems: A survey. *CoRR*, abs/2005.12873, 2020. URL: https://arxiv.org/abs/2005.12873, arXiv:2005.12873.

54 Roberto De Virgilio, Antonio Maccioni, and Riccardo Torlone. Model-driven design of graph databases. In *International Conference on Conceptual Modeling*, pages 172–185. Springer, 2014.

55 Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, et al. Graph pattern matching in gql and sql/pgq. In *Proceedings of the 2022 International Conference on Management of Data*, pages 2246–2258, 2022.

56 Alin Deutsch, Yu Xu, Mingxi Wu, and Victor Lee. TigerGraph: A Native MPP Graph Database. *arXiv preprint arXiv:1901.08248*, 2019.

57 Dgraph Labs, Inc. A Tour of Dgraph. [Online; accessed 17-December-2025], April 2020. URL: https://dgraph.io/docs/query-language/.

58 Dgraph Labs, Inc. Dgraph - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/dgraph-io/dgraph.

59 David Dominguez-Sal, Peter Urbón-Bayes, Aleix Giménez-Vanó, Sergio Gómez-Villamor, Norbert Martínez-Bazan, and Josep-Lluis Larriba-Pey. Survey of graph database performance on the hpc scalable graph analysis benchmark. In *International Conference on Web-Age Information Management*, pages 37–48. Springer, 2010.

60 Ayush Dubey. Weaver - Source Code. [Online, GitHub; accessed 17-December-2025], 2016. URL: https://github.com/dubey/weaver.

61 Ayush Dubey, Greg D. Hill, Robert Escriva, and Emin Gün Sirer. Weaver: A High-Performance, Transactional Graph Database Based on Refinable Timestamps. *PVLDB*, 9(11):852–863, 2016. URL: http://www.vldb.org/pvldb/vol9/p852-dubey.pdf.

62 Peter Eisentraut. SQL Property Graph Queries (SQL/PGQ), 2024. URL: https://www.postgresql.org/message-id/a855795d-e697-4fa5-8698-d20122126567%40eisentraut.org.

63 Orri Erling. Virtuoso, a Hybrid RDBMS/Graph Column Store. *IEEE Data Eng. Bull.*, 35(1):3–8, 2012.

64 Simone Erven et al. Designing graph databases with graphed. *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 2574–2577, 2019. doi:10.1109/ICDE.2019.00266.

65 Facebook Database Engineering Team. RocksDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2012. URL: https://github.com/facebook/rocksdb.

66 Facebook, Inc. GraphQL. [Online, GitHub; accessed 17-December-2025], October 2016.

67 Fauna, Inc. FaunaDB | The database built for serverless, featuring native GraphQL. [Online; accessed 17-December-2025], 2020. URL: https://fauna.com/.

68 Fauna, Inc. FaunaDB - Source Code. [Online; accessed 17-December-2025], 2025. URL: https://github.com/fauna/faunadb.

69 Diogo Fernandes and Jorge Bernardino. Graph Databases Comparison: AllegroGraph, ArangoDB, InfiniteGraph, Neo4J, and OrientDB. In Jorge Bernardino and Christoph Quix, editors, *Proceedings of the 7th International Conference on Data Science, Technology and Applications, DATA 2018, Porto, Portugal, July 26-28, 2018.*, pages 373–380. SciTePress, 2018. doi:10.5220/0006910203730380.

70 Mary F. Fernandez, Daniela Florescu, Alon Y. Levy, and Dan Suciu. A Query Language for a Web-Site Management System. *SIGMOD Rec.*, 26(3):4–11, 1997. doi:10.1145/262762.262763.

71 George Fletcher, Jan Hidders, and Josep Lluís Larriba-Pey. *Graph Data Management: Fundamental Issues and Recent Developments*. Springer International Publishing, 2018. doi:10.1007/978-3-319-96193-4.

72 Fluree. Fluree. [Online, Homepage; accessed 17-December-2025], 2023. URL: https://flur.ee/.

73 Fluree. Fluree - Source Code. [Online, GitHub; accessed 17-December-2025], 2023. URL: https://github.com/fluree/db.

74 The Apache Software Foundation. Apache TinkerPop. [Online; accessed 17-December-2025], 2019. URL: https://tinkerpop.apache.org/.

75 Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. Cypher: An evolving query language for property graphs. In *Proceedings of the 2018 International Conference on Management of Data*, pages 1433–1445, 2018.

76 Franz Inc. AllegroGraph. [Online; accessed 17-December-2025], 1984. URL: https://franz.com/agraph/allegrograph/.

77 Per Fuchs, Domagoj Margan, and Jana Giceva. Sortledton: a universal, transactional graph data structure. *Proceedings of the VLDB Endowment*, 15(6):1173–1186, 2022.

78 Santhosh Kumar Gajendran. A survey on nosql databases. Technical report, University of Illinois, 2012.

79 Lars George. *HBase - The Definitive Guide: Random Access to Your Planet-Size Data*. O'Reilly, 2011. URL: http://www.oreilly.de/catalog/9781449396107/index.html.

80 GHCQ. Gaffer - Source Code. [Online, GitHub; accessed 17-December-2025], 2022. URL: https://github.com/gchq/Gaffer.

81 Google. Google Cayley - Source Code. [Online, GitHub; accessed 17-December-2025], 2017. URL: https://github.com/cayleygraph/cayley.

82 Google Spanner. Spanner Graph and ISO standards, 2024. URL: https://docs.cloud.google.com/spanner/docs/graph/iso-standards.

83 Goetz Graefe et al. Modern b-tree techniques. *Foundations and Trends® in Databases*, 3(4):203–402, 2011.

84 John Grant and Francesco Parisi. On measuring inconsistency in graph databases with regular path constraints. *Artificial Intelligence*, 335:104197, 2024. URL: https://www.sciencedirect.com/science/article/pii/S0004370224001334, doi:10.1016/j.artint.2024.104197.

85 Alastair Green, Martin Junghanns, Max Kießling, Tobias Lindaaker, Stefan Plantikow, and Petra Selmer. openCypher: New Directions in Property Graph Querying. In *EDBT*, pages 520–523, 2018. URL: https://dbs.uni-leipzig.de/file/opencypher.pdf.

86 Andrey Gubichev and Manuel Then. Graph Pattern Matching: Do We Have to Reinvent the Wheel? In *Proceedings of Workshop on GRAph Data management Experiences and Systems*, pages 1–7, 2014.

87 Claudio Gutierrez and Juan F. Sequeda. Knowledge graphs. *Commun. ACM*, 64(3):96–104, feb 2021. doi:10.1145/3418294.

88 Martin Haeusler. ChronoGraph - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/MartinHaeusler/chronos.

89 Martin Haeusler, Thomas Trojer, Johannes Kessler, Matthias Farwick, Emmanuel Nowakowski, and Ruth Breu. ChronoGraph: A Versioned TinkerPop Graph Database. In *International Conference on Data Management Technologies and Applications*, pages 237–260. Springer, 2017.

90 Guðmundur Jón Halldórsson. *Apache Accumulo for Developers*. Packt Publishing, 2013.

91 Jing Han, Haihong E, Guan Le, and Jian Du. Survey on nosql database. In *2011 6th International Conference on Pervasive Computing and Applications*, pages 363–366, 2011. doi:10.1109/ICPCA.2011.6106531.

92 Steve Harris and Andy Seaborne. SPARQL 1.1 Query Language. W3C Recommendation, 2013. URL: http://www.w3.org/TR/sparql11-query/.

93 Olaf Hartig. Reconciliation of RDF* and property graphs. *arXiv preprint arXiv:1409.3288*, 2014.

94 Olaf Hartig and Jorge Pérez. An initial analysis of Facebook's GraphQL language. [Online; accessed 17-December-2025], 2017. URL: http://repositorio.uchile.cl/bitstream/handle/2250/169110/An-initial-analysis-of-facebooks-GraphQL-language.pdf?sequence=1&isAllowed=y.

95 Olaf Hartig and Jorge Pérez. Semantics and complexity of GraphQL. In *Proceedings of the 2018 World Wide Web Conference*, pages 1155–1164, 2018.

96 Charles E Henderson. System and method for creating, deploying, integrating, and distributing nodes in a grid of distributed graph databases, 2014. US Patent 8,775,476.

97 Daniel Hernández, Aidan Hogan, and Markus Krötzsch. Reifying RDF: what works well with wikidata? In Thorsten Liebig and Achille Fokoue, editors, *Proceedings of the 11th International Workshop on Scalable Semantic Web Knowledge Base Systems co-located with 14th International Semantic Web Conference (ISWC 2015), Bethlehem, PA, USA, October 11, 2015*, volume 1457 of *CEUR Workshop Proceedings*, pages 32–47. CEUR-WS.org, 2015. URL: https://ceur-ws.org/Vol-1457/SSWS2015_paper3.pdf.

98 Tobie Hitchcock, Rushmore Mushambi, Emmanuel Keller, Przemyslaw Kaznowski, et al. SurrealDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2023. URL: https://github.com/surrealdb/surrealdb.

99 Loc Hoang, Roshan Dathathri, Gurbinder Gill, and Keshav Pingali. Cusp: A customizable streaming edge partitioner for distributed graph analytics. *ACM SIGOPS Operating Systems Review*, 55(1):47–60, 2021.

100 Florian Holzschuher and René Peinl. Performance of Graph Query Languages: Comparison of Cypher, Gremlin and Native Access in Neo4J. In *Proceedings of the Joint EDBT/ICDT 2013 Workshops*, EDBT '13, pages 195–204, New York, NY, USA, 2013. ACM. URL: http://doi.acm.org/10.1145/2457317.2457351, doi:10.1145/2457317.2457351.

101 Paul Horn and Mats Rydberg. openCypher - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/opencypher/.

102 Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, et al. Tidb: a raft-based htap database. *Proceedings of the VLDB Endowment*, 13(12):3072–3084, 2020.

103 Jiewen Huang, Daniel J Abadi, and Kun Ren. Scalable SPARQL querying of large RDF graphs. *Proceedings of the VLDB Endowment*, 4(11):1123–1134, 2011.

104 Mark Hwang. Graph Processing Using SAP HANA: A Teaching Case. *e-Journal of Business Education and Scholarship of Teaching*, 12(2):155–165, 2018.

105 IBM System G Team. The World is Big and Linked: Whole Spectrum Industry Solutions towards Big Graphs - Graph Computing and Tutorial of IBM System G. [Online; accessed 17-December-2025], Oct 2015. URL: `https://cci.drexel.edu/bigdata/bigdata2015/Compressed-Tutorial_IEEE-BigData-2015-IBM-SystemG-20151031_v3.compressed.pdf`.

106 Filip Ilievski, Daniel Garijo, Hans Chalupsky, Naren Teja Divvala, Yixiang Yao, Craig Milo Rogers, Ronpeng Li, Jun Liu, Amandeep Singh, Daniel Schwabe, and Pedro A. Szekely. KGTK: A toolkit for large knowledge graph manipulation and analysis. In Jeff Z. Pan, Valentina Tamma, Claudia d'Amato, Krzysztof Janowicz, Bo Fu, Axel Polleres, Oshani Seneviratne, and Lalana Kagal, editors, *The Semantic Web - ISWC 2020 - 19th International Semantic Web Conference, Athens, Greece, November 2-6, 2020, Proceedings, Part II*, volume 12507 of *Lecture Notes in Computer Science*, pages 278–293. Springer, 2020. `doi:10.1007/978-3-030-62466-8\_18`.

107 Borislav Iordanov. HyperGraphDB: a generalized graph database. In *International Conference on Web-age Information Management*, pages 25–36. Springer, 2010.

108 Borislav Iordanov. Distributed HypergraphDB: partial replication. [Online; accessed 17-December-2025], 2020. URL: `https://groups.google.com/d/msg/hypergraphdb/ef8eWRWjc2E/0GyGcEsEDwAJ`.

109 ISO. ISO/IEC 9075-16:2023 Information technology — Database languages SQL - Part 16: Property Graph Queries (SQL/PGQ). https://www.iso.org/standard/79473.html, June 2023.

110 ISO. ISO/IEC 39075:2024 Information technology — Database languages — GQL. https://www.iso.org/standard/76120.html, April 2024.

111 JanusGraph Authors. JanusGraph: Distributed Graph Database. [Online; accessed 17-December-2025], 2017. URL: `https://janusgraph.org/`.

112 Jena Team. Jena TDB, 2021. URL: `https://jena.apache.org/documentation/tdb/`.

113 Changjiu Jin, Sourav S Bhowmick, Xiaokui Xiao, James Cheng, and Byron Choi. GBLENDER: towards blending visual query formulation and query processing in graph databases. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 111–122, 2010.

114 Guodong Jin, Xiyang Feng, Ziyi Chen, Chang Liu, and Semih Salihoglu. Kùzu graph database management system. In *13th Conference on Innovative Data Systems Research, CIDR 2023, Amsterdam, The Netherlands, January 8-11, 2023*. www.cidrdb.org, 2023. URL: `https://www.cidrdb.org/cidr2023/papers/p48-jin.pdf`.

115 Martin Junghanns, Max Kießling, Alex Averbuch, André Petermann, and Erhard Rahm. Cypher-based Graph Pattern Matching in Gradoop. In Peter A. Boncz and Josep-Lluís Larriba-Pey, editors, *Proceedings of the Fifth International Workshop on Graph Data-management Experiences & Systems, GRADES@SIGMOD/PODS 2017, Chicago, IL, USA, May 14 - 19, 2017*, pages 3:1–3:8. ACM, 2017. `doi:10.1145/3078447.3078450`.

116 Martin Junghanns, Max Kießling, Niklas Teichmann, Kevin Gómez, André Petermann, and Erhard Rahm. Declarative and distributed graph analytics with GRADOOP. *PVLDB*, 11(12):2006–2009, 2018. URL: `http://www.vldb.org/pvldb/vol11/p2006-junghanns.pdf`, `doi:10.14778/3229863.3236246`.

117 Rohit Kumar Kaliyar. Graph databases: A survey. *International Conference on Computing, Communication & Automation*, 2015. `doi:10.1109/CCAA.2015.7148480`.

118 Nikos Kanakaris, Dimitrios Michail, and Iraklis Varlamis. *A Comparative Survey of Graph Databases and Software for Social Network Analytics: The Link Prediction Perspective*, page 20. CRC Press, 1st edition, 2023.

119 Chathura Kankanamge, Siddhartha Sahu, Amine Mhedbhi, Jeremy Chen, and Semih Salihoglu. Graphflow: An active graph database. In *Proceedings of the 2017 ACM International Conference on Management of Data*, pages 1695–1698, 2017.

120 Kankanamge, Chathura and Sahu, Siddhartha and Mhedbhi, Amine and Chen, Jeremy and Salihoglu, Semih. Graphflow - Source Code. [Online, GitHub; accessed 17-December-2025], 2017. URL: `https://github.com/colinsongf/graphflow`.

121 Katana Graph, Inc. KatanaGraph - Source Code. [Online; accessed 17-December-2025], 2020. URL: `https://github.com/KatanaGraph/katana-releases`.

122 Kensho Technologies, LLC. OrientDB GraphQL to Gremlin - Source Code. [Online, GitHub; accessed 17-December-2025], April 2020.

123 Anurag Khandelwal, Zongheng Yang, Evan Ye, Rachit Agarwal, and Ion Stoica. ZipG - Source Code. [Online, GitHub; accessed 17-December-2025], 2017. URL: `https://github.com/amplab/zipg`.

124 Anurag Khandelwal, Zongheng Yang, Evan Ye, Rachit Agarwal, and Ion Stoica. Zipg: A memory-efficient graph store for interactive queries. In *Proceedings of the 2017 ACM International Conference on Management of Data*, pages 1149–1164, 2017.

125 Tariq N. Khasawneh, Mahmoud H. AL-Sahlee, and Ali A. Safia. Sql, newsql, and nosql databases: A comparative survey. In *2020 11th International Conference on Information and Communication Systems (ICICS)*, pages 013–021, 2020. `doi:10.1109/ICICS49469.2020.239513`.

126 Vojtěch Kolomičenko, Martin Svoboda, and Irena Holubová Mlỳnková. Experimental comparison of graph databases. In *Proceedings of International Conference on Information Integration and Web-based Applications & Services*, pages 115–124, 2013.

127 Georgia Koloniari and E. Pitoura. Transaction management for cloud-based graph databases. *Lecture Notes in Computer Science*, pages 99–113, 2015. doi:10.1007/978-3-319-29919-8_8.

128 KuzuDB Team. Kuzu - source code. [Online, GitHub; accessed 17 December 2025], 2025. URL: https://github.com/kuzudb/kuzu.

129 Avinash Lakshman and Prashant Malik. Cassandra: A Decentralized Structured Storage System. *SIGOPS Oper. Syst. Rev.*, 44(2):35–40, April 2010. URL: http://doi.acm.org/10.1145/1773912.1773922, doi:10.1145/1773912.1773922.

130 Ora Lassila, Michael Schmidt, Olaf Hartig, Brad Bebee, Dave Bechberger, Willem Broekema, Ankesh Khandelwal, Kelvin Lawrence, Carlos Manuel López Enríquez, Ronak Sharda, and Bryan B. Thompson. The onegraph vision: Challenges of breaking the graph model lock-in. *Semantic Web*, 14(1):125–134, 2023. doi:10.3233/SW-223273.

131 Changji Li, Hongzhi Chen, Shuai Zhang, Yingqian Hu, Chao Chen, Zhenjie Zhang, Meng Li, Xiangchen Li, Dongqing Han, Xiaohui Chen, et al. ByteGraph - Query Set. [Online, GitHub; accessed 17-December-2025], 2022. URL: https://github.com/Aaronchangji/ByteGraph-Paper-Query-Set.

132 Changji Li, Hongzhi Chen, Shuai Zhang, Yingqian Hu, Chao Chen, Zhenjie Zhang, Meng Li, Xiangchen Li, Dongqing Han, Xiaohui Chen, et al. Bytegraph: a high-performance distributed graph database in bytedance. *Proceedings of the VLDB Endowment*, 15(12):3306–3318, 2022.

133 Jermy Li et al. Apache HugeGraph - Source Code. [Online, GitHub; accessed 17-December-2025], 2023. URL: https://github.com/apache/incubator-hugegraph.

134 Leonid Libkin, Wim Martens, and Domagoj Vrgoc. Querying graphs with data. *J. ACM*, 63(2):14:1–14:53, 2016. doi:10.1145/2850413.

135 Linked Data Benchmark Council. Existing Languages, 2022. URL: https://www.gqlstandards.org/existing-languages.

136 Matteo Lissandrini, Martin Brugnara, and Yannis Velegrakis. Beyond macrobenchmarks: microbenchmark-based graph database evaluation. *Proceedings of the VLDB Endowment*, 12(4):390–403, 2018.

137 Matteo Lissandrini, Davide Mottin, Katja Hose, and Torben Bach Pedersen. Knowledge graph exploration systems: Are we lost? In *Annual Conference on Innovative Data Systems Research*, 2022.

138 Jaime I. Lopez-Veyna, Ivan Castillo-Zuñiga, and Mariana Ortiz-Garcia. A review of graph databases. In *Lecture Notes in Networks and Systems*, volume 576 of *LNNS*. Springer, 2022. First Online: 30 October 2022.

139 Arcade Data Ltd. ArcadeDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2023. URL: https://github.com/ArcadeData/arcadedb.

140 Memgraph Ltd. Memgraph - Source Code. [Online, GitHub; accessed 17-December-2025], 2022. URL: https://github.com/memgraph/memgraph.

141 Redis Labs Ltd. RedisGraph - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/RedisGraph/RedisGraph.

142 William Lyon. GRANDstack. [Online; accessed 17-December-2025], 2020. URL: https://grandstack.io/.

143 Valerio Malenchino. GQL is Here: Your Cypher Queries in a GQL World, 2024. URL: https://neo4j.com/blog/cypher-and-gql/cypher-gql-world/.

144 MariaDB. Open Query GRAPH computation engine. [Online; accessed 17-December-2025], 2016. URL: https://mariadb.com/kb/en/oqgraph-storage-engine/.

145 Norbert Martinez-Bazan, Sergio Gomez-Villamor, and Francesc Escale-Claveras. DEX: A high-performance graph database management system. In *2011 IEEE 27th International Conference on Data Engineering Workshops*, pages 124–127. IEEE, 2011.

146 Miguel A Martínez-Prieto, Mario Arias Gallego, and Javier D Fernández. Exchange and consumption of huge RDF data. In *Extended Semantic Web Conference*, pages 437–452. Springer, 2012.

147 R. McColl, David Ediger, Jason A. Poovey, D. Campbell, and David A. Bader. A performance evaluation of open source graph databases. *Proceedings of the First International Workshop on Graph Data Management Experiences and Systems*, pages 11–18, 2014. doi:10.1145/2567634.2567638.

148 Scott M. Meyer, Jutta Degener, John Giannandrea, and Barak Michener. Optimizing Schema-last Tuple-store Queries in Graphd. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, SIGMOD '10, pages 1047–1056, New York, NY, USA, 2010. ACM. URL: http://doi.acm.org/10.1145/1807167.1807283, doi:10.1145/1807167.1807283.

149 Amine Mhedhbi, Pranjal Gupta, Shahid Khaliq, and Semih Salihoglu. A+ Indexes: Lightweight and Highly Flexible Adjacency Lists for Graph Database Management Systems. arXiv preprint arXiv:2004.00130, 2020.

150 Amine Mhedhbi, Matteo Lissandrini, Laurens Kuiper, Jack Waudby, and Gábor Szárnyas. LSQB: a large-scale subgraph query benchmark. In *Proceedings of the 4th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 1–11, 2021.

151 Amine Mhedhbi and Semih Salihoglu. Optimizing Subgraph Queries by Combining Binary and Worst-Case Optimal Joins. *Proc. VLDB Endow.*, 12(11):1692–1704, July 2019. doi:10.14778/3342263.3342643.

152 Microsoft Corporation. Azure Cosmos DB - Source Code. [Online, GitHub; accessed 17-December-2025], 2019. URL: https://github.com/Azure/cosmos.

153 Microsoft Fabric. GQL language guide, 2025. URL: https://learn.microsoft.com/en-us/fabric/graph/gql-language-guide.

154 Justin J Miller. Graph database applications and concepts with neo4j. *Proceedings of the Southern Association for Information Systems Conference, Atlanta, GA, USA*, 2324(36), 2013. URL: https://pdfs.semanticscholar.org/322a/6e1f464330751dea2eb6beecac24466322ad.pdf.

155 Vahab Mirrokni, Mikkel Thorup, and Morteza Zadimoghaddam. Consistent hashing with bounded loads. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 587–604. SIAM, 2018.

156 Raju Kumar Mishra and Sundar Rajan Raman. GraphFrames. In *PySpark SQL Recipes*, pages 297–315. Springer, 2019. URL: https://link.springer.com/chapter/10.1007/978-1-4842-4335-0_9.

157 Jéssica Monteiro, Filipe Sá, and Jorge Bernardino. Experimental evaluation of graph databases: Janusgraph, nebula graph, neo4j, and tigergraph. *Applied Sciences*, 13(9), 2023. URL: https://www.mdpi.com/2076-3417/13/9/5770, doi:10.3390/app13095770.

158 Bruno Morais, Miguel E. Coimbra, and Luís Veiga. Pk-graph: Partitioned k2-trees to enable compact and dynamic graphs in sparkgraphx. In *Cooperative Information Systems: 28th International Conference, CoopIS 2022, Bozen-Bolzano, Italy, October 4–7, 2022, Proceedings*, page 149–167, Berlin, Heidelberg, 2022. Springer-Verlag. doi:10.1007/978-3-031-17834-4_9.

159 J. Ian Munro, Yakov Nekrich, and Jeffrey Scott Vitter. Dynamic Data Structures for Document Collections and Graphs. In *ACM Symposium on Principles of Database Systems (PODS)*, pages 277–289, 2015.

160 Nebula Graph. Announcing NebulaGraph Enterprise v5.0 RC release: Delivering Full-Fledged GQL Support, 2024. URL: https://www.nebula-graph.io/posts/rc_release_of_nebulagraph_enterprise_v5.0.

161 Neo4j, Inc. What is openCypher? [Online; accessed 17-December-2025], 2018. URL: http://www.opencypher.org/.

162 Neo4j, Inc. Cypher for Gremlin - Source Code. [Online, GitHub; accessed 17-December-2025], 2019. URL: https://github.com/opencypher/cypher-for-gremlin.

163 Neo4j Inc. Neo4j - Source Code. [Online; accessed 17-December-2025], 2020. URL: https://github.com/neo4j/neo4j.

164 Neo4j, Inc. Neo4j and GraphQL. [Online; accessed 17-December-2025], 2020. URL: https://neo4j.com/developer/graphql/.

165 Neo4j Inc. Neo4j APOC Library - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/neo4j-contrib/neo4j-apoc-procedures.

166 Neo4j, Inc. Neo4j GraphQL to Cypher - Source Code. [Online, GitHub; accessed 17-December-2025], April 2020.

167 Neo4j, Inc. What is a Graph Database? [Online; accessed 17-December-2025], 2020.

168 Thomas Neumann and Gerhard Weikum. RDF-3X: a RISC-style engine for RDF. *Proc. VLDB Endow.*, 1(1):647–659, 2008. URL: http://www.vldb.org/pvldb/vol1/1453927.pdf, doi:10.14778/1453856.1453927.

169 Objectivity. Objectivity/DB. [Online; accessed 17-December-2025], 2016. URL: https://www.objectivity.com/products/objectivitydb/.

170 Ontotext. GraphDB | The Best RDF Database for Knowledge Graphs. [Online; accessed 17-December-2025], 2020. URL: https://www.ontotext.com/products/graphdb/#GraphDB-table.

171 OpenLink Software. Virtuoso - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/openlink/virtuoso-opensource.

172 Optiver. Ruruki - In-Memory Directed Property Graph. [Online; accessed 17-December-2025], 2016. URL: https://ruruki.readthedocs.io/en/master/readme.html#introduction-to-ruruki-in-memory-directed-property-grap.

173 Oracle. Spatial and Graph features in Oracle Database. [Online; accessed 17-December-2025], 2020. URL: https://www.oracle.com/database/technologies/spatialandgraph.html.

174 OrientDB LTD. OrientDB - Source Code. [Online; accessed 17-December-2025], 2020. URL: https://github.com/orientechnologies/orientdb.

175 Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. The log-structured merge-tree (lsm-tree). *Acta Informatica*, 33:351–385, 1996.

176 Jan Paredaens, Peter Peelman, and Letizia Tanca. G-Log: A graph-based query language. *IEEE Transactions on Knowledge and Data Engineering*, 7(3):436–453, 1995.

177 Peter F. Patel-Schneider. Benchmarking SPARQL Engines on Wikidata Queries. In *Wikidata Workshop at ISWC 2025*, pages 1–13, Nara, Japan, 2025. Wikidata Workshop at ISWC 2025. URL: https://ceur-ws.org/Vol-4108/paper3.pdf.

178 N. S. Patil, P. Kiran, N. P. Kiran, and K. Patel. A survey on graph database management techniques for huge unstructured data. *International Journal of Electrical and Computer Engineering*, 8:1140–1149, 2018. doi:10.11591/IJECE.V8I2.PP1140-1149.

179 José Rolando Guay Paz. Introduction to Azure Cosmos DB. In *Microsoft Azure Cosmos DB Revealed*, pages 1–23. Springer, 2018.

180 Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. Semantics and complexity of SPARQL. *ACM Transactions on Database Systems (TODS)*, 34(3):1–45, 2009.

181 C.L. Philip Chen and Chun-Yang Zhang. Data-intensive applications, challenges, techniques and technologies: A survey on big data. *Information Sciences*, 275:314–347, 2014. URL: https://www.sciencedirect.com/science/article/pii/S0020025514000346, doi:10.1016/j.ins.2014.01.015.

182 Kasper Piskorski, Felix Chapman, Filipe Teixeira, Joshua Send, and Ganeshwara others Hananda. TypeDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2023. URL: https://github.com/vaticle/typedb.

183 Stefan Plantikow. Towards an International Standard for the GQL Graph Query Language. [Online; accessed 17-December-2025], March 2019. URL: https://www.w3.org/Data/events/data-ws-2019/assets/position/Stefan%20Plantikow.pdf.

184 Jaroslav Pokorný. Graph databases: Their power and limitations. In Khalid Saeed and Wladyslaw Homenda, editors, *Computer Information Systems and Industrial Management*, pages 58–69, Cham, 2015. Springer International Publishing.

185 Jaroslav Pokorný, Michal Valenta, and Jiří Kovačič. Integrity constraints in graph databases. *Procedia Computer Science*, 109:975–981, 2017. 8th International Conference on Ambient Systems, Networks and Technologies, ANT-2017 and the 7th International Conference on Sustainable Energy Information Technology, SEIT 2017, 16-19 May 2017, Madeira, Portugal. URL: https://www.sciencedirect.com/science/article/pii/S1877050917311390, doi:10.1016/j.procs.2017.05.456.

186 J. M. Pujol, Vijay Erramilli, Georgos Siganos, Xiaoyuan Yang, Nikolaos Laoutaris, Parminder Chhabra, and P. Rodriguez. The little engine(s) that could: Scaling online social networks. *IEEE/ACM Transactions on Networking*, 20:1162–1175, 2010. doi:10.1145/1851182.1851227.

187 Juan L. Reutter, Miguel Romero, and Moshe Y. Vardi. Regular queries on graph databases. *Theory Comput. Syst.*, 61(1):31–83, 2017. URL: https://doi.org/10.1007/s00224-016-9676-2, doi:10.1007/S00224-016-9676-2.

188 Pedro Ribeiro, Pedro Paredes, Miguel E. P. Silva, David Aparicio, and Fernando Silva. A survey on subgraph counting: Concepts, algorithms, and applications to network motifs and graphlets. *ACM Comput. Surv.*, 54(2), March 2021. doi:10.1145/3433652.

189 Christopher D Rickett, Utz-Uwe Haus, James Maltby, and Kristyn J Maschhoff. Loading and Querying a Trillion RDF triples with Cray Graph Engine on the Cray XC. *Cray User Group*, 2018.

190 Ritter, Daniel and Dell'Aquila, Luigi and Lomakin, Andrii and Tagliaferri, Emanuele. OrientDB: A NoSQL, Open Source MMDMS. In *Proceedings of the The British International Conference on Databases 2021, London, United Kingdom, March 28, 2022*, volume 3163 of *CEUR Workshop Proceedings*, pages 10–19. CEUR-WS.org, 2021.

191 Ian Robinson, Jim Webber, and Emil Eifrem. *Graph databases: new opportunities for connected data*. O'Reilly Media, Inc., 2015.

192 Marko A Rodriguez. The gremlin graph traversal machine and language (invited talk). In *Proceedings of the 15th Symposium on Database Programming Languages*, pages 1–10, 2015. URL: https://dl.acm.org/doi/abs/10.1145/2815072.2815073.

193 Chandrima Roy, M. Pandey, and S. Rautaray. A Proposal for Optimization of Horizontal Scaling in Big Data Environment. *In book: Advances in Data and Information Sciences*, pages 223–230, 2018. doi:10.1007/978-981-10-8360-0_21.

194 Michael Rudolf, Marcus Paradies, Christof Bornhövd, and Wolfgang Lehner. The graph story of the SAP HANA database. *Datenbanksysteme für Business, Technologie und Web (BTW) 2037*, 2013.

195 Tomer Sagi, Matteo Lissandrini, Torben Bach Pedersen, and Katja Hose. A design space for RDF data representations. *The VLDB journal*, 31(2):347–373, 2022.

196 Sherif Sakr, Angela Bonifati, Hannes Voigt, Alexandru Iosup, Khaled Ammar, Renzo Angles, Walid Aref, Marcelo Arenas, Maciej Besta, Peter A. Boncz, Khuzaima Daudjee, Emanuele Della Valle, Stefania Dumbrava, Olaf Hartig, Bernhard Haslhofer, Tim Hegeman, Jan Hidders, Katja Hose, Adriana Iamnitchi, Vasiliki Kalavri, Hugo Kapp, Wim Martens, M. Tamer Özsu, Eric Peukert, Stefan Plantikow, Mohamed Ragab, Matei R. Ripeanu, Semih Salihoglu, Christian Schulz, Petra Selmer, Juan F. Sequeda, Joshua Shinavier, Gábor Szárnyas, Riccardo Tommasini, Antonino Tumeo, Alexandru Uta, Ana Lucia Varbanescu, Hsiang-Yun Wu, Nikolay Yakovets, Da Yan, and Eiko Yoneki. The future is big graphs: A community view on graph processing systems. *Commun. ACM*, 64(9):62–71, August 2021. doi:10.1145/3434642.

197 SAP SE. SAP HANA Graph Academy - Source Code. [Online, GitHub; accessed 17-December-2025], 2018. URL: https://github.com/saphanaacademy/Graph.

198 Alexander Schätzle, Martin Przyjaciel-Zablocki, Thorsten Berberich, and Georg Lausen. S2X: graph-parallel querying of RDF with GraphX. In *Biomedical Data Management and Graph Online Querying*, pages 155–168. Springer, 2015.

199 Thomas Schmidts, Jan Steemann, and Frank Celler. ArangoDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/arangodb/arangodb.

200 Alon Shalita, Brian Karrer, Igor Kabiljo, Arun Sharma, Alessandro Presta, Aaron Adcock, Herald Kllapi, and Michael Stumm. Social hash: an assignment framework for optimizing distributed systems operations on social networks. In *13th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 16)*, pages 455–468, 2016.

201 Zhihong Shen, Zihao Zhao, Mingjie Tang, Chuan Hu, Huajin Wang, and Yuanchun Zhou. Pandadb: Understanding unstructured data in graph database. *arXiv preprint arXiv:2107.01963*, 2021.

202 Zhihong Shen, Zihao Zhao, Mingjie Tang, Chuan Hu, Huajin Wang, and Yuanchun Zhou. PandaDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2022. URL: https://github.com/grapheco/pandadb-v0.3.

203 Darshana Shimpi and Sangita Chaudhari. An overview of graph databases. In *Int. Conf. on Recent Trends in Information Technology and Computer Science*, pages 16–22, 2012.

204 Sachin Sinha et al. BangDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2023. URL: https://github.com/sachin-sinha/BangDB.

205 SKAI Worldwide Co., Ltd. AgensGraph. [Online, Homepage; accessed 17-December-2025], 2023. URL: https://www.skaiworldwide.com.

206 SKAI Worldwide Co., Ltd. AgensGraph - Source Code. [Online, Homepage; accessed 17-December-2025], 2023. URL: https://github.com/skaiworldwide-oss/agensgraph.

207 Sparsity Technologies. Scalable high-performance graph database. [Online, GitHub; accessed 17-December-2025], 2015. URL: http://www.sparsity-technologies.com/.

208 Stardog. Enterprise Knowledge Graph platform. [Online; accessed 17-December-2025], 2020. URL: https://www.stardog.com/platform/.

209 StellarDB. StellarDB. [Online, Homepage; accessed 17-December-2025], 2023. URL: https://www.transwarp.cn/en/product/stellardb.

210 Wen Sun, Achille Fokoue, Kavitha Srinivas, Anastasios Kementsietsidis, Gang Hu, and Guotong Xie. Sqlgraph: An efficient relational-based property graph store. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 1887–1901, 2015.

211 Systap. Blazegraph - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/blazegraph/database.

212 Systap. Blazegraph High Performance Graph Database. [Online; accessed 17-December-2025], 2020. URL: https://blazegraph.com/.

213 Yuanyuan Tian. The world of graph databases from an industry perspective. *SIGMOD Rec.*, 51(4):60–67, jan 2023. doi:10.1145/3582302.3582320.

214 TigerGraph. The World's Fastest and Most Scalable Graph Platform. [Online; accessed 17-December-2025], 2020. URL: https://www.tigergraph.com/.

215 Bing Tong, Yan Zhou, Chen Zhang, Jianheng Tang, Jing Tang, Leihong Yang, Qiye Li, Manwu Lin, Zhongxin Bao, Jia Li, et al. Galaxybase: A high performance native distributed graph database for htap. *Proceedings of the VLDB Endowment*, 17(12):3893–3905, 2024.

216 TuGraph. Alibaba Graph Database / TuGraph - Source Code. [Online; accessed 17-December-2025], 2022. URL: https://github.com/TuGraph-family/tugraph-db.

217 Ultipa Inc. Ultipa Graph Database. [Online, Homepage; accessed 17-December-2025], 2023. URL: https://www.ultipa.com/product/ultipa-graph.

218 Aparna Vaikuntam and Vinodh Kumar Perumal. Evaluation of contemporary graph databases. In *Proceedings of the 7th ACM India Computing Conference*, COMPUTE '14, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2675744.2675752.

219 Matthijs van Otterdijk, Gavin Mendel-Gleason, and Kevin Feeney. Succinct data structures and delta encoding for modern databases. 2020.

220 Matthijs van Otterdijk, Gavin Mendel-Gleason, and Kevin Feeney. TerminusDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2021. URL: https://github.com/terminusdb/terminusdb.

221 Oskar van Rest. Property Graphs in Oracle Database 23ai: The SQL/PGQ Standard, 2025. URL: https://blogs.oracle.com/database/property-graphs-in-oracle-database-23ai-the-sql-pgq-standard.

222 Oskar van Rest, Sungpack Hong, Jinha Kim, Xuming Meng, and Hassan Chafi. Pgql: a property graph query language. In *Proceedings of the Fourth International Workshop on Graph Data Management Experiences and Systems*, pages 1–6, 2016. URL: https://dl.acm.org/doi/abs/10.1145/2960414.2960421.

223 Todd L. Veldhuizen. Triejoin: A simple, worst-case optimal join algorithm. In Nicole Schweikardt, Vassilis Christophides, and Vincent Leroy, editors, *Proc. 17th International Conference on Database Theory (ICDT), Athens, Greece, March 24-28, 2014*, pages 96–106. OpenProceedings.org, 2014. URL: https://doi.org/10.5441/002/icdt.2014.13, doi:10.5441/002/ICDT.2014.13.

224 VESoft Inc. NebulaGraph - Source Code. [Online; accessed 17-December-2025], 2020. URL: https://github.com/vesoft-inc/nebula.

225 Deepak Vohra. Apache parquet. In *Practical Hadoop Ecosystem*, pages 325–335. Springer, 2016.

226 Domagoj Vrgoc, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. Millenniumdb: An open-source graph database system. *Data Intell.*, 5(3):560–610, 2023. URL: https://doi.org/10.1162/dint_a_00229, doi:10.1162/DINT_A_00229.

227 Domagoj Vrgoc, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. MillenniumDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2025. URL: https://github.com/MillenniumDB/MillenniumDB.

228 Daniel Walke, Daniel Micheel, Kay Schallert, Thilo Muth, David Broneske, Gunter Saake, and Robert Heyer. The importance of graph databases and graph learning for clinical applications. *Database*, 2024:baad045, 07 2023. arXiv:https://academic.oup.com/database/article-pdf/doi/10.1093/database/baad045/56714746/baad045.pdf, doi:10.1093/database/baad045.

229 Jim Webber. A Programmatic Introduction to Neo4J. In *Proceedings of the 3rd Annual Conference on Systems, Programming, and Applications: Software for Humanity*, SPLASH '12, pages 217–218, New York, NY, USA, 2012. ACM. URL: http://doi.acm.org/10.1145/2384716.2384777, doi:10.1145/2384716.2384777.

230 Ian H Witten, Alistair Moffat, Timothy C Bell, Timothy C Bell, Ed Fox, and Timothy C Bell. *Managing gigabytes: compressing and indexing documents and images*. Morgan Kaufmann, 1999.

231 Daniel ten Wolde, Gábor Szárnyas, and Peter Boncz. Duckpgq: Bringing sql/pgq to duckdb. *Proc. VLDB Endow.*, 16(12):4034–4037, August 2023. doi:10.14778/3611540.3611614.

232 Robert Yokota, Misha Brukman, Huangya Feng, Zac Rosenbauer, et al. HGraphDB - Source Code. [Online, GitHub; accessed 17-December-2025], 2023. URL: https://github.com/rayokota/hgraphdb.

233 Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster Computing with Working Sets. In *Proceedings of the 2Nd USENIX Conference on Hot Topics in Cloud Computing*, HotCloud'10, pages 10–10, Berkeley, CA, USA, 2010. USENIX Association. URL: http://dl.acm.org/citation.cfm?id=1863103.1863113.

234 Erfan Zamanian, Carsten Binnig, Tim Kraska, and Tim Harris. The end of a myth: Dis-

tributed transactions can scale. *arXiv preprint arXiv:1607.00655*, 2016.

235 Chao Zhang, Angela Bonifati, and M. Tamer Özsu. An overview of reachability indexes on graphs. In *Companion of the 2023 International Conference on Management of Data*, SIGMOD '23, page 61–68, New York, NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3555041.3589408.

236 Chao Zhang, Angela Bonifati, and M. Tamer Özsu. Indexing techniques for graph reachability queries, 2023. arXiv:2311.03542.

237 Xiaowei Zhu, Guanyu Feng, Marco Serafini, Xiaosong Ma, Jiping Yu, Lei Xie, Ashraf Aboulnaga, and Wenguang Chen. Livegraph: A transactional graph storage system with purely sequential adjacency list scans. *arXiv preprint arXiv:1910.05773*, 2019.

238 Xiaowei Zhu, Guanyu Feng, Marco Serafini, Xiaosong Ma, Jiping Yu, Lei Xie, Ashraf Aboulnaga, and Wenguang Chen. LiveGraph - Source Code. [Online, GitHub; accessed 17-December-2025], 2020. URL: https://github.com/thu-pacman/LiveGraph.

239 Justin Zobel and Alistair Moffat. Inverted files for text search engines. *ACM computing surveys (CSUR)*, 38(2):6–es, 2006.